

Raport z projektu

Przewidywanie satysfakcji pasażerów linii lotniczej

Airline Passenger Satisfaction

Podstawy Uczenia Maszynowego

Projekt semestralny

Autorzy: Yevhen Koval (s30874), Maciej Bereda (s31616), Raman Yaraslautsau (s22349)

PJATK

2026

Spis treści

1.	Opis problemu	4
1.1.	Krótki opis problemu i jego kontekstu	4
1.2.	Kto i w jaki sposób może skorzystać z modelu?	4
1.3.	Dlaczego problem jest interesujący?	4
2.	Dane	5
2.1.	Źródło danych i ocena wiarygodności	5
2.2.	Zmienna celu	5
2.3.	Charakterystyka zmiennych	5
2.4.	Braki danych	6
2.5.	Krótką analizę opisową danych	6
3.	Eksploracyjna analiza danych	7
3.1.	Zmienne kategoryczne	7
3.2.	Zmienne liczbowe	8
3.3.	Kolumny ocenowe i interpretacja wartości 0	9
3.4.	Oceny usług a satysfakcja	10
3.5.	Korelacje	11
3.6.	Opóźnienia, outliers i train-test drift	13
3.7.	Segmenty pasażerów	15
4.	Sposób rozwiązania problemu	16
4.1.	Przygotowanie danych	16
4.2.	Modele	19
4.3.	Miary ewaluacji	20
5.	Wyniki modelowania	21
5.1.	Modele bazowe	21
5.2.	Walidacja najlepszego modelu bazowego	22
5.3.	Eksperyment z grupami cech	22
5.4.	Walidacja krzyżowa	24
5.5.	Krzywe uczenia modeli	25
5.6.	Strojenie hiperparametrów	26
6.	Ewaluacja końcowa	27
6.1.	Wynik na zbiorze testowym	27
6.2.	Interpretacja błędów	28
6.3.	Wyjaśnialność modelu	28
7.	Dyskusja wyników	30
7.1.	Co mówią wyniki?	30
7.2.	Czy opóźnienia są najważniejsze?	30
7.3.	Znaczenie <code>type_of_travel</code> i <code>customer_type</code>	30
7.4.	Ograniczenia projektu	30
8.	Podsumowanie	31
8.1.	Co się udało?	31
8.2.	Jakie były problemy i jak je rozwiązano?	31
8.3.	Jak można wykorzystać wyniki?	31
8.4.	Możliwe rozwinięcia	31

9.	Załączniki	32
9.1.	Załącznik 1. Pairplot wybranych cech	32
9.2.	Załącznik 2. Rozkład zmiennej celu	33
9.3.	Załącznik 3. Braki danych	34
9.4.	Załącznik 4. Liczebności kategorii	35
9.5.	Załącznik 5. Procent satysfakcji w kategoriach	36
9.6.	Załącznik 6. Rozkłady zmiennych liczbowych	37
9.7.	Załącznik 7. Boxploty zmiennych liczbowych	38
9.8.	Załącznik 8. Wartości 0 w ocenach usług	39
9.9.	Załącznik 9. Interpretacja wartości 0	40
9.10.	Załącznik 10. Rozkłady ocen usług	41
9.11.	Załącznik 11. Średnie oceny usług	42
9.12.	Załącznik 12. Macierz korelacji	43
9.13.	Załącznik 13. Korelacje ze zmienną celu	44
9.14.	Załącznik 14. Opóźnienie odlotu i przylotu	45
9.15.	Załącznik 15. Satysfakcja według opóźnienia	46
9.16.	Załącznik 16. Outliery zmiennych liczbowych	47
9.17.	Załącznik 17. Porównanie rozkładów train i test	48
9.18.	Załącznik 18. Porównanie celu train i test	49
9.19.	Załącznik 19. Satysfakcja w segmentach	50
9.20.	Załącznik 20. Porównanie modeli bazowych	51
9.21.	Załącznik 21. Walidacja najlepszego modelu	52
9.22.	Załącznik 22. Modele według grup cech	53
9.23.	Załącznik 23. Ewaluacja finalna na teście	54
9.24.	Załącznik 24. Permutation importance	55
9.25.	Załącznik 25. Krzywe uczenia - porównanie modeli	56
9.26.	Załącznik 26. Czas trenowania modeli	57
9.27.	Załącznik 27. Krzywa uczenia - Dummy majority	58
9.28.	Załącznik 28. Krzywa uczenia - Logistic regression	59
9.29.	Załącznik 29. Krzywa uczenia - Gaussian Naive Bayes	60
9.30.	Załącznik 30. Krzywa uczenia - Decision tree	61
9.31.	Załącznik 31. Krzywa uczenia - Random forest	62
9.32.	Załącznik 32. Krzywa uczenia - Extra trees	63
9.33.	Załącznik 33. Krzywa uczenia - Hist gradient boosting	64
10.	Bibliografia i źródła	65

1. Opis problemu

1.1. Krótki opis problemu i jego kontekstu

Projekt dotyczy problemu klasyfikacji satysfakcji pasażerów linii lotniczej. Celem jest przewidzenie, czy pasażer należy do klasy *satisfied*, czy do klasy *neutral or dissatisfied*. Jest to klasyczne zadanie uczenia nadzorowanego, ponieważ dla każdego rekordu w danych treningowych i testowych znana jest etykieta celu.

Kontekst biznesowy jest czytelny: linie lotnicze stale konkurują jakością doświadczenia pasażera, punktualnością, obsługą, wygodą podróży, sprawnością odprawy i ofertą usług w trakcie lotu. Satysfakcja pasażera wpływa na lojalność, prawdopodobieństwo ponownego zakupu, ocenę marki, rekomendacje oraz koszty obsługi reklamacji. Model predykcyjny może pomóc w zrozumieniu, które elementy podróży najmocniej wpływają na pozytywną ocenę klienta.

W projekcie nie chodzi wyłącznie o osiągnięcie wysokiej skuteczności klasyfikacji. Równie ważne jest zrozumienie danych, wykrycie problemów jakościowych, przygotowanie cech, porównanie wielu modeli i interpretacja tego, czego model się nauczył. Dlatego praca została podzielona na kilka etapów: eksploracyjna analiza danych, czyszczenie, inżynieria cech, modelowanie, walidacja, strojenie hiperparametrów, ewaluacja końcowa oraz wyjaśnialność.

1.2. Kto i w jaki sposób może skorzystać z modelu?

Z modelu mogą skorzystać przede wszystkim zespoły analityczne, marketingowe, operacyjne i jakościowe w liniach lotniczych. Model może wspierać:

- przewidywanie ryzyka niezadowolenia pasażera,
- identyfikację usług, które najbardziej wpływają na ocenę podróży,
- segmentację pasażerów według prawdopodobieństwa satysfakcji,
- priorytetyzację inwestycji w poprawę obsługi,
- monitorowanie jakości doświadczenia pasażera na podstawie ankiet i danych operacyjnych,
- przygotowywanie działań naprawczych dla segmentów o niskiej satysfakcji.

W praktyce model mógłby działać na dwa sposoby. Pierwszy wariant to model poankietowy, który korzysta z ocen usług i pozwala zrozumieć, dlaczego pasażer był zadowolony albo niezadowolony. Drugi wariant to model wcześniejszy, oparty tylko na danych obiektywnych, takich jak wiek, płeć, klasa, dystans lotu i opóźnienia. Taki model byłby słabszy, ale mógłby działać jeszcze zanim pasażer wypełni ankietę.

1.3. Dlaczego problem jest interesujący?

Problem jest interesujący, ponieważ łączy dane obiektywne z danymi samoopisowymi. W zbiorze występują cechy opisujące pasażera i lot, takie jak wiek, płeć, klasa biletu, dystans lotu i opóźnienia. Są też zmienne subiektywne, czyli oceny usług w skali 0-5. To rodzi ważne pytanie: czy satysfakcję można przewidzieć głównie na podstawie faktów o podróży, czy potrzebne są ankietowe oceny poszczególnych usług?

Drugim ciekawym elementem jest interpretacja wartości θ w kolumnach ocenowych. Komentarze do zbioru sugerują, że skala ocen wynosi 1-5, natomiast θ oznacza brak odpowiedzi. To istotne, ponieważ potraktowanie zera jako najgorszej oceny mogłoby wprowadzić błędne założenie porządkowe. W projekcie przyjęto więc, że θ w ocenach jest specjalnym kodem braku odpowiedzi. Dodano osobne flagi informujące o braku odpowiedzi, a same zera zamieniono na braki danych przed imputacją.

2. Dane

2.1. Źródło danych i ocena wiarygodności

W projekcie wykorzystano zbiór danych *Airline Passenger Satisfaction* dostępny na platformie Kaggle. Dane zostały dostarczone w dwóch plikach:

- `train.csv` - zbiór treningowy, 103 904 obserwacje i 25 kolumn,
- `test.csv` - zbiór testowy, 25 976 obserwacji i 25 kolumn.

Oba pliki zawierają zmienną celu `satisfaction`, dlatego `test.csv` można wykorzystać do finalnej ewaluacji jakości modelu. Jest to korzystne, bo pozwala oddzielić walidację w trakcie pracy od końcowej oceny na osobnym zbiorze.

Zbiór danych jest popularny i dobrze nadaje się do projektu klasyfikacyjnego. Jego wiarygodność należy jednak oceniać ostrożnie. Nie mamy pełnej dokumentacji sposobu zbierania ankiet, definicji `Customer Type` ani procesu modyfikacji oryginalnego zbioru. Z tego powodu część decyzji, zwłaszcza interpretacja wartości 0 jako braku odpowiedzi, została opisana jako założenie projektowe.

2.2. Zmienna celu

Zmienna celu ma dwie klasy:

- `neutral or dissatisfied` - pasażer neutralny lub niezadowolony,
- `satisfied` - pasażer zadowolony.

W modelowaniu klasy zakodowano jako:

- 0 dla `neutral or dissatisfied`,
- 1 dla `satisfied`.

Rozkład klas w zbiorze treningowym jest umiarkowanie niezbalansowany: około 56,67% obserwacji należy do klasy `neutral or dissatisfied`, a około 43,33% do klasy `satisfied`. Nie jest to skrajny problem, ale sam wskaźnik `accuracy` nie wystarczy do rzetelnej oceny modelu. Dlatego raportowane są również `precision`, `recall`, `F1` oraz `ROC-AUC`.

2.3. Charakterystyka zmiennych

W zbiorze występują trzy główne typy cech:

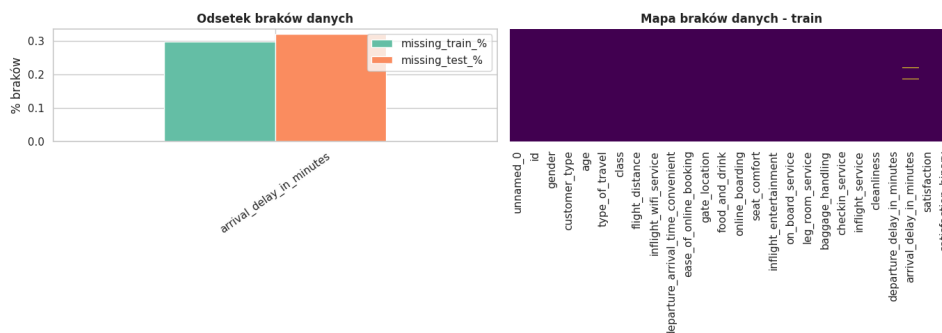
1. Cechy obiektywne: `Gender`, `Age`, `Class`, `Flight Distance`, `Departure Delay in Minutes`, `Arrival Delay in Minutes`.
2. Cechy kontekstowe: `Type of Travel`, `Customer Type`. Są one użyteczne, ale ich interpretacja wymaga ostrożności. Szczególnie `Customer Type` nie jest w pełni zdefiniowane w danych.
3. Cechy samoopisowe: oceny usług, m.in. `Inflight wifi service`, `Online boarding`, `Seat comfort`, `Inflight entertainment`, `Cleanliness`. Są to dane ankietowe, zależne od subiektywnej skali oceny pasażera.

Podział na te grupy został wykorzystany nie tylko w opisie danych, ale także w eksperymencie modelowym. Porównano modele pełne z modelami opartymi wyłącznie na cechach obiektywnych oraz z modelami używającymi tylko ocen usług.

2.4. Braki danych

Braki danych występują prawie wyłącznie w kolumnie `arrival_delay_in_minutes`. W zbiorze treningowym jest ich 310, a w testowym 83. Łącznie daje to 393 braki, czyli około 0,303% wszystkich rekordów. Skala problemu jest niewielka, a rozkład braków między train i test jest proporcjonalny do rozmiaru podziału.

W projekcie przyjęto, że braki w zmiennych liczbowych będą uzupełniane medianą w pipeline przetwarzania danych. Dodatkowo dla `arrival_delay_in_minutes` utworzono flagę `arrival_delay_missing`, aby model miał informację, że wartość była pierwotnie brakująca.



Rysunek 1: Analiza braków danych w zbiorach train i test.

2.5. Krótka analiza opisowa danych

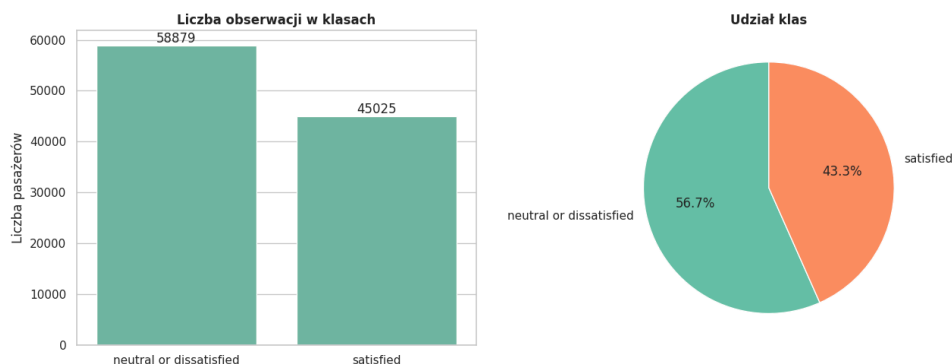
Analiza opisowa pokazała, że dane są duże i wystarczające do porównania wielu modeli. W zbiorze nie znaleziono duplikatów pełnych wierszy ani duplikatów identyfikatora `id`. Zmienne `id` oraz `Unnamed: 0` potraktowano jako techniczne identyfikatory i usunięto z predyktorów.

Zmienna celu jest umiarkowanie niezbalansowana, ale nie wymagała agresywnych metod balansowania klas. Zamiast tego zastosowano metryki odporne na nierówny rozkład klas oraz parametr `class_weight` tam, gdzie było to uzasadnione.

Najsilniej z satysfakcją skorelowane cechy liczbowe to:

- `online_boarding` z korelacją około 0,504,
- `inflight_entertainment` z korelacją około 0,398,
- `seat_comfort` z korelacją około 0,349,
- `on_board_service` z korelacją około 0,322,
- `leg_room_service` z korelacją około 0,313,
- `cleanliness` z korelacją około 0,305,
- `flight_distance` z korelacją około 0,299.

Opóźnienia mają korelacje ujemne, ale słabsze: `arrival_delay_in_minutes` około $-0,058$, a `departure_delay_in_minutes` około $-0,050$. Oznacza to, że opóźnienia są związane z satysfakcją, ale w tym zbiorze mniej niż oceny usług.



Rysunek 2: Rozkład zmiennej celu w zbiorze treningowym.

3. Eksploracyjna analiza danych

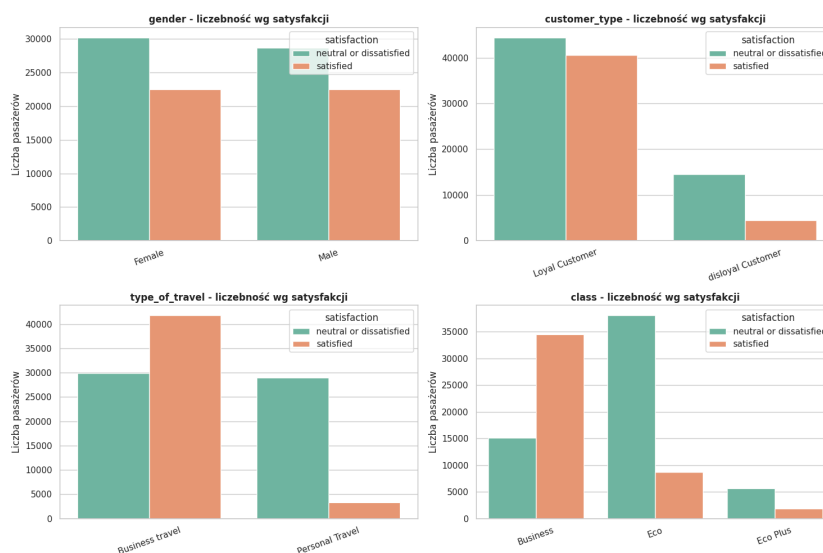
3.1. Zmienne kategoryczne

Analiza zmiennych kategorycznych ujawniła duże różnice między segmentami pasażerów. Największe różnice dotyczą celu podróży oraz klasy biletu. Pasażerowie podróżujący służbowo byli zdecydowanie częściej zadowoleni niż pasażerowie podróżujący prywatnie. Również klasa Business miała znacznie wyższy odsetek zadowolonych niż klasy Eco i Eco Plus.

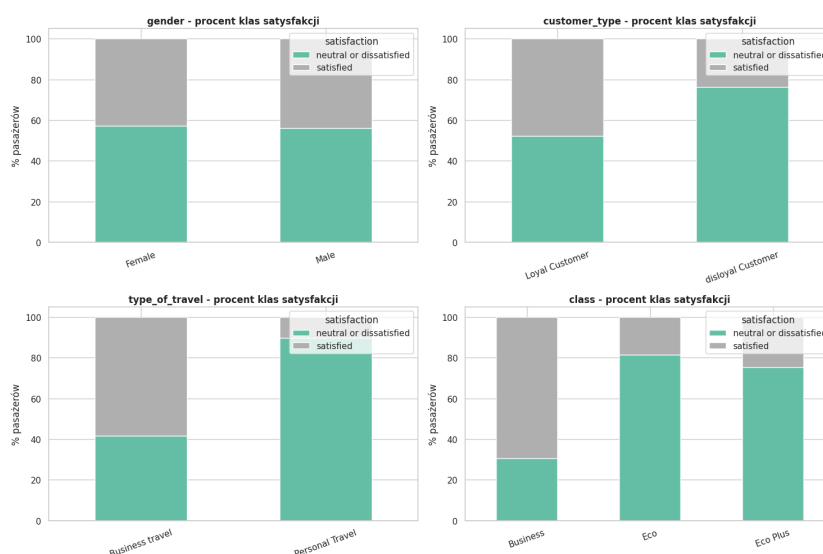
W zbiorze treningowym odsetek zadowolonych wynosił:

- około 58,26% dla Business travel,
- około 10,17% dla Personal Travel,
- około 69,43% dla klasy Business,
- około 18,61% dla klasy Eco,
- około 24,61% dla klasy Eco Plus.

Wyniki te są intuicyjne. Podróż służbowa i klasa biznesowa wiążą się z innymi warunkami podróży, wyższym standardem obsługi i prawdopodobnie innym profilem pasażera. Jednocześnie należy uważać, aby nie interpretować tych zależności jako przyczynowych. Model wykrywa zależności statystyczne, a nie bezpośrednie przyczyny.



Rysunek 3: Liczebności kategorii z podziałem na klasy satysfakcji.



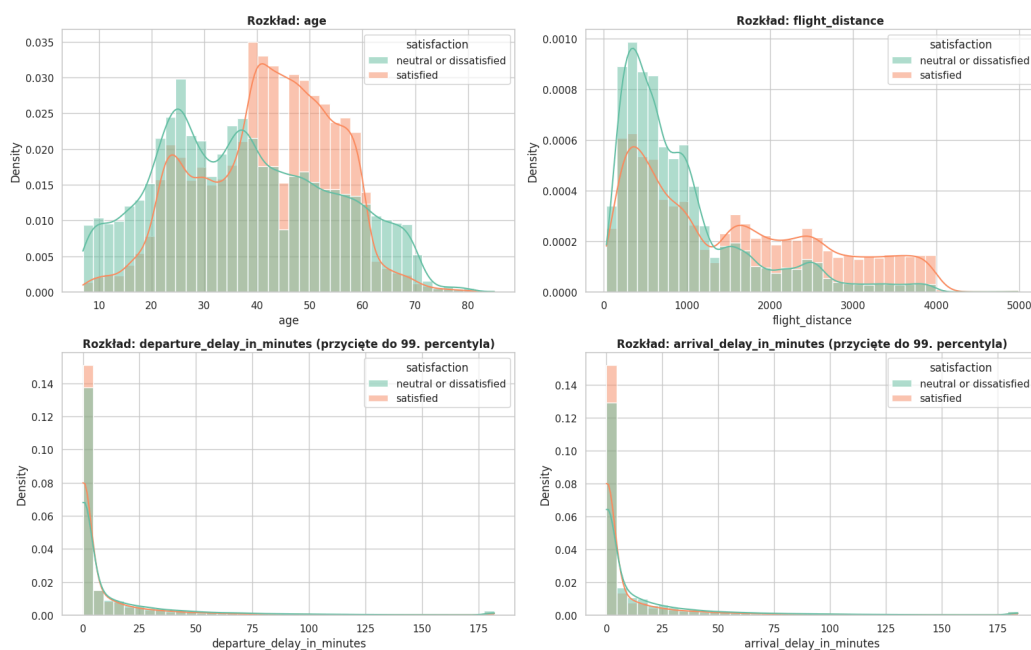
Rysunek 4: Procentowy udział klas satysfakcji w zmiennych kategorycznych.

3.2. Zmienne liczbowe

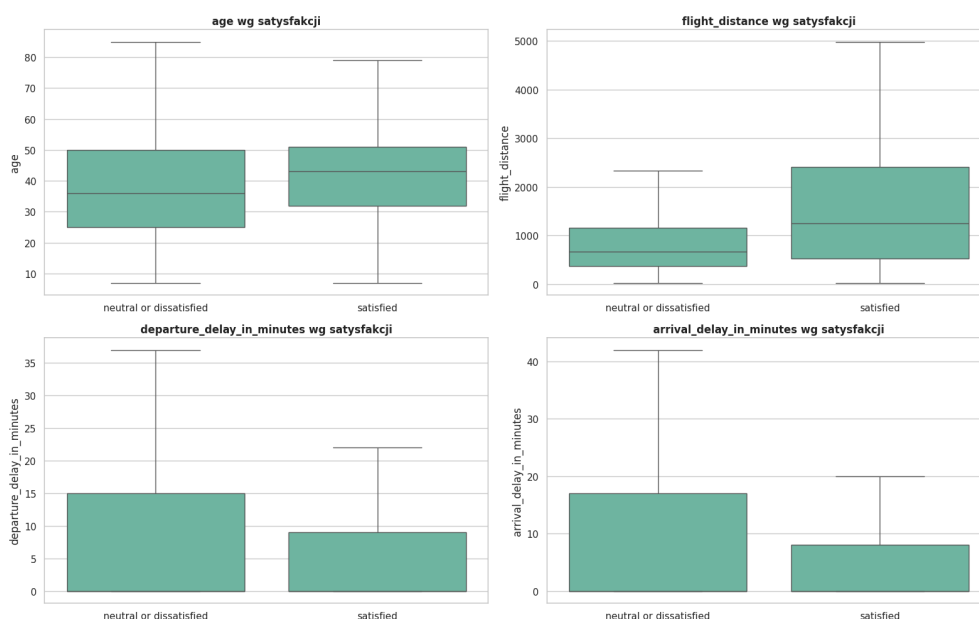
W analizie zmiennych liczbowych uwzględniono wiek, dystans lotu oraz opóźnienia. Rozkłady opóźnień są silnie skośne: większość lotów ma opóźnienie zerowe albo małe, ale występują pojedyncze bardzo wysokie wartości. Z tego powodu w wizualizacjach opóźnień przycięto do 99. percentyla, aby wykresy były czytelne.

Wiek pasażerów mieści się w przedziale od 7 do 85 lat. Dystans lotu mieści się w przedziale od 31 do 4983. Zmienna dystansu również jest prawostronnie skośna, dlatego w inżynierii cech dodano `log_flight_distance`.

Zmienna `arrival_delay_in_minutes` jest bardzo silnie skorelowana ze zmienną `departure_delay_in_minutes` (korelacja około 0,965). Jest to logiczne: opóźnienie odlotu często przekłada się na opóźnienie przylotu. Mimo tej korelacji obie zmienne pozostawiono, bo modele drzewiaste dobrze radzą sobie z powiązanymi predyktorami, a dodatkowo utworzono cechy sumaryczne i logarytmiczne.



Rysunek 5: Rozkłady zmiennych liczbowych z podziałem na klasę satysfakcji.



Rysunek 6: Boxploty zmiennych liczbowych z podziałem na klasę satysfakcji.

3.3. Kolumny ocenowe i interpretacja wartości 0

Kolumny ocenowe są zapisane w skali 0-5. Wartości 0 pojawiają się najczęściej w:

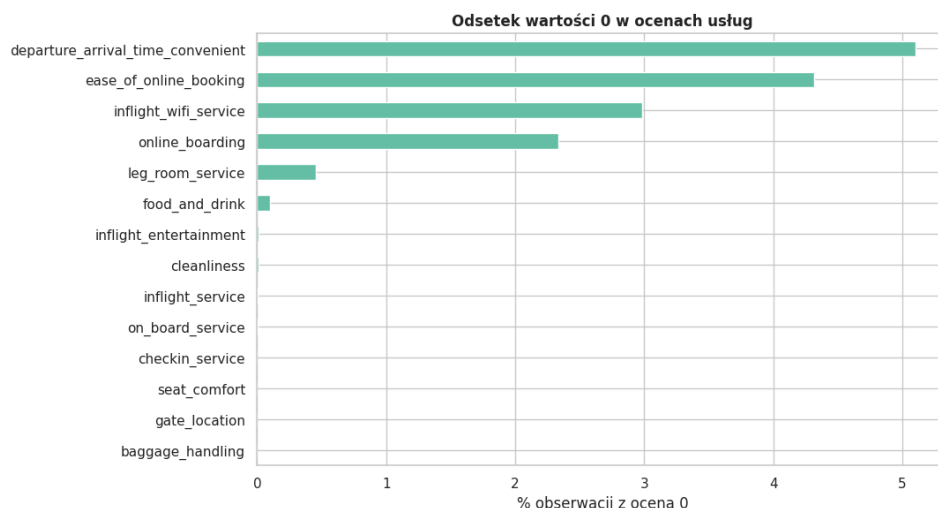
- `departure_arrival_time_convenient`: 5300 przypadków, około 5,10%,
- `ease_of_online_booking`: 4487 przypadków, około 4,32%,
- `inflight_wifi_service`: 3103 przypadki, około 2,99%,
- `online_boarding`: 2428 przypadków, około 2,34%.

Na podstawie komentarzy do zbioru przyjęto, że 0 oznacza brak odpowiedzi, a nie najgorszą ocenę. To założenie ma znaczenie modelowe. Gdyby pozostawić 0 jako zwykłą wartość liczbową, model mógłby uznać ją za ocenę niższą od 1. W rzeczywistości brak odpowiedzi może mieć inne znaczenie: pasażer mógł nie skorzystać z usługi, nie pamiętać jej albo nie udzielić odpowiedzi w ankiecie.

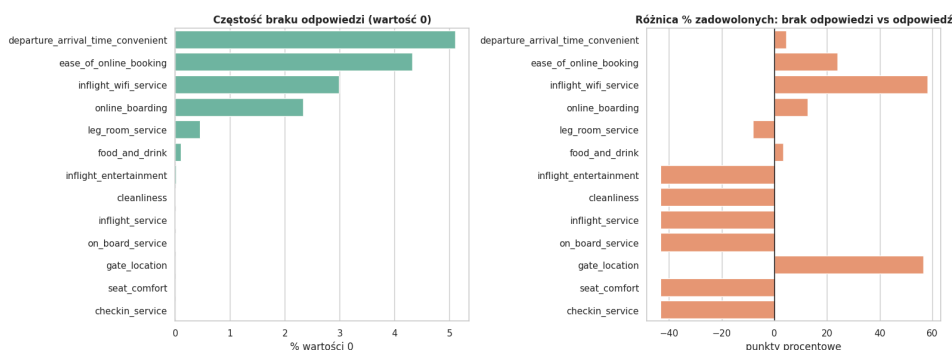
Dlatego w modelowaniu:

- dla każdej kolumny ocenowej dodano flagę `nazwa_cechy_no_answer`,
- wartości 0 zamieniono na NaN,
- imputacja braków odbywa się później w pipeline.

Takie podejście pozwala modelowi rozróżnić dwa sygnały: samą ocenę usługi oraz fakt braku odpowiedzi.



Rysunek 7: Odsetek wartości 0 w kolumnach ocenowych.



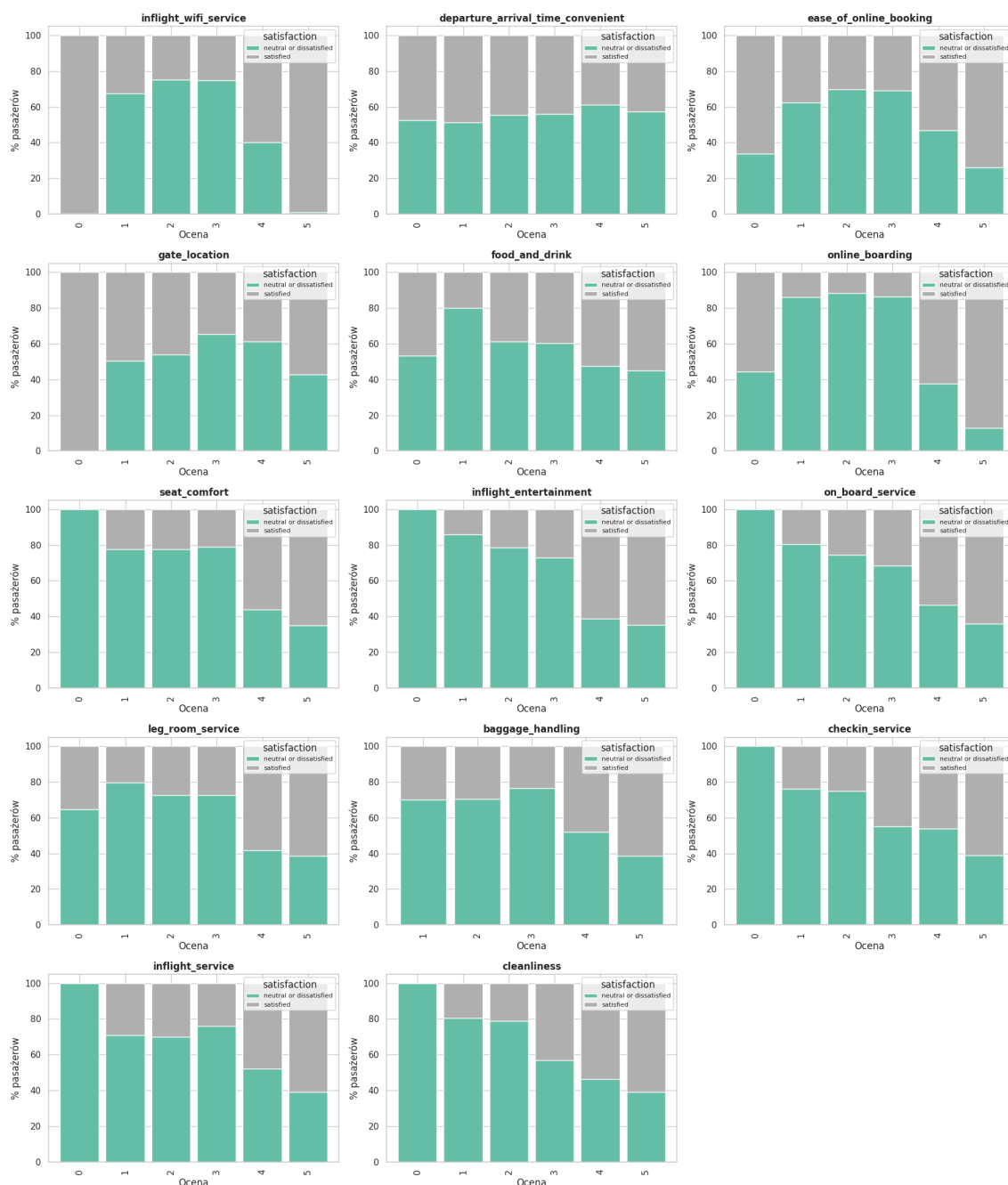
Rysunek 8: Analiza wartości 0 jako potencjalnego braku odpowiedzi.

3.4. Oceny usług a satysfakcja

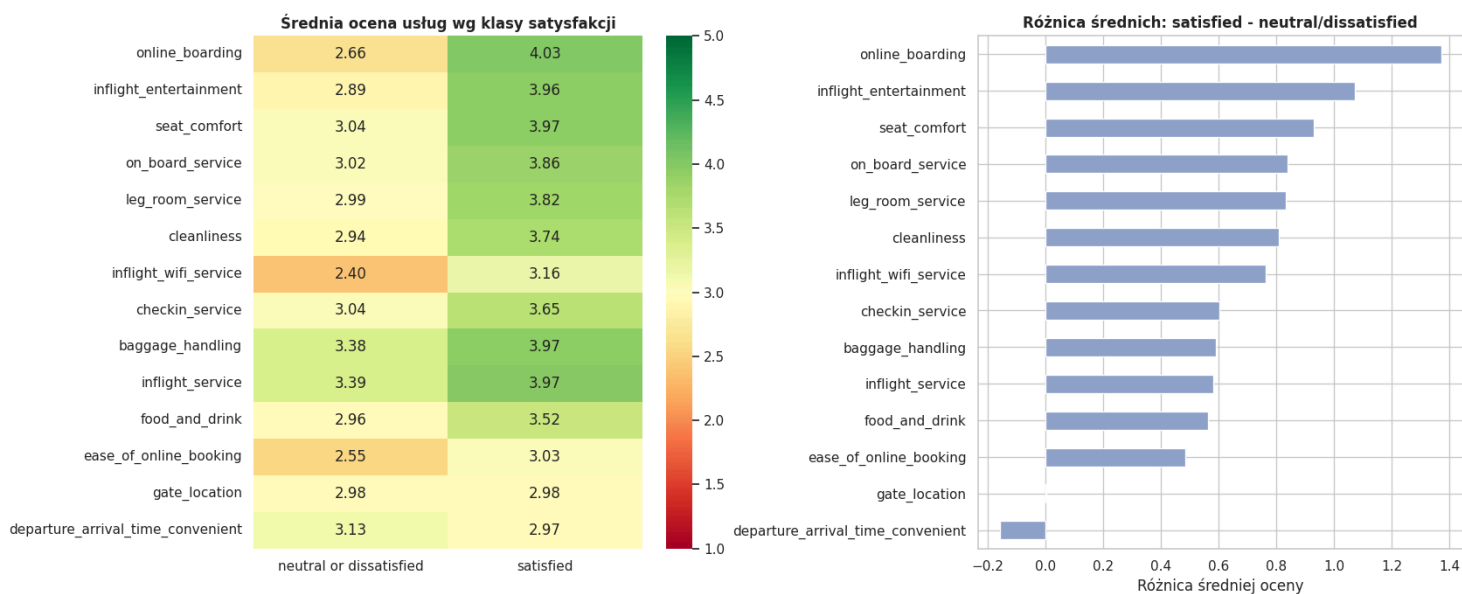
Największe różnice średnich ocen między klasą `satisfied` i `neutral or dissatisfied` zaobserwowano dla:

- `online_boarding`: różnica około 1,37 punktu,
- `inflight_entertainment`: różnica około 1,07 punktu,
- `seat_comfort`: różnica około 0,93 punktu,
- `on_board_service`: różnica około 0,84 punktu,
- `leg_room_service`: różnica około 0,83 punktu,
- `cleanliness`: różnica około 0,81 punktu.

Wskazuje to, że satysfakcja jest bardzo silnie związana z odczuwalną jakością usług. Szczególnie ważne wydaje się doświadczenie związane z odprawą online i boardingiem. Wynik ten jest spójny z późniejszą analizą permutation importance, gdzie `inflight_wifi_service`, `online_boarding` oraz inne oceny usług należą do najważniejszych predyktorów.



Rysunek 9: Rozkład satysfakcji dla poszczególnych ocen usług.



Rysunek 10: Średnie oceny usług według klasy satysfakcji.

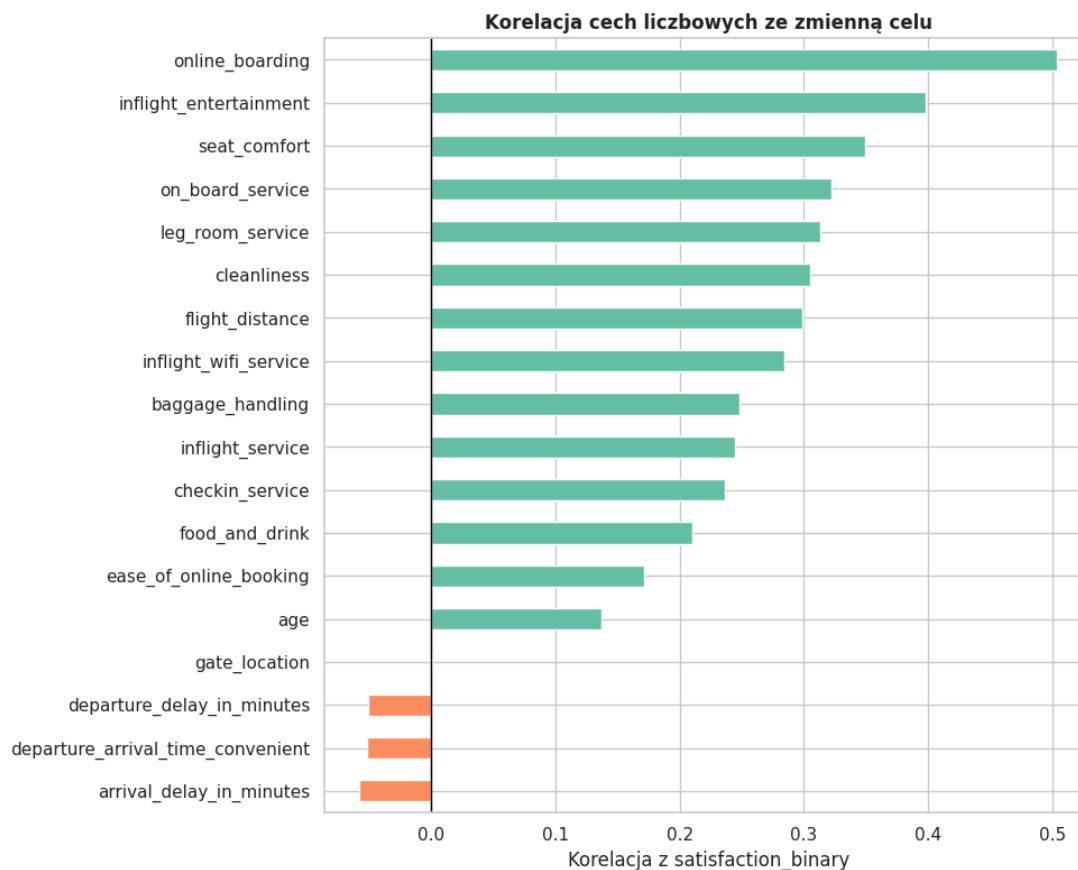
3.5. Korelacje

Macierz korelacji została policzona dla cech liczbowych metodą Pearsona. Wartości korelacji mieszczą się w zakresie od -1 do 1 . Wykres korelacji pokazuje, że najsilniejsze dodatnie zależności ze zmienną celu mają głównie zmienne ocenowe. Najważniejsza korelacja dotyczy **online_boarding**, natomiast opóźnień mają słabsze korelacje ujemne.

Interpretacja korelacji musi być ostrożna. Korelacja Pearsona mierzy zależność liniową i nie oznacza przyczynowości. Ponadto wiele modeli użytych w projekcie, zwłaszcza modele drzewiaste i boostingowe, potrafi wykorzystywać relacje nieliniowe oraz interakcje między cechami.



Rysunek 11: Macierz korelacji zmiennych liczbowych.



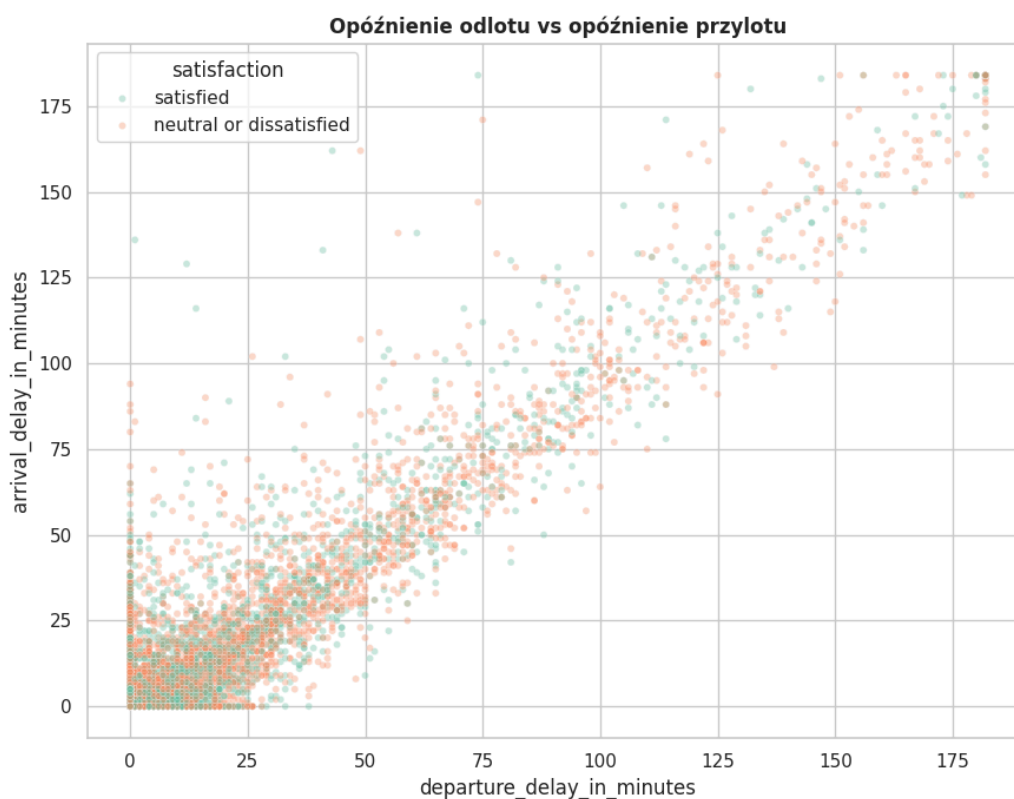
Rysunek 12: Korelacja cech liczbowych ze zmienną celu.

3.6. Opóźnienia, outliery i train-test drift

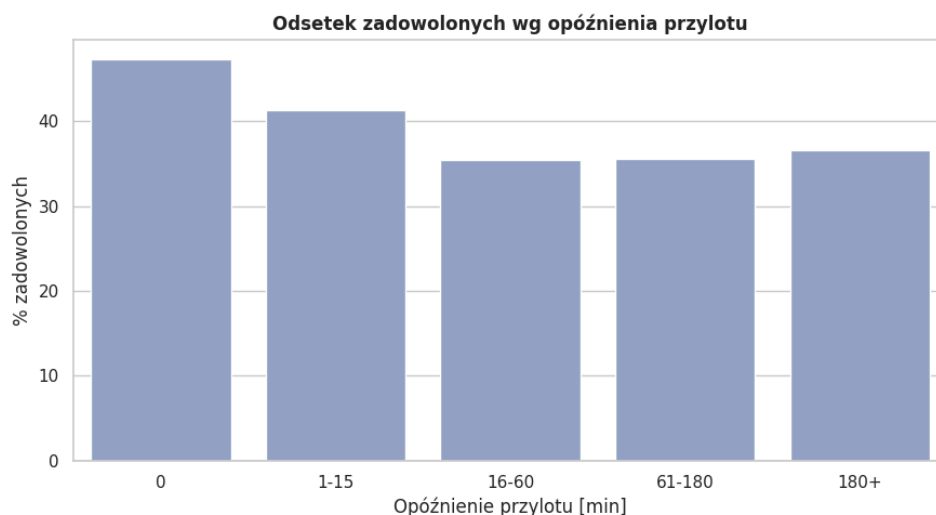
Opóźnienia są silnie skośne i zawierają obserwacje odstające. Według reguły IQR wartości odstające stanowią około 13,98% dla `departure_delay_in_minutes` oraz około 13,47% dla `arrival_delay_in_minutes`. Nie usunięto ich, ponieważ w kontekście lotów bardzo duże opóźnienia są rzeczywistymi zdarzeniami, a nie oczywistymi błędami danych.

Dla dystansu lotu wykryto około 2,21% obserwacji odstających według IQR. Również ich nie usunięto, ponieważ długie trasy są naturalną częścią danych lotniczych. Zamiast usuwania zastosowano transformację logarytmiczną.

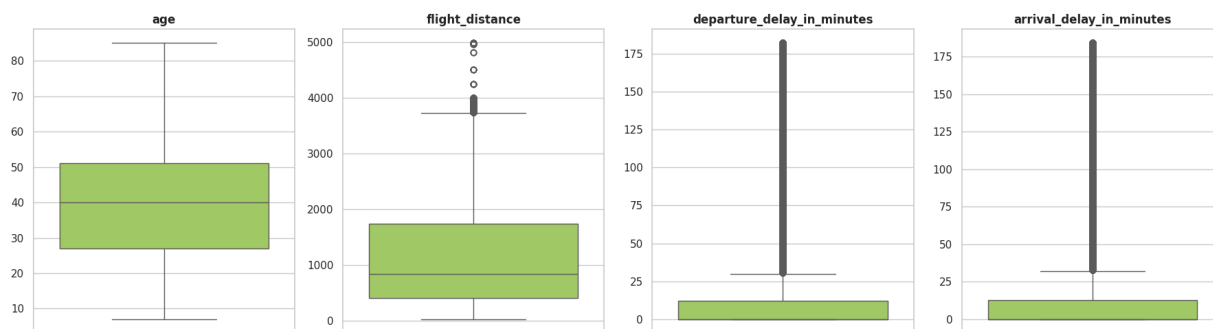
Porównanie train i test nie wykazało istotnego przesunięcia rozkładów. Średnie wartości najważniejszych zmiennych są bardzo podobne. Rozkład zmiennej celu również jest zbliżony: w train około 56,67% obserwacji to `neutral` or `dissatisfied`, a w test około 56,10%.



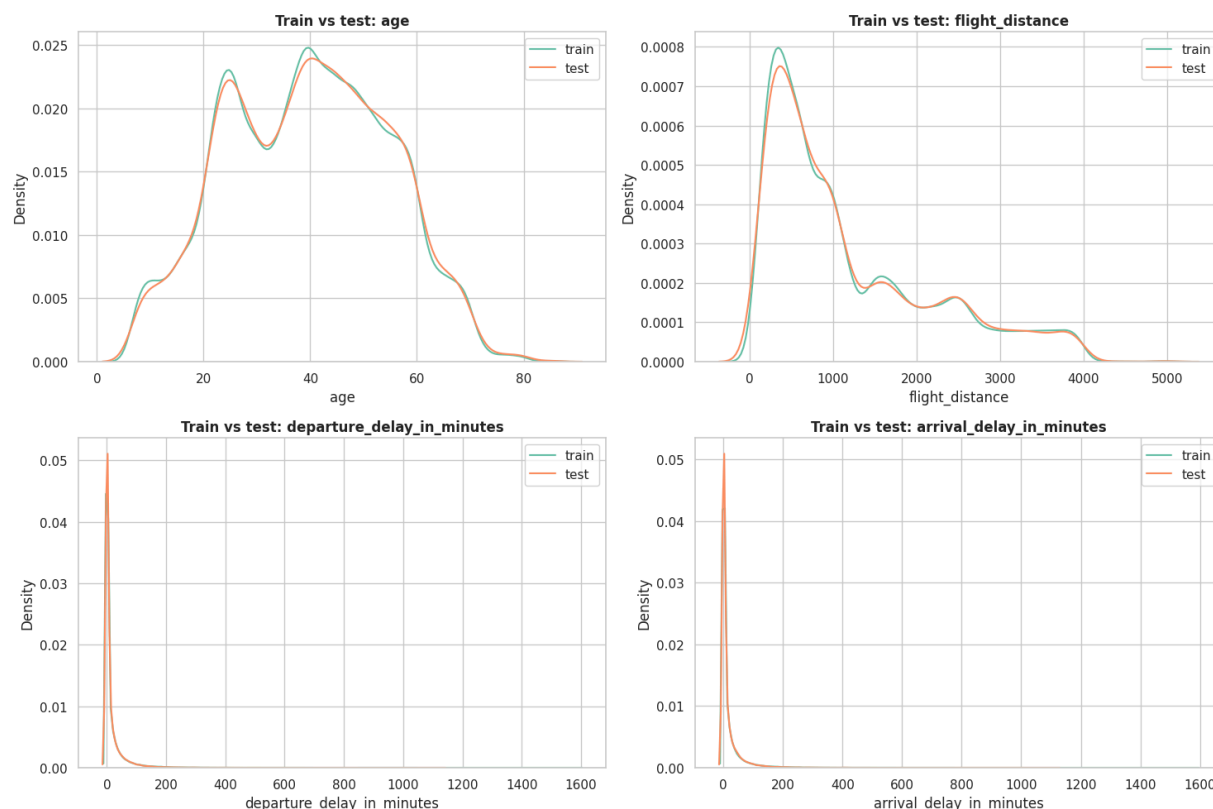
Rysunek 13: Zależność opóźnienia odlotu i przylotu.



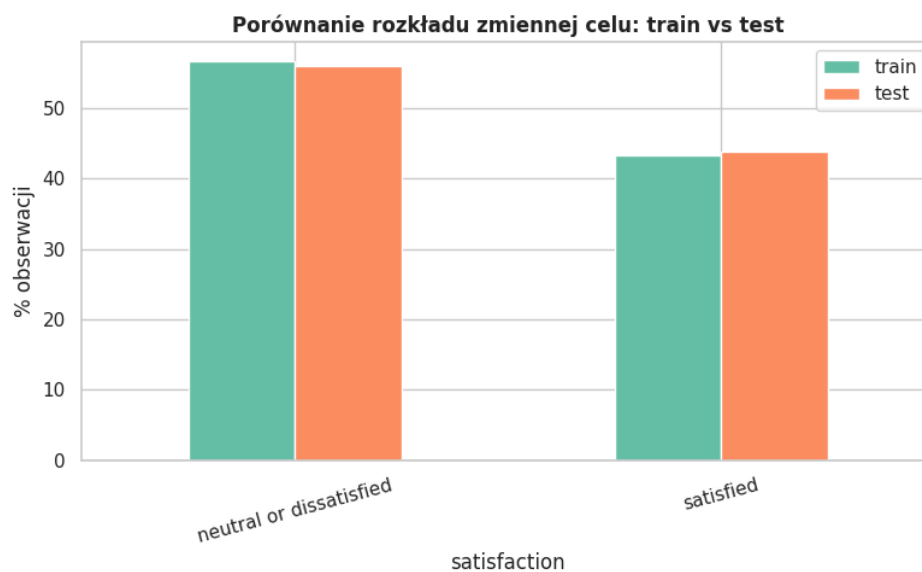
Rysunek 14: Odsetek zadowolonych według przedziału opóźnienia przylotu.



Rysunek 15: Analiza obserwacji odstających dla zmiennych liczbowych.



Rysunek 16: Porównanie rozkładów zmiennych liczbowych w train i test.

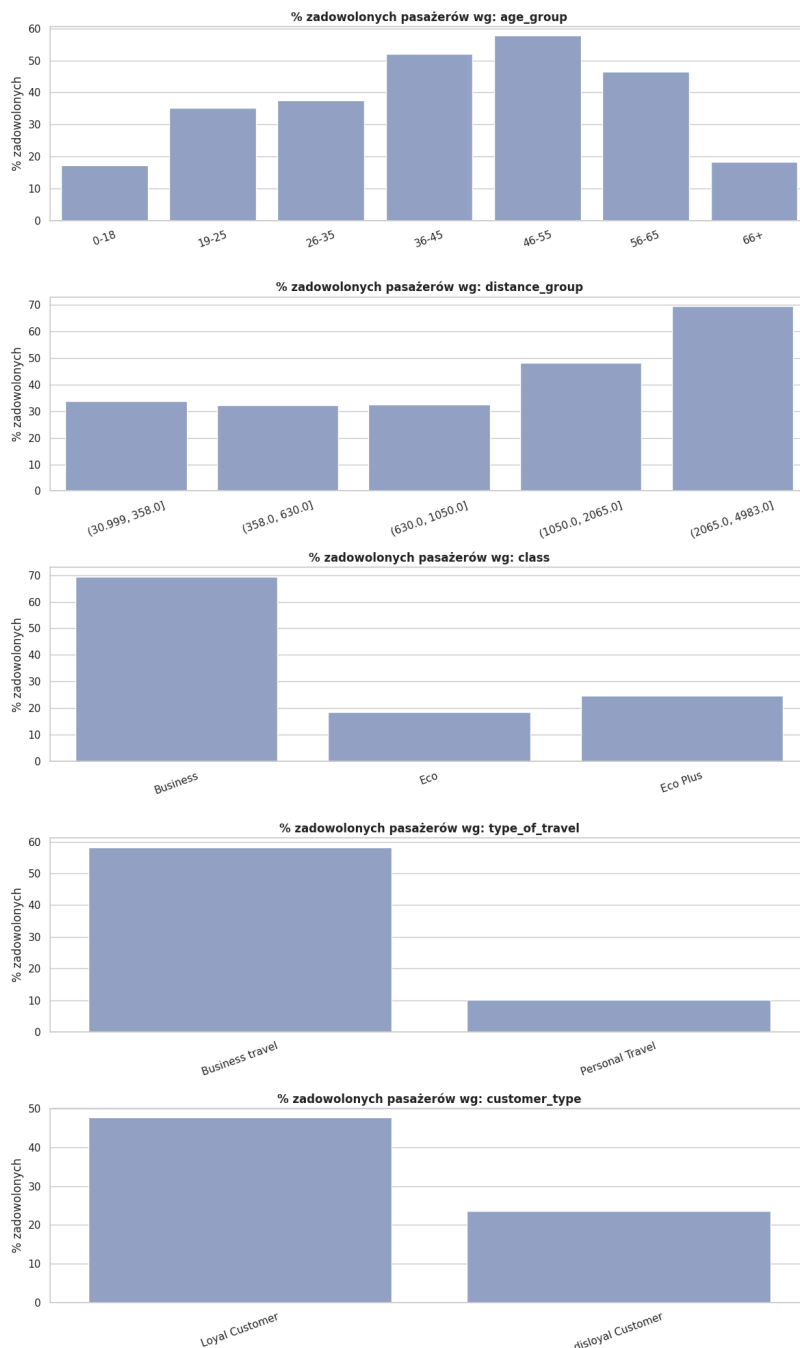


Rysunek 17: Porównanie rozkładu zmiennej celu w train i test.

3.7. Segmenty pasażerów

Segmentacja pasażerów według wieku, dystansu, klasy, typu podróży i typu klienta potwierdziła najważniejsze obserwacje z wcześniejszej EDA. Najwyższy odsetek zadowolonych występuje dla pasażerów w klasie Business oraz przy najdłuższych dystansach lotu. Dla najdłuższego kwintyla dystansu odsetek zadowolonych wynosi około 69,57%.

Wiek również różnicuje satysfakcję. Najwyższy odsetek zadowolonych zaobserwowano w grupach 36-45 oraz 46-55 lat, natomiast najniższy w grupie 0-18 oraz 66+. Sama interpretacja wieku wymaga jednak ostrożności, bo wiek może być powiązany z typem podróży, klasą biletu i innymi cechami.



Rysunek 18: Odsetek zadowolonych pasażerów w wybranych segmentach.

4. Sposób rozwiązania problemu

4.1. Przygotowanie danych

Proces przygotowania danych obejmował:

1. Ujednolicenie nazw kolumn do formatu `snake_case`.
2. Zakodowanie zmiennej celu jako 0 i 1.
3. Usunięcie identyfikatorów `id` oraz `unnamed_0` z predyktorów.
4. Dodanie flagi `arrival_delay_missing`.
5. Utworzenie cech opóźnień:
 - `total_delay_minutes`,
 - `arrival_departure_delay_diff`,
 - `had_departure_delay`,
 - `had_arrival_delay`,
 - `log_departure_delay`,
 - `log_arrival_delay`.
6. Utworzenie `log_flight_distance`.
7. Dodanie flag `*_no_answer` dla wszystkich kolumn ocenowych.
8. Zamianę 0 w ocenach na NaN.
9. Dodanie cech zbiorczych ocen, m.in. średniej, minimum, maksimum, odchylenia standardowego, liczby niskich ocen i liczby braków odpowiedzi.
10. Dodanie grup wieku `age_group`.

Po feature engineeringu liczba cech przed kodowaniem one-hot wzrosła do 54. Pipeline rozdziela zmienne liczbowe i kategoryczne. Dla zmiennych liczbowych stosowana jest imputacja medianą i skalowanie. Dla zmiennych kategorycznych stosowana jest imputacja najczęstszą wartością oraz one-hot encoding.

Poniżej znajdują się najważniejsze fragmenty kodu wykorzystane w notebooku. Nie jest to pełny kod projektu, tylko kluczowe etapy realizacji, analogicznie do sposobu pokazania kodu w przykładowym raporcie.

1. Wczytanie danych i ujednolicenie nazw kolumn. Dane zostały wczytane bezpośrednio z plików `train.csv` i `test.csv`, a nazwy kolumn zamieniono na format wygodny do dalszej pracy.

```
train_path = "train.csv"
test_path = "test.csv"

raw_train = pd.read_csv(train_path)
raw_test = pd.read_csv(test_path)

def clean_column_name(name):
    name = re.sub(r"^[^0-9a-zA-Z]+", "_", str(name).strip())
    return name.strip("_").lower()

train = raw_train.copy()
test = raw_test.copy()
train.columns = [clean_column_name(col) for col in train.columns]
test.columns = [clean_column_name(col) for col in test.columns]
```

2. Kodowanie zmiennej celu i podział kolumn. Zmienna `satisfaction` została zakodowana binarnie, a kolumny podzielono na identyfikatory, zmienne katagoryczne, zmienne ciągłe i oceny usług.

```
TARGET = "satisfaction"
TARGET_BINARY = "satisfaction_binary"
TARGET_MAP = {"neutral or dissatisfied": 0, "satisfied": 1}

for df in [train, test]:
    df[TARGET_BINARY] = df[TARGET].map(TARGET_MAP)

id_cols = ["unnamed_0", "id"]
categorical_cols = ["gender", "customer_type", "type_of_travel", "class"]
continuous_cols = [
    "age",
    "flight_distance",
    "departure_delay_in_minutes",
    "arrival_delay_in_minutes",
]

non_rating_cols = set(id_cols + categorical_cols + continuous_cols + [TARGET,
TARGET_BINARY])
rating_cols = [
    col for col in train.select_dtypes(include="number").columns
    if col not in non_rating_cols
]
```

3. Obsługa wartości 0 w ocenach. Najważniejszą decyzją czyszczenia było potraktowanie 0 jako braku odpowiedzi. Dlatego dodano flagi `*_no_answer`, a następnie zera zamieniono na `NaN`.

```
available_rating_cols = [col for col in rating_cols if col in df.columns]

for col in available_rating_cols:
    df[f"{col}_no_answer"] = df[col].eq(0).astype(int)

ratings_without_zero = df[available_rating_cols].replace(0, np.nan)
df[available_rating_cols] = ratings_without_zero

df["service_rating_mean"] = ratings_without_zero.mean(axis=1)
df["service_rating_min"] = ratings_without_zero.min(axis=1)
df["service_rating_std"] = ratings_without_zero.std(axis=1)
df["rating_no_answer_count"] = ratings_without_zero.isna().sum(axis=1)
df["low_service_rating_count"] = ratings_without_zero.le(2).sum(axis=1)
df["high_service_rating_count"] = ratings_without_zero.ge(4).sum(axis=1)
```

4. Inżynieria cech dla opóźnień i dystansu. Dla zmiennych skośnych dodano transformacje logarytmiczne oraz flagi informujące, czy opóźnienie wystąpiło.

```
df["arrival_delay_missing"] = df["arrival_delay_in_minutes"].isna().astype(int)
df["total_delay_minutes"] = df[
    ["departure_delay_in_minutes", "arrival_delay_in_minutes"]
].sum(axis=1, min_count=2)

df["arrival_departure_delay_diff"] = (
    df["arrival_delay_in_minutes"] - df["departure_delay_in_minutes"]
)
df["had_departure_delay"] = (df["departure_delay_in_minutes"].fillna(0) > 0).astype(int)
df["had_arrival_delay"] = (df["arrival_delay_in_minutes"].fillna(0) > 0).astype(int)
df["log_departure_delay"] = np.log1p(df["departure_delay_in_minutes"].clip(lower=0))
df["log_arrival_delay"] = np.log1p(df["arrival_delay_in_minutes"].clip(lower=0))
df["log_flight_distance"] = np.log1p(df["flight_distance"].clip(lower=0))
```

5. Pipeline przetwarzania danych. Preprocessing został zapisany jako pipeline, żeby uniknąć wycieku danych i stosować te same operacje na train, walidacji i test.

```
numeric_preprocess = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_preprocess = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocess = ColumnTransformer(transformers=[
    ("num", numeric_preprocess, numeric_cols_model),
    ("cat", categorical_preprocess, categorical_cols_model),
])
```

4.2. Modele

Porównano następujące modele:

- DummyClassifier jako punkt odniesienia,
- regresję logistyczną,
- Gaussian Naive Bayes,
- drzewo decyzyjne,
- random forest,
- extra trees,
- histogram gradient boosting.

Regresja logistyczna została użyta jako model szybki i relatywnie interpretowalny. Modele drzewiaste wybrano ze względu na ich zdolność do wykrywania zależności nieliniowych i interakcji między cechami. Histogram gradient boosting okazał się najlepszym modelem, co jest typowe dla wielu problemów tablicowych z dużą liczbą cech i nieliniowymi zależnościami.

Definicja modeli w notebooku wyglądała następująco:

```
models = {
    "Dummy majority": DummyClassifier(
        strategy="most_frequent",
        random_state=RANDOM_STATE,
    ),
    "Logistic regression": LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=RANDOM_STATE,
    ),
    "Gaussian Naive Bayes": GaussianNB(),
    "Decision tree": DecisionTreeClassifier(
        max_depth=18,
        min_samples_leaf=20,
        random_state=RANDOM_STATE,
        class_weight="balanced",
    ),
    "Random forest": RandomForestClassifier(
        n_estimators=220,
        min_samples_leaf=2,
        n_jobs=-1,
        random_state=RANDOM_STATE,
        class_weight="balanced_subsample",
    ),
    "Extra trees": ExtraTreesClassifier(
        n_estimators=220,
        min_samples_leaf=2,
        n_jobs=-1,
        random_state=RANDOM_STATE,
        class_weight="balanced",
    ),
    "Hist gradient boosting": HistGradientBoostingClassifier(
        random_state=RANDOM_STATE,
    ),
}
```

4.3. Miary ewaluacji

Do oceny modeli wykorzystano:

- accuracy,
- precision,
- recall,
- F1,
- ROC-AUC,
- macierz pomyłek,
- krzywą ROC,
- krzywą Precision-Recall.

Najważniejszą metryką rankingową w projekcie jest F1, ponieważ łączy precision i recall. Accuracy również jest raportowane, ale nie powinno być jedynym kryterium. Zbiór jest umiarkowanie niezbalansowany, więc model przewidujący tylko klasę większościową uzyskałby zauważalną accuracy, ale byłby bezużyteczny dla wykrywania klasy `satisfied`.

Definicje podstawowych metryk:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = T \frac{P}{TP + FP}$$

$$\text{Recall} = T \frac{P}{TP + FN}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROC-AUC mierzy zdolność modelu do rozdzielania klas przy różnych progach decyzyjnych. Wartość 0,5 odpowiada modelowi losowemu, a wartość bliska 1 oznacza bardzo dobrą separację.

Do liczenia metryk wykorzystano funkcję pomocniczą. Dzięki temu każdy model był oceniany w ten sam sposób.

```
def metric_row(name, y_true, pred, scores):
    return {
        "model": name,
        "accuracy": accuracy_score(y_true, pred),
        "precision": precision_score(y_true, pred, zero_division=0),
        "recall": recall_score(y_true, pred, zero_division=0),
        "f1": f1_score(y_true, pred, zero_division=0),
        "roc_auc": roc_auc_score(y_true, scores),
    }

def evaluate_model(name, model, X_train, y_train, X_valid, y_valid):
    pipe = Pipeline(steps=[
        ("preprocess", make_preprocess(X_train)),
        ("model", clone(model)),
    ])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_valid)
    scores = get_positive_scores(pipe, X_valid)
    return pipe, metric_row(name, y_valid, pred, scores)
```

5. Wyniki modelowania

5.1. Modele bazowe

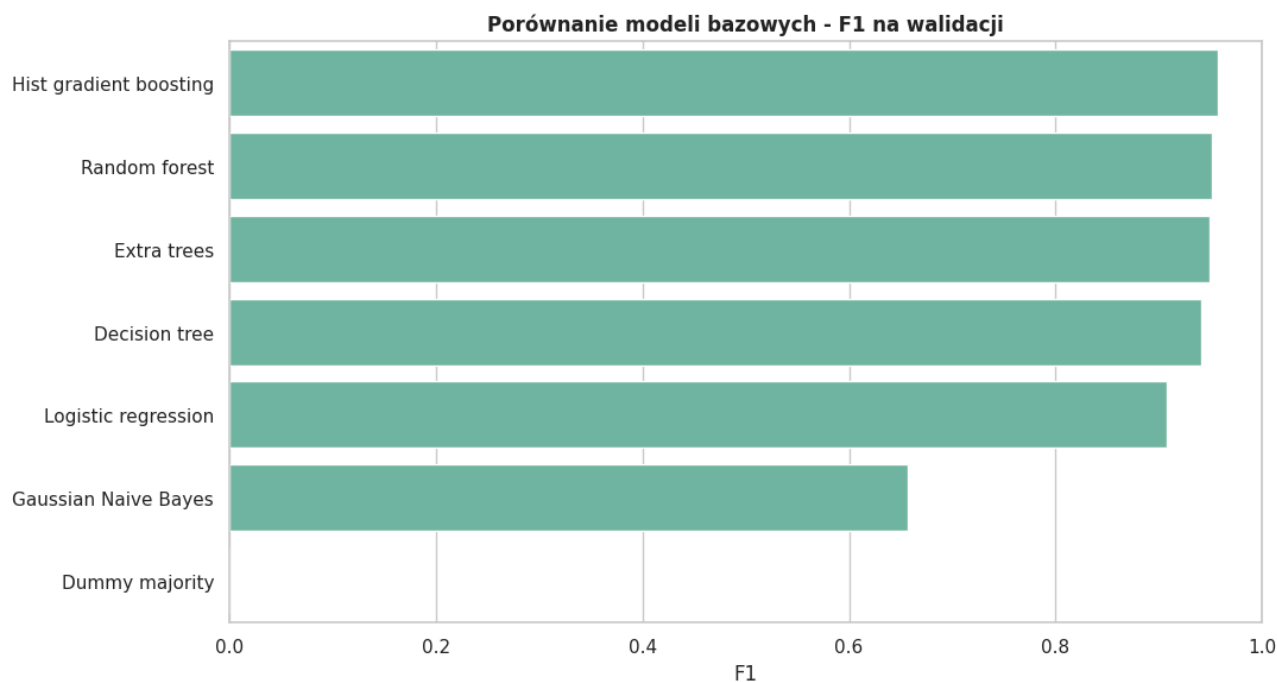
Najlepszym modelem bazowym na zbiorze walidacyjnym był `HistGradientBoostingClassifier`. Osiągnął:

- accuracy = 0,964,
- precision = 0,971,
- recall = 0,944,
- F1 = 0,957,
- ROC-AUC = 0,995.

Drugie miejsce zajął Random Forest z F1 = 0,952, a trzecie Extra Trees z F1 = 0,949. Regresja logistyczna również była bardzo solidna, osiągając F1 = 0,908, co pokazuje, że w danych istnieje silny i względnie stabilny sygnał predykcyjny.

Najgorzej zachował się `DummyClassifier`, który zawsze przewiduje klasę większościową. Jego F1 wyniosło 0,000 dla klasy pozytywnej, co potwierdza, że proste accuracy bez dodatkowych metryk mogłoby być mylące.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Hist gradient boosting	0,964	0,971	0,944	0,957	0,995
Random forest	0,959	0,960	0,943	0,952	0,993
Extra trees	0,956	0,955	0,943	0,949	0,993
Decision tree	0,949	0,940	0,942	0,941	0,987
Logistic regression	0,919	0,895	0,921	0,908	0,974
Gaussian Naive Bayes	0,552	0,492	0,992	0,657	0,923
Dummy majority	0,567	0,000	0,000	0,000	0,500

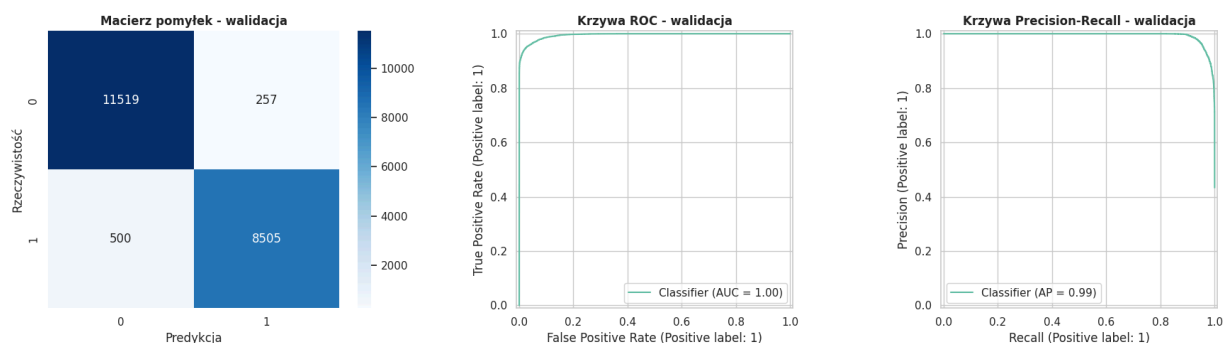


Rysunek 19: Porównanie modeli bazowych według F1 na walidacji.

5.2. Walidacja najlepszego modelu bazowego

Dla najlepszego modelu bazowego, czyli histogram gradient boosting, klasyfikacja na zbiorze walidacyjnym była bardzo dobra. Klasa *neutral/dissatisfied* uzyskała precision około 0,96 i recall około 0,98. Klasa *satisfied* uzyskała precision około 0,97 i recall około 0,94.

Oznacza to, że model jest nieco ostrożny w przewidywaniu klasy pozytywnej. Jeżeli model przewiduje, że pasażer jest zadowolony, zwykle ma rację. Jednocześnie część zadowolonych pasażerów jest klasyfikowana jako neutralna lub niezadowolona. Z punktu widzenia biznesowego taki błąd może być mniej ryzykowny niż odwrotny, ponieważ lepiej jest niepotrzebnie zainteresować się pasażerem niż przeoczyć potencjalny problem.



Rysunek 20: Macierz pomyłek, ROC i Precision-Recall dla najlepszego modelu walidacyjnego.

5.3. Eksperyment z grupami cech

Zrealizowano dodatkowy eksperyment zaproponowany podczas pracy grupowej: porównano modele używające różnych grup cech. Wyniki są bardzo pouczające.

Model pełny z wszystkimi cechami osiągnął $F1 = 0,957$. Model używający wyłącznie ocen usług i flag braku odpowiedzi osiągnął $F1 = 0,934$. To bardzo wysoki wynik, który pokazuje, że ankietywane oceny usług niosą większość informacji potrzebnej do przewidywania satysfakcji.

Model oparty tylko na danych obiektywnych był wyraźnie słabszy. Regresja logistyczna na danych obiektywnych osiągnęła $F1 = 0,733$, a histogram gradient boosting $F1 = 0,728$. Po dodaniu kontekstu podróży i klienta wynik wzrósł do około 0,783 dla histogram gradient boosting. Oznacza to, że *type_of_travel* i *customer_type* są bardzo istotne, ale nadal nie zastępują informacji z ocen usług.

Podział cech na grupy został zapisany jawnie w kodzie, aby można było łatwo powtórzyć eksperyment.

```
objective_base_cols = [  
    "gender",  
    "age",  
    "class",  
    "flight_distance",  
    "departure_delay_in_minutes",  
    "arrival_delay_in_minutes",  
]  
  
travel_context_cols = ["type_of_travel", "customer_type"]  
rating_indicator_cols = [f"{col}_no_answer" for col in rating_cols]  
rating_summary_cols = [  
    "service_rating_mean",  
    "service_rating_min",  
    "service_rating_std",  
    "rating_no_answer_count",  
    "low_service_rating_count",  
    "high_service_rating_count",  
]
```

```

self_reported_cols = [
    col for col in rating_cols + rating_indicator_cols + rating_summary_cols
    if col in X_train_full.columns
]

feature_sets = {
    "objective_only": objective_cols,
    "objective_plus_context": objective_plus_context_cols,
    "self_reported_ratings_only": self_reported_cols,
    "all_features": X_train_full.columns.tolist(),
}

```

Następnie dla każdej grupy cech trenowano te same modele i zapisywano porównywalne metryki.

```

feature_set_rows = []

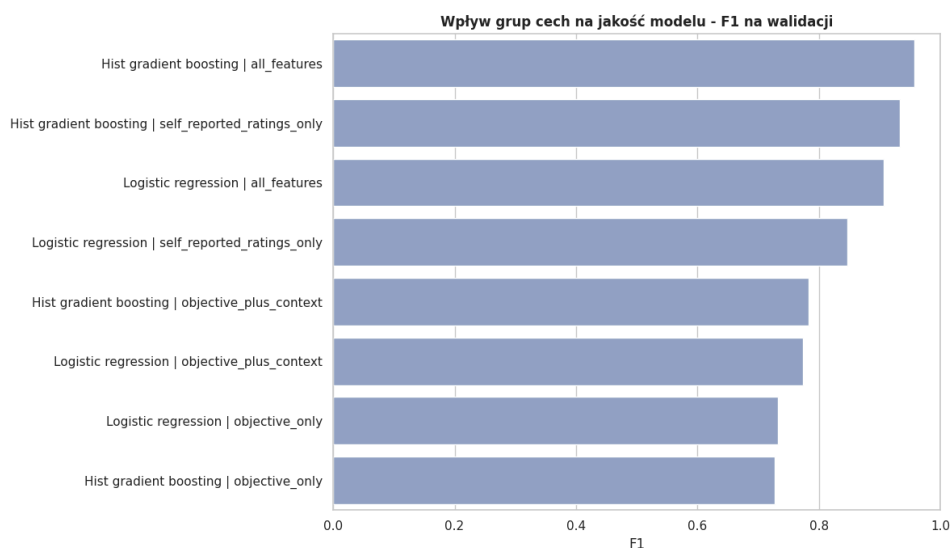
for feature_set_name, cols in feature_sets.items():
    X_train_fs = X_train_full.loc[X_train.index, cols]
    X_valid_fs = X_train_full.loc[X_valid.index, cols]

    for model_name, model in feature_set_models.items():
        pipe = Pipeline(steps=[
            ("preprocess", make_preprocess(X_train_fs)),
            ("model", clone(model)),
        ])
        pipe.fit(X_train_fs, y_train)
        pred = pipe.predict(X_valid_fs)
        scores = get_positive_scores(pipe, X_valid_fs)

        row = metric_row(
            f"{model_name} | {feature_set_name}",
            y_valid,
            pred,
            scores,
        )
        row["feature_set"] = feature_set_name
        row["base_model"] = model_name
        feature_set_rows.append(row)

```

Model i grupa cech	Accura- cy	Preci- sion	Recall	F1	ROC- -AUC
HGB wszystkie cechy	0,964	0,971	0,944	0,957	0,995
HGB tylko oceny samoopisowe	0,944	0,949	0,920	0,934	0,989
Regresja logistyczna wszystkie cechy	0,919	0,895	0,921	0,908	0,974
Regresja logistyczna tylko oceny samo- opisowe	0,862	0,818	0,878	0,847	0,937
HGB obiektywne + kontekst	0,808	0,768	0,799	0,783	0,874
Regresja logistyczna obiektywne + kontekst	0,790	0,726	0,828	0,774	0,855
Regresja logistyczna tylko obiektywne	0,759	0,705	0,764	0,733	0,810
HGB tylko obiektywne	0,769	0,743	0,713	0,728	0,825



Rysunek 21: Porównanie jakości modeli dla różnych grup cech.

5.4. Walidacja krzyżowa

Walidacja krzyżowa 5-fold potwierdziła stabilność wyników. Histogram gradient boosting osiągnął średnie $F1 = 0,956$ przy odchyleniu standardowym 0,002. Random Forest uzyskał średnie $F1 = 0,950$, a Extra Trees $F1 = 0,948$. Różnice są niewielkie, ale gradient boosting pozostaje najlepszy.

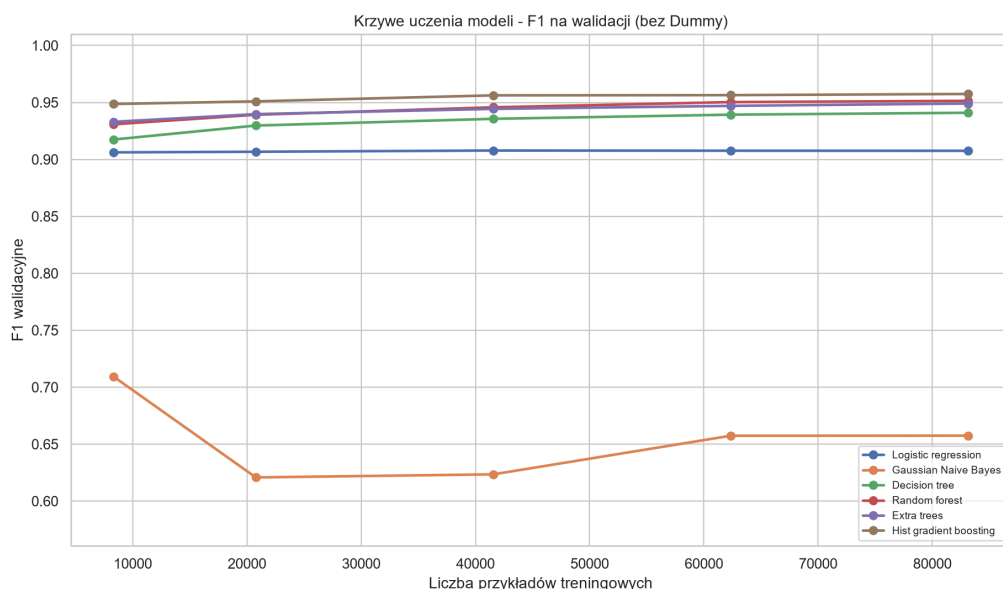
Stabilność wyników w walidacji krzyżowej sugeruje, że model nie opiera się na przypadkowym podziale train/valid. Jest to ważne, bo pojedynczy split walidacyjny może czasem zawyżać lub zaniżać wynik.

Model	Accuracy mean	F1 mean	F1 std	ROC-AUC mean
Hist gradient boosting	0,963	0,956	0,002	0,995
Random forest	0,958	0,950	0,002	0,993
Extra trees	0,955	0,948	0,003	0,992
Logistic regression	0,921	0,909	0,003	0,974
Dummy majority	0,567	0,000	0,000	0,500

5.5. Krzywe uczenia modeli

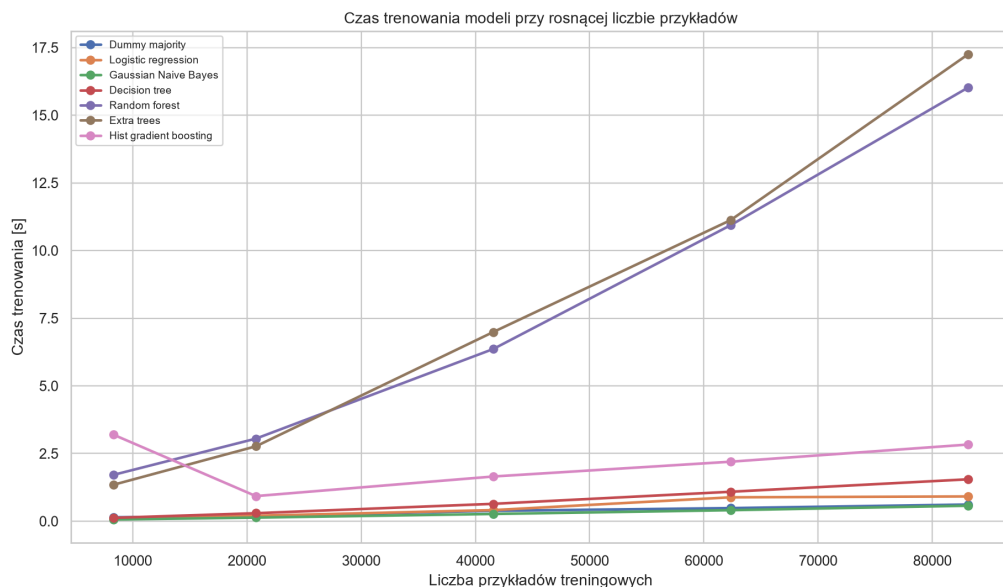
Krzywe uczenia sprawdzają, jak jakość modeli zmienia się wraz ze wzrostem liczby przykładów treningowych. W każdym eksperymencie użyto tego samego podziału train/valid, a model był trenowany kolejno na 10%, 25%, 50%, 75% i 100% części treningowej. Dla każdego rozmiaru zapisano F1 na treningu, F1 na walidacji oraz czas trenowania.

Wyniki pokazują, że najlepsze modele drzewiaste rzeczywiście korzystają z większej liczby danych. Random Forest rośnie od F1 około 0,931 do 0,951, Extra Trees od 0,933 do 0,949, a histogram gradient boosting od 0,949 do 0,957. Regresja logistyczna jest bardzo stabilna i szybko osiąga swój poziom około $F1 = 0,908$, co oznacza, że dodatkowe dane nie zmieniają jej jakości tak mocno jak w modelach nieliniowych. Gaussian Naive Bayes pozostaje wyraźnie słabszy, a Dummy majority nie uczy się klasy pozytywnej.



Rysunek 22: Krzywe uczenia modeli według F1 na walidacji, bez modelu Dummy majority.

Z punktu widzenia czasu trenowania najdroższe są modele zespołowe oparte na wielu drzewach, czyli Random Forest i Extra Trees. Histogram gradient boosting daje najlepszą jakość przy relatywnie rozsądnym czasie trenowania. Pomiary czasu należy traktować orientacyjnie, ponieważ zależą od obciążenia środowiska, optymalizacji biblioteki i szczegółów algorytmu.



Rysunek 23: Czas trenowania modeli przy rosnącej liczbie przykładów treningowych.

5.6. Strojenie hiperparametrów

Dla najlepszego typu modelu wykonano `RandomizedSearchCV` dla `HistGradientBoostingClassifier`. Przetestowano 8 losowych kombinacji hiperparametrów w 3-krotnej walidacji krzyżowej, łącznie 24 dopasowania.

Najlepsze parametry:

- `min_samples_leaf = 20`,
- `max_leaf_nodes = 63`,
- `max_iter = 300`,
- `learning_rate = 0.03`,
- `l2_regularization = 0.01`.

Najlepszy wynik walidacyjny F1 w strojeniu wyniósł około 0,9578. Strojenie nie zmieniło radykalnie jakości modelu, ale lekko poprawiło wynik finalny i potwierdziło, że model gradient boosting jest najlepszym wyborem.

Kod strojenia hiperparametrów:

```
hgb_pipe = Pipeline(steps=[
    ("preprocess", make_preprocess(X_train_full)),
    ("model", HistGradientBoostingClassifier(random_state=RANDOM_STATE)),
])

hgb_param_dist = {
    "model__learning_rate": [0.03, 0.05, 0.08, 0.10],
    "model__max_iter": [120, 200, 300],
    "model__max_leaf_nodes": [15, 31, 63],
    "model__min_samples_leaf": [20, 50, 100],
    "model__l2_regularization": [0.0, 0.01, 0.1],
}

hgb_search = RandomizedSearchCV(
    estimator=hgb_pipe,
    param_distributions=hgb_param_dist,
    n_iter=8,
    scoring="f1",
    cv=3,
    n_jobs=-1,
    random_state=RANDOM_STATE,
    refit=True,
)

hgb_search.fit(X_train_full, y_train_full)
tuned_model = hgb_search.best_estimator_
```

6. Ewaluacja końcowa

6.1. Wynik na zbiorze testowym

Finalny model to dostrojony `HistGradientBoostingClassifier`. Na zbiorze testowym osiągnął:

- `accuracy = 0,965`,
- `precision = 0,975`,
- `recall = 0,943`,
- `F1 = 0,959`,
- `ROC-AUC = 0,995`.

Są to bardzo dobre wyniki. Model poprawnie klasyfikuje zdecydowaną większość pasażerów, a bardzo wysoka wartość ROC-AUC oznacza, że dobrze rozdziela klasy także niezależnie od konkretnego progu decyzyjnego.

Macierz pomyłek na zbiorze testowym:

- `TN = 14 302`,
- `FP = 271`,
- `FN = 649`,
- `TP = 10 754`.

Model popełnia więcej błędów typu FN niż FP, czyli częściej klasyfikuje zadowolonego pasażera jako neutralnego lub niezadowolonego niż odwrotnie. Przy obecnym celu biznesowym jest to akceptowalne, ponieważ model jest precyzyjny w wykrywaniu klasy `satisfied`.

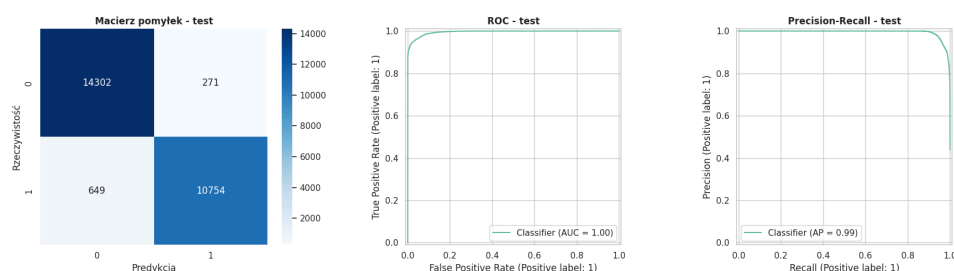
Finalna ewaluacja na zbiorze testowym została wykonana po wyborze najlepszego modelu. Kod liczył predykcje, prawdopodobieństwa oraz zestaw metryk.

```
final_model = tuned_model
y_test_pred = final_model.predict(X_test_final)
y_test_score = get_positive_scores(final_model, X_test_final)

final_metrics = metric_row(
    "Tuned Hist gradient boosting",
    y_test_final,
    y_test_pred,
    y_test_score,
)

print(classification_report(
    y_test_final,
    y_test_pred,
    target_names=["neutral/dissatisfied", "satisfied"],
    zero_division=0,
))
```

Metryka	Wartość	Interpretacja
Accuracy	0,965	Model poprawnie klasyfikuje 96,5% pasażerów ze zbioru testowego.
Precision	0,975	Gdy model przewiduje klasę satisfied , bardzo rzadko się myli.
Recall	0,943	Model wykrywa większość zadowolonych pasażerów, ale część z nich klasyfikuje zachowawczo.
F1	0,959	Najważniejsza metryka projektu potwierdza bardzo dobrą równowagę precision i recall.
ROC-AUC	0,995	Model bardzo dobrze rozdziela obie klasy niezależnie od progu decyzyjnego.



Rysunek 24: Macierz pomyłek oraz krzywe ROC i Precision-Recall na zbiorze testowym.

6.2. Interpretacja błędów

W zbiorze testowym wystąpiło 920 błędnych klasyfikacji na 25 976 obserwacji. Najwięcej błędów to fałszywie negatywne przypadki: 649 zadowolonych pasażerów zostało zaklasyfikowanych jako neutralni lub niezadowoleni. Fałszywie pozytywnych przypadków było 271.

Takie zachowanie oznacza, że model jest bardziej zachowawczy w przypisywaniu klasy **satisfied**. Precision dla klasy pozytywnej wyniosło 0,975, więc gdy model przewiduje satysfakcję, jest bardzo wiarygodny. Recall wyniosło 0,943, więc model wykrywa większość zadowolonych pasażerów, ale nie wszystkich.

W zastosowaniu biznesowym można zmienić próg decyzyjny. Jeśli celem byłoby wykrycie jak największej liczby zadowolonych pasażerów, można obniżyć próg, zwiększając recall kosztem precision. Jeśli celem byłaby bardzo pewna identyfikacja zadowolonych pasażerów, obecny próg jest sensowny.

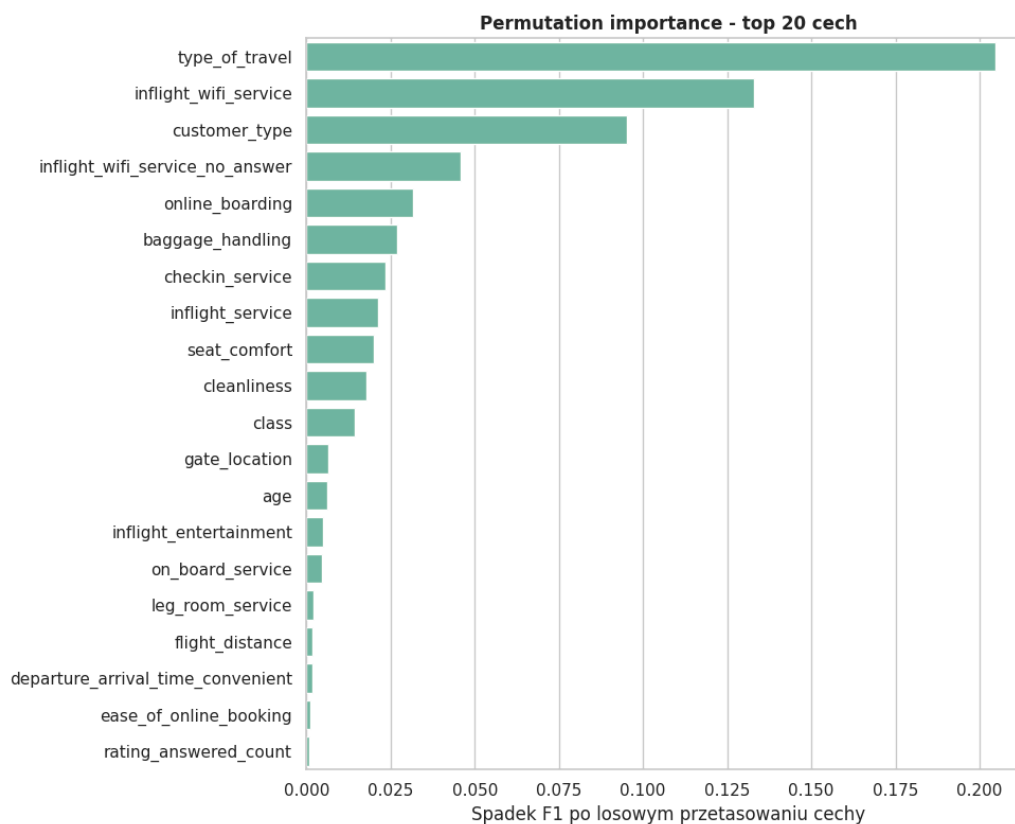
6.3. Wyjaśnialność modelu

Do interpretacji modelu zastosowano permutation importance na zbiorze testowym. Metoda mierzy spadek jakości modelu po losowym przetasowaniu wartości danej cechy. Jeżeli przetasowanie cechy mocno pogarsza F1, cecha jest ważna dla modelu.

Najważniejsze cechy według permutation importance:

1. `type_of_travel` - spadek F1 około 0,205,
2. `inflight_wifi_service` - około 0,133,
3. `customer_type` - około 0,095,
4. `inflight_wifi_service_no_answer` - około 0,046,
5. `online_boarding` - około 0,032,
6. `baggage_handling` - około 0,027,
7. `checkin_service` - około 0,023,
8. `inflight_service` - około 0,021,
9. `seat_comfort` - około 0,020,
10. `cleanliness` - około 0,018.

Najważniejsze są więc zarówno cechy kontekstowe, jak i oceny usług. Szczególnie ciekawa jest wysoka pozycja `inflight_wifi_service_no_answer`. Oznacza to, że sam brak odpowiedzi dotyczącej Wi-Fi jest informacją predykcyjną. To potwierdza sens dodania flag `*_no_answer`.



Rysunek 25: Permutation importance dla finalnego modelu.

Kod obliczenia permutation importance:

```
perm = permutation_importance(  
    final_model,  
    X_importance,  
    y_importance,  
    n_repeats=8,  
    random_state=RANDOM_STATE,  
    scoring="f1",  
    n_jobs=-1,  
)  
  
perm_df = pd.DataFrame({  
    "feature": X_importance.columns,  
    "importance_mean": perm.importances_mean,  
    "importance_std": perm.importances_std,  
}).sort_values("importance_mean", ascending=False)
```

7. Dyskusja wyników

7.1. Co mówią wyniki?

Wyniki wskazują, że satysfakcja pasażera jest w dużym stopniu przewidywalna na podstawie danych ankietowych i kontekstowych. Model pełny osiąga bardzo wysoką skuteczność, ale eksperyment z grupami cech pokazuje, skąd pochodzi ten wynik.

Najbardziej informacyjne są oceny usług. Model oparty tylko na ocenach samoopisowych osiąga $F1 = 0,934$, czyli niewiele mniej niż model pełny. Oznacza to, że odpowiedzi ankietowe wprost odzwierciedlają końcową satysfakcję. Jednocześnie model tylko na danych obiektywnych osiąga $F1$ około 0,73, więc obiektywne informacje o locie i pasażerze również mają znaczenie, ale nie wystarczają do jakości porównywalnej z pełnym modelem.

7.2. Czy opóźnienia są najważniejsze?

Intuicyjnie można oczekiwać, że opóźnienia będą głównym powodem niezadowolenia. W danych widać, że opóźnienia rzeczywiście obniżają odsetek zadowolonych pasażerów. Jednak korelacja opóźnień ze zmienną celu jest relatywnie słaba, a permutation importance nie wskazuje opóźnień jako najważniejszych cech.

To nie znaczy, że punktualność nie ma znaczenia. Oznacza raczej, że w tym zbiorze końcowa satysfakcja jest silniej związana z całym doświadczeniem obsługi: boardingiem, Wi-Fi, obsługą pokładową, bagażem, odprawą, komfortem siedzenia i czystością.

7.3. Znaczenie `type_of_travel` i `customer_type`

Najważniejszą cechą w permutation importance jest `type_of_travel`. Pasażerowie podróżujący służbowo znacznie częściej są zadowoleni niż pasażerowie podróżujący prywatnie. Może to wynikać z innych klas biletów, innych oczekiwań, innych tras lub innego sposobu finansowania podróży.

`customer_type` również jest bardzo ważne, ale jego interpretacja jest ograniczona. Nie mamy pełnej definicji tej zmiennej, dlatego nie należy budować na niej zbyt mocnych wniosków biznesowych bez dodatkowej dokumentacji. W raporcie należy traktować ją jako silny predyktor, ale nie jako w pełni wyjaśniony mechanizm.

7.4. Ograniczenia projektu

Najważniejsze ograniczenia:

- brak pełnej dokumentacji procesu zbierania danych,
- niepewna interpretacja `Customer Type`,
- założenie, że 0 w ocenach oznacza brak odpowiedzi,
- możliwość zależności ankietowych: zmienna celu i oceny usług pochodzą prawdopodobnie z tej samej ankiety,
- brak danych o cenie biletu, trasie, przewoźniku, lotnisku, godzinie lotu i historii klienta,
- brak zewnętrznej walidacji na danych z innego okresu lub innej linii lotniczej.

Szczególnie ważny jest punkt o danych ankietowych. Jeśli celem biznesowym jest przewidywanie satysfakcji przed ankietą, model pełny jest zbyt optymistyczny, bo korzysta z informacji, które powstają po podróży. W takim scenariuszu należałoby używać modelu opartego tylko na danych obiektywnych i kontekstowych.

8. Podsumowanie

8.1. Co się udało?

Udało się zrealizować pełny proces uczenia maszynowego:

- przeprowadzono eksploracyjną analizę danych,
- wykryto i opisano braki danych,
- przeanalizowano wartości θ w ocenach usług,
- przygotowano pipeline przetwarzania danych,
- zbudowano wiele modeli klasyfikacyjnych,
- porównano modele na walidacji i w walidacji krzyżowej,
- wykonano strojenie hiperparametrów,
- oceniono finalny model na osobnym zbiorze testowym,
- przeprowadzono analizę wyjaśnialności,
- porównano modele używające różnych grup cech.

Finalny model osiągnął bardzo wysoką jakość: $F1 = 0,959$ i $ROC-AUC = 0,995$ na zbiorze testowym. Wynik jest stabilny i potwierdzony walidacją krzyżową.

8.2. Jakie były problemy i jak je rozwiązano?

Pierwszym problemem były braki w `arrival_delay_in_minutes`. Ich liczba była niewielka, dlatego zastosowano imputację medianą oraz dodatkową flagę informującą o braku.

Drugim problemem była interpretacja wartości θ w kolumnach ocenowych. Zamiast traktować θ jako najgorszą ocenę, potraktowano je jako brak odpowiedzi. Dodano flagi `*_no_answer`, a zera zamieniono na braki danych.

Trzecim problemem była możliwość zbyt optymistycznej oceny modelu pełnego, ponieważ korzysta on z ocen ankietowych. Rozwiązaniem było dodanie eksperymentu z modelami opartymi tylko na danych obiektywnych. Dzięki temu raport pokazuje nie tylko najlepszy wynik, ale też praktyczne ograniczenia predykcji.

8.3. Jak można wykorzystać wyniki?

Wyniki mogą posłużyć do:

- identyfikacji najważniejszych elementów doświadczenia pasażera,
- projektowania ankiet satysfakcji,
- priorytetyzacji działań poprawiających obsługę,
- budowy systemu wczesnego ostrzegania o ryzyku niezadowolenia,
- segmentacji klientów i typów podróży,
- monitorowania jakości usług w czasie.

Najważniejszy wniosek biznesowy jest taki, że satysfakcja zależy przede wszystkim od jakości doświadczenia pasażera, a nie tylko od obiektywnych parametrów lotu. Oznacza to, że poprawa usług takich jak boarding, Wi-Fi, obsługa bagażu, odprawa, czystość i komfort siedzenia może mieć realny wpływ na ogólne zadowolenie.

8.4. Możliwe rozwinięcia

Projekt można rozwinąć przez:

- sprawdzenie modelu na danych z innej linii lotniczej,
- dodanie danych o cenie biletu, lotnisku, trasie i godzinie lotu,
- budowę osobnych modeli dla podróży służbowych i prywatnych,
- analizę progów decyzyjnych pod różne cele biznesowe,
- użycie SHAP do bardziej szczegółowej interpretacji pojedynczych predykcji,
- porównanie wyników z modelem uczonym wyłącznie na danych dostępnych przed lotem.

9. Załączniki

9.1. Załącznik 1. Pairplot wybranych cech



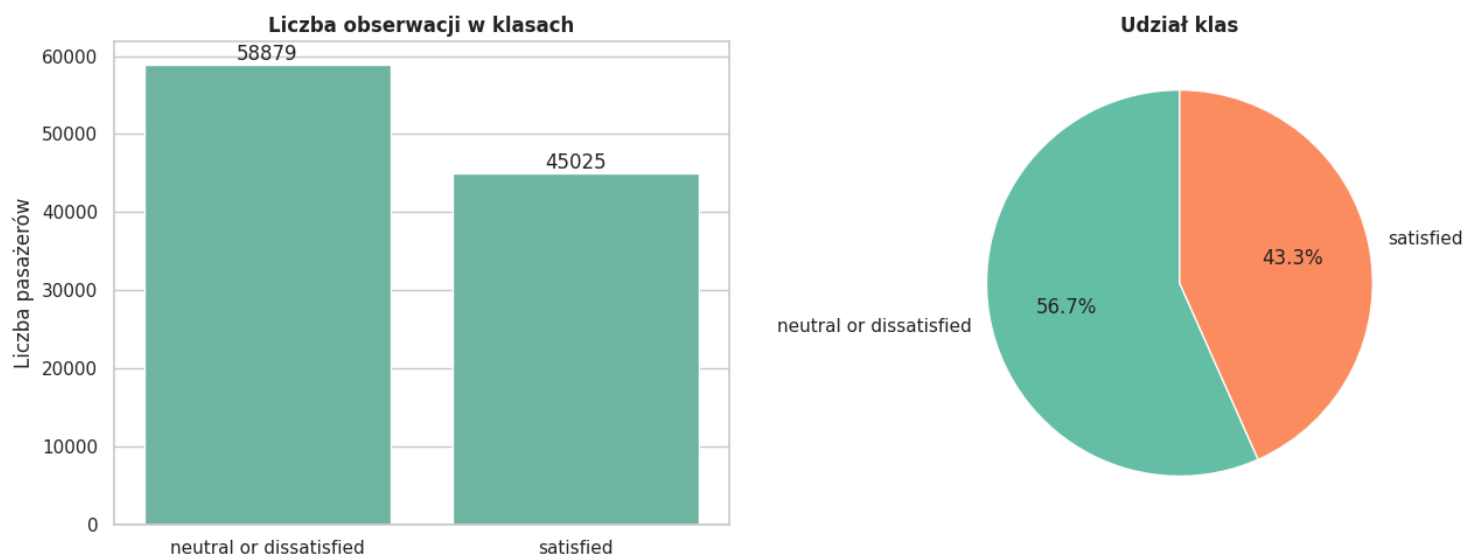
Rysunek 26: Pairplot wybranych cech liczbowych.

Wniosek: Pairplot pokazuje, że żadna z prostych par zmiennych liczbowych nie rozdziela klas satysfakcji jednoznacznie. Widać raczej nakładanie się obserwacji, co uzasadnia użycie modeli potrafiących korzystać z wielu cech naraz oraz z relacji nieliniowych. Wykres traktujemy jako ogólną diagnozę struktury danych, a nie jako podstawę do samodzielnej klasyfikacji.

Kod generujący wykres:

```
selected_for_pairplot = ["online_boarding", "inflight_entertainment", "seat_comfort",  
"flight_distance", "age", TARGET,]  
selected_for_pairplot = [col for col in selected_for_pairplot if col in train.columns]  
pair_sample = train[selected_for_pairplot].dropna().sample(  
    n=min(4000, len(train)),  
    random_state=RANDOM_STATE,  
)  
sns.pairplot(pair_sample, hue=TARGET, diag_kind="hist", corner=True, plot_kws={"alpha":  
0.35, "s": 12},)  
plt.suptitle("Pairplot wybranych cech", y=1.02)  
plt.show()
```

9.2. Załącznik 2. Rozkład zmiennej celu



Rysunek 27: Rozkład zmiennej celu.

Wniosek: Rozkład klas jest umiarkowanie niebalansowany: grupa `neutral or dissatisfied` jest większa, ale klasa `satisfied` nadal ma dużą reprezentację. Oznacza to, że model nie wymaga agresywnego balansowania danych, jednak sama accuracy mogłaby być myląca. Dlatego w dalszej części projektu jako kluczowe metryki przyjęto F1, precision, recall oraz ROC-AUC.

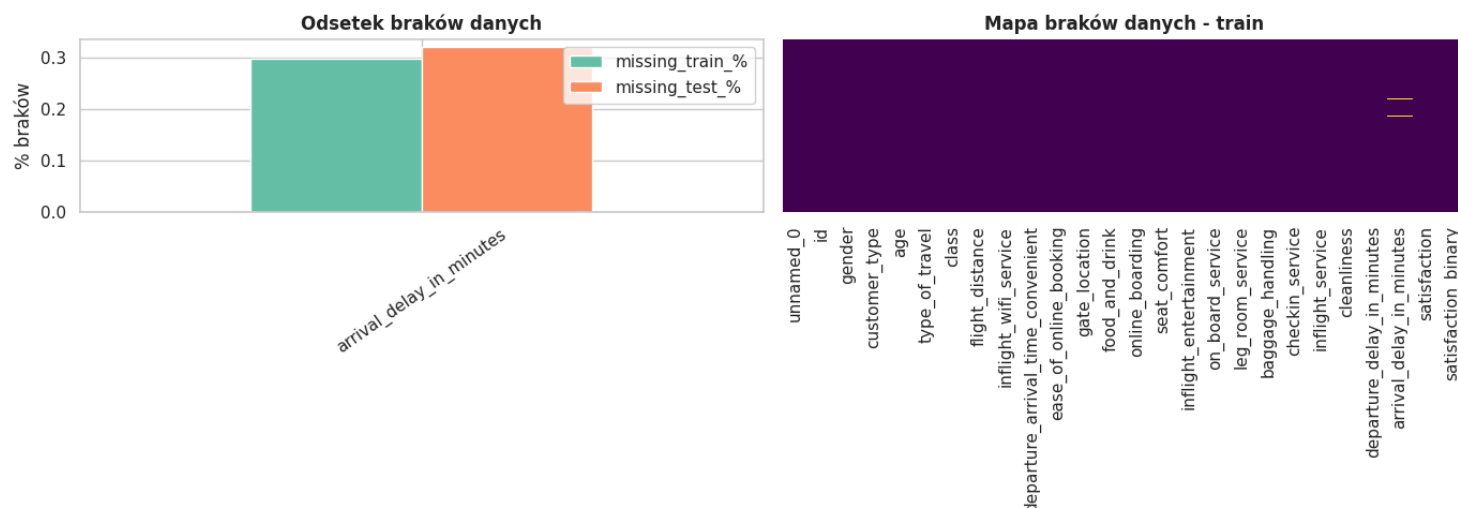
Kod generujący wykres:

```
target_counts = train[TARGET].value_counts()
target_share = train[TARGET].value_counts(normalize=True).mul(100).round(2)
display(pd.DataFrame({"count": target_counts, "share_%": target_share}))

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
sns.countplot(data=train, x=TARGET, order=target_counts.index, ax=axes[0])
axes[0].set_title("Liczba obserwacji w klasach")
axes[0].set_xlabel("")
axes[0].set_ylabel("Liczba pasażerów")
for container in axes[0].containers:
    axes[0].bar_label(container)

target_counts.plot(kind="pie", autopct="%1.1f%%", startangle=90, ax=axes[1])
axes[1].set_title("Udział klas")
axes[1].set_ylabel("")
plt.tight_layout()
plt.show()
```

9.3. Załącznik 3. Braki danych



Rysunek 28: Braki danych w train i test.

Wniosek: Braki danych są skoncentrowane praktycznie w jednej kolumnie, czyli `arrival_delay_in_minutes`. Skala braków jest bardzo mała względem liczby rekordów, więc nie ma potrzeby odrzucania obserwacji ani stosowania skomplikowanych metod imputacji. W modelowaniu użyto imputacji medianą oraz dodatkowej flagi informującej, że wartość opóźnienia przylotu była pierwotnie brakująca.

Kod generujący wykres:

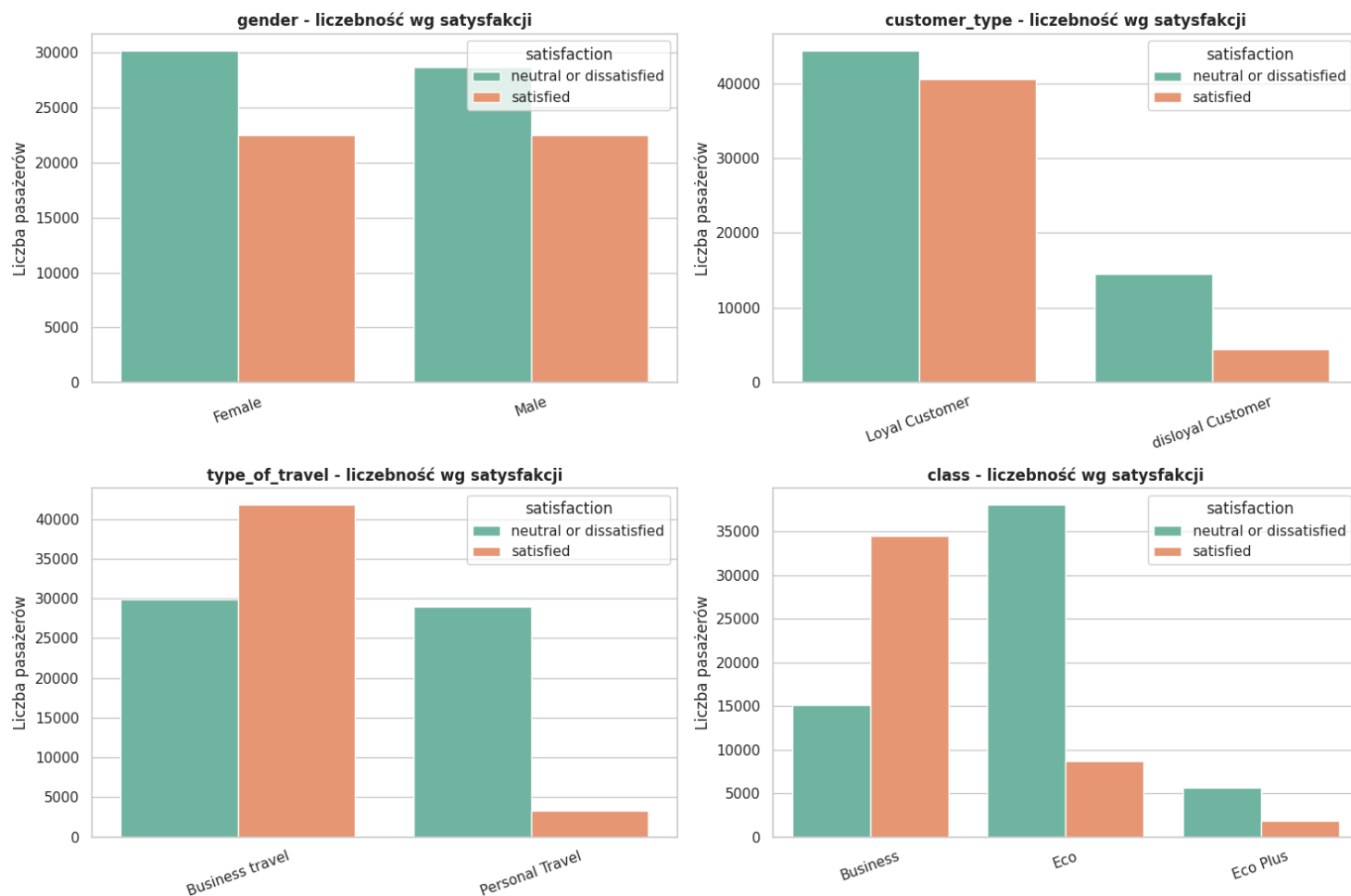
```
missing = pd.DataFrame({
    "missing_train": train.isna().sum(),
    "missing_train_%": train.isna().mean() * 100,
    "missing_test": test.isna().sum(),
    "missing_test_%": test.isna().mean() * 100,
}).sort_values("missing_train", ascending=False)

missing_nonzero = missing[
    (missing["missing_train"] > 0) | (missing["missing_test"] > 0)
]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
missing_nonzero[["missing_train%", "missing_test%"]].plot(
    kind="bar",
    ax=axes[0],
)
axes[0].set_title("Odsetek braków danych")
axes[0].set_ylabel("% braków")
axes[0].tick_params(axis="x", rotation=35)

sns.heatmap(train.isna(), cbar=False, yticklabels=False, ax=axes[1], cmap="viridis")
axes[1].set_title("Mapa braków danych - train")
plt.tight_layout()
plt.show()
```

9.4. Załącznik 4. Liczebności kategorii



Rysunek 29: Liczebności kategorii z podziałem na satysfakcję.

Wniosek: Wykres pokazuje nie tylko liczebność kategorii, ale też to, że niektóre segmenty mają wyraźnie inny rozkład satysfakcji. Szczególnie ważne są `type_of_travel` i `class`, gdzie różnice między podróżą biznesową, prywatną i klasą biletu są duże. To sugeruje, że kontekst podróży jest istotny i powinien pozostać w modelu.

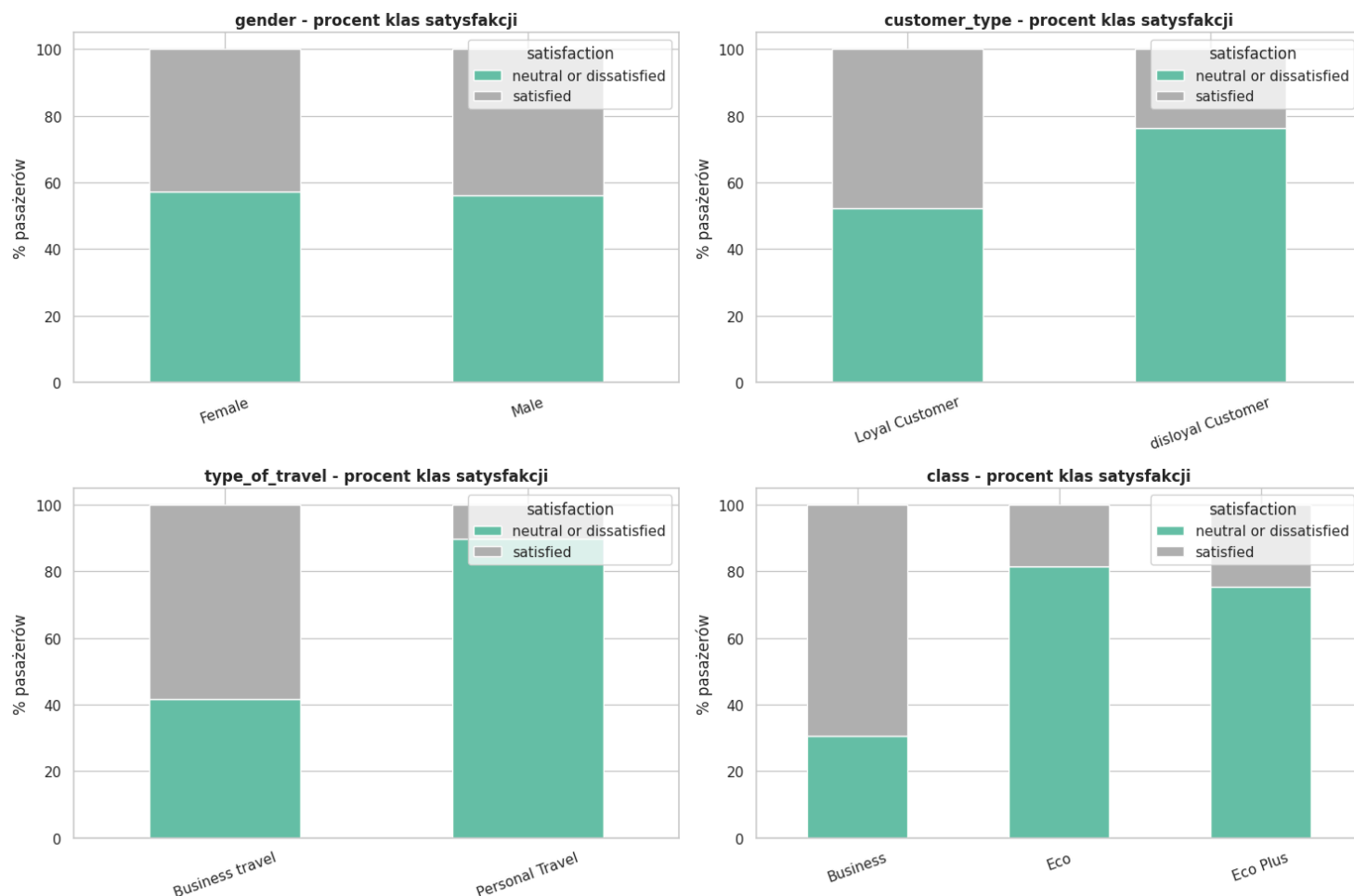
Kod generujący wykres:

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.ravel()
for ax, col in zip(axes, categorical_cols):
    order = train[col].value_counts().index
    sns.countplot(data=train, x=col, hue=TARGET, order=order, ax=ax)
    ax.set_title(f"{col} - liczebność wg satysfakcji")
    ax.set_xlabel("")
    ax.set_ylabel("Liczba pasażerów")
    ax.tick_params(axis="x", rotation=20)

for ax in axes[len(categorical_cols):]:
    ax.set_axis_off()

plt.tight_layout()
plt.show()
```

9.5. Załącznik 5. Procent satysfakcji w kategoriach



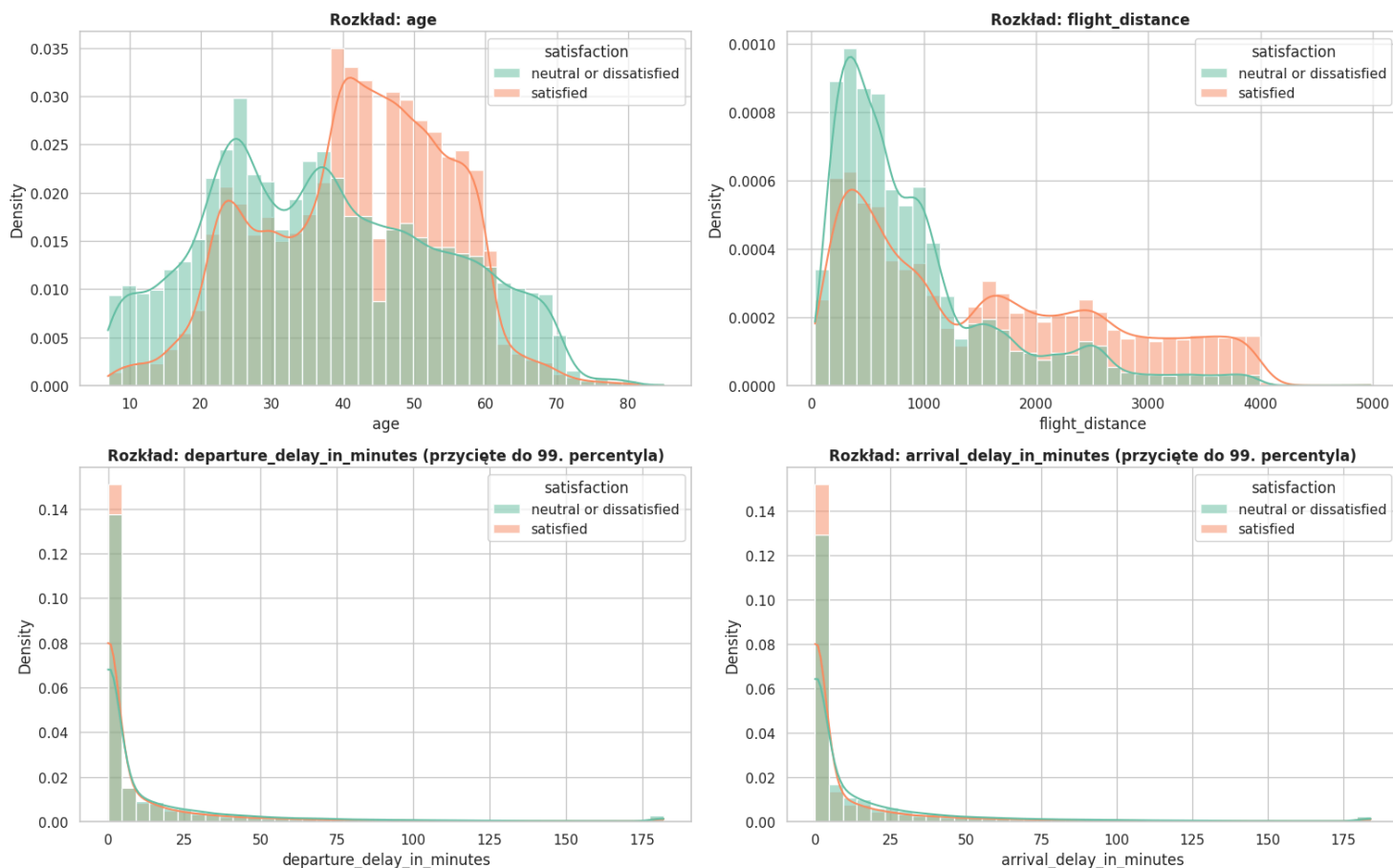
Rysunek 30: Udział klas satysfakcji w zmiennych kategoriycznych.

Wniosek: Procentowe zestawienie ułatwia porównanie kategorii niezależnie od ich liczebności. Największe różnice widać dla typu podróży oraz klasy biletu, co oznacza, że pasażerowie nie oceniają doświadczenia w oderwaniu od kontekstu podróży. Wniosek ten później potwierdza eksperyment z grupami cech oraz permutation importance.

Kod generujący wykres:

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.ravel()
for ax, col in zip(axes, categorical_cols):
    tab = pd.crosstab(train[col], train[TARGET], normalize="index").mul(100)
    tab = tab.reindex(train[col].value_counts().index)
    tab.plot(kind="bar", stacked=True, ax=ax, colormap="Set2")
    ax.set_title(f"{col} - procent klas satysfakcji")
    ax.set_xlabel("")
    ax.set_ylabel("% pasażerów")
    ax.tick_params(axis="x", rotation=20)
    ax.legend(title=TARGET, loc="upper right")
for ax in axes[len(categorical_cols):]:
    ax.set_axis_off()
plt.tight_layout()
plt.show()
```

9.6. Załącznik 6. Rozkłady zmiennych liczbowych



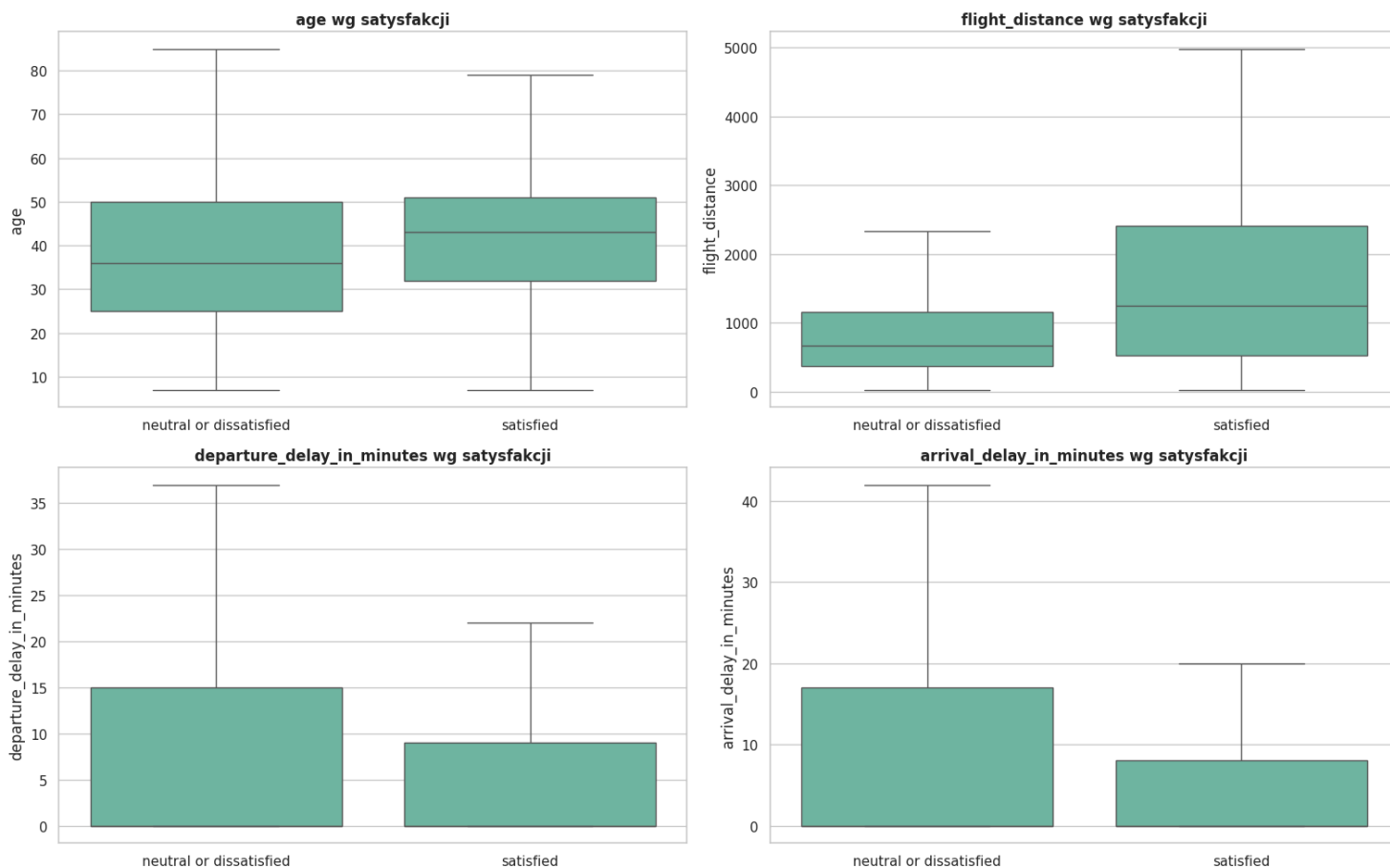
Rysunek 31: Rozkłady zmiennych liczbowych z podziałem na satysfakcję.

Wniosek: Rozkłady zmiennych liczbowych pokazują, że wiek, dystans i opóźnienia różnią się między klasami, ale nie tworzą prostego progu oddzielającego pasażerów zadowolonych od niezadowolonych. Opóźnienia mają silnie skośne rozkłady, a dystans lotu również nie jest symetryczny. Z tego powodu w feature engineering dodano cechy logarytmiczne i flagi opóźnień.

Kod generujący wykres:

```
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.ravel()
for ax, col in zip(axes, continuous_cols):
    plot_df = train[[col, TARGET]].dropna().copy()
    title_suffix = ""
    if "delay" in col:
        plot_df[col] = plot_df[col].clip(upper=plot_df[col].quantile(0.99))
        title_suffix = "(przycięte do 99. percentyla)"
    sns.histplot(data=plot_df, x=col, hue=TARGET, kde=True, bins=40, stat="density",
common_norm=False, ax=ax,)
    ax.set_title(f"Rozkład: {col} {title_suffix}")
    ax.set_xlabel(col)
plt.tight_layout()
plt.show()
```

9.7. Załącznik 7. Boxploty zmiennych liczbowych



Rysunek 32: Boxploty zmiennych liczbowych.

Wniosek: Boxploty potwierdzają dużą liczbę wartości odstających, szczególnie dla opóźnień. Nie są to jednak oczywiste błędy danych, ponieważ bardzo duże opóźnienia realnie mogą występować w lotnictwie. Zamiast usuwać rekordy, zachowano je w danych i dodano transformacje, które pomagają modelom korzystać z tej informacji stabilniej.

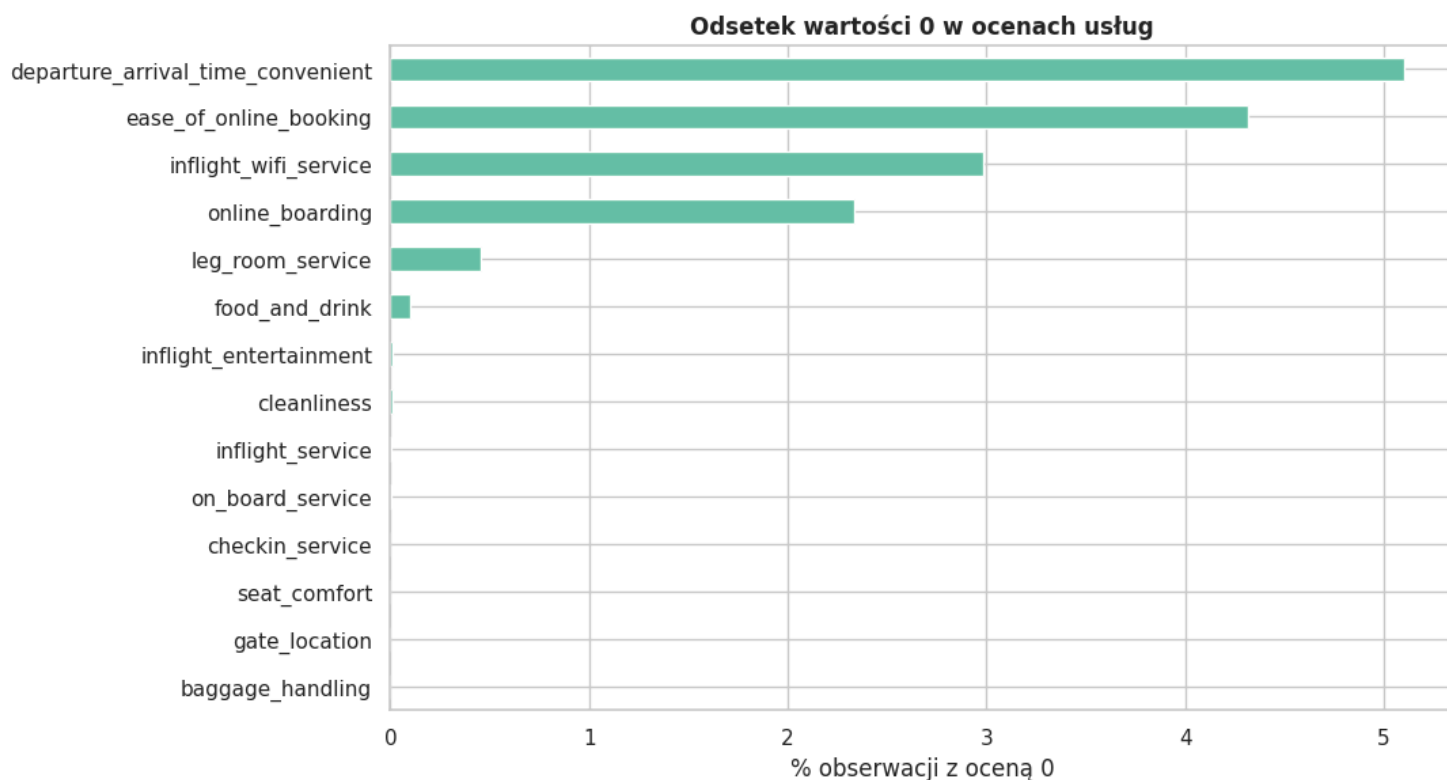
Kod generujący wykres:

```
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.ravel()

for ax, col in zip(axes, continuous_cols):
    plot_df = train[[col, TARGET]].dropna().copy()
    if "delay" in col:
        plot_df[col] = plot_df[col].clip(upper=plot_df[col].quantile(0.99))
    sns.boxplot(data=plot_df, x=TARGET, y=col, showfliers=False, ax=ax)
    ax.set_title(f"{col} wg satysfakcji")
    ax.set_xlabel("")

plt.tight_layout()
plt.show()
```

9.8. Załącznik 8. Wartości 0 w ocenach usług



Rysunek 33: Odsetek wartości 0 w kolumnach ocenowych.

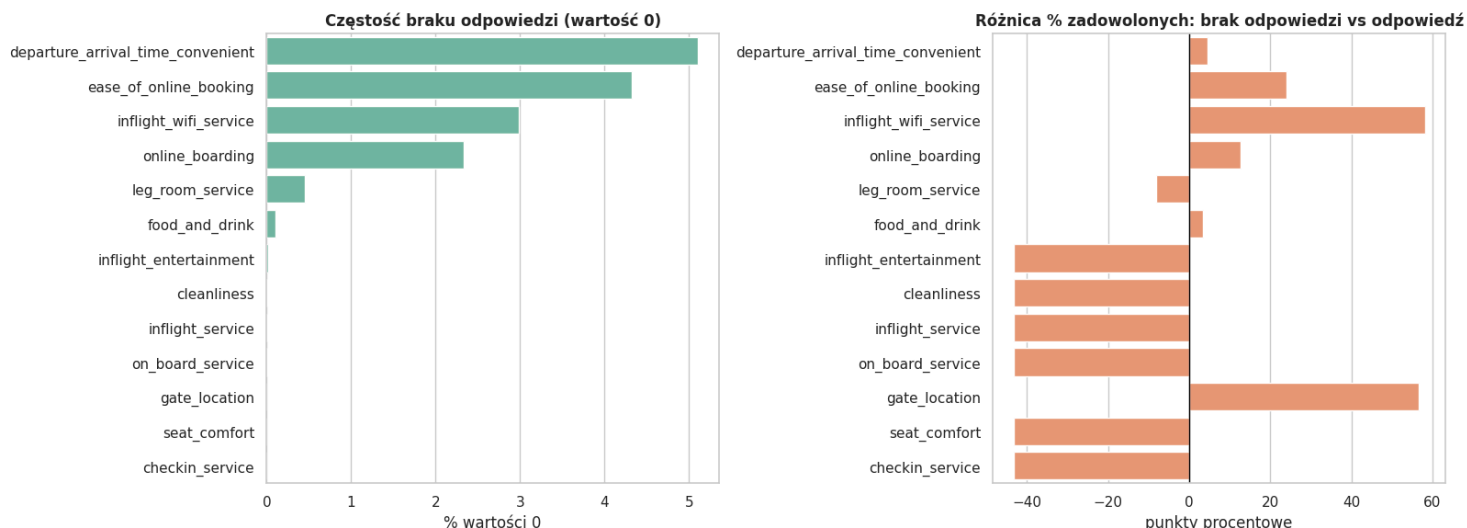
Wniosek: Wartości 0 nie występują równomiernie we wszystkich ocenach usług, co wskazuje, że nie należy traktować ich jak zwykłej najniższej oceny. Najbardziej sensowna interpretacja to brak odpowiedzi lub brak możliwości oceny danej usługi. Dlatego w projekcie dodano flagi *_no_answer, a same zera zamieniono na braki danych przed imputacją.

Kod generujący wykres:

```
zero_ratings = train[rating_cols].eq(0).sum().sort_values(ascending=False)
zero_ratings_pct = train[rating_cols].eq(0).mean().mul(100).sort_values(ascending=False)
zero_table = pd.DataFrame({"zero_count": zero_ratings, "zero_%": zero_ratings_pct})
display(zero_table)

plt.figure(figsize=(11, 6))
zero_table["zero_%"].sort_values().plot(kind="barh", color="#66c2a5")
plt.title("Odsetek wartości 0 w ocenach usług")
plt.xlabel("% obserwacji z oceną 0")
plt.ylabel("")
plt.tight_layout()
plt.show()
```

9.9. Załącznik 9. Interpretacja wartości 0



Rysunek 34: Wpływ braku odpowiedzi na odsetek satysfakcji.

Wniosek: Porównanie satysfakcji dla odpowiedzi zerowych i niezerowych pokazuje, że sam fakt braku odpowiedzi może zawierać sygnał predykcyjny. Jest to ważne, bo usunięcie zer bez dodatkowej flagi pozbawiłoby model tej informacji. Przyjęte rozwiązanie rozdziela dwa znaczenia: poziom oceny usługi oraz fakt, że pasażer nie udzielił oceny.

Kod generujący wykres:

```
zero_response_rows = []
for col in rating_cols:
    zero_mask = train[col].eq(0)
    answered_mask = ~zero_mask
    zero_response_rows.append({
        "feature": col, "zero_count": int(zero_mask.sum()), "zero_%": zero_mask.mean() *
100, "satisfied_rate_zero_%": train.loc[zero_mask, TARGET_BINARY].mean() * 100,
"satisfied_rate_answered_%": train.loc[answered_mask, TARGET_BINARY].mean() * 100,})

zero_response_df = pd.DataFrame(zero_response_rows)
zero_response_df["difference_zero_minus_answered_pp"] = (
    zero_response_df["satisfied_rate_zero_%"]
    -zero_response_df["satisfied_rate_answered_%"]
)

zero_response_df = zero_response_df.sort_values("zero_%", ascending=False)
plot_zero_df = zero_response_df[zero_response_df["zero_count"] > 0].copy()
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
sns.barplot(data=plot_zero_df, y="feature", x="zero_%", ax=axes[0], color="#66c2a5")
axes[0].set_title("Częstość braku odpowiedzi (wartość 0)")
axes[0].set_xlabel("% wartości 0")
axes[0].set_ylabel("")

sns.barplot(data=plot_zero_df, y="feature", x="difference_zero_minus_answered_p,
ax=axes[1],color="#fc8d62",)
axes[1].axvline(0, color="black", linewidth=1)
axes[1].set_title("Różnica % zadowolonych: brak odpowiedzi vs odpowiedź")
axes[1].set_xlabel("punkty procentowe")
axes[1].set_ylabel("")
plt.tight_layout()
plt.show()
```

9.10. Załącznik 10. Rozkłady ocen usług



Rysunek 35: Rozkład satysfakcji dla poszczególnych ocen usług.

Wniosek: Wykres pokazuje ogólną zależność: im wyższa ocena usługi, tym większy udział pasażerów zadowolonych. Najbardziej widoczne jest to dla cech bezpośrednio związanych z doświadczeniem podróży, takich jak boarding, Wi-Fi i rozrywka pokładowa. Ze względu na dużą liczbę paneli wykres jest bardziej syntetycznym potwierdzeniem trendu niż miejscem do analizy każdej usługi osobno.

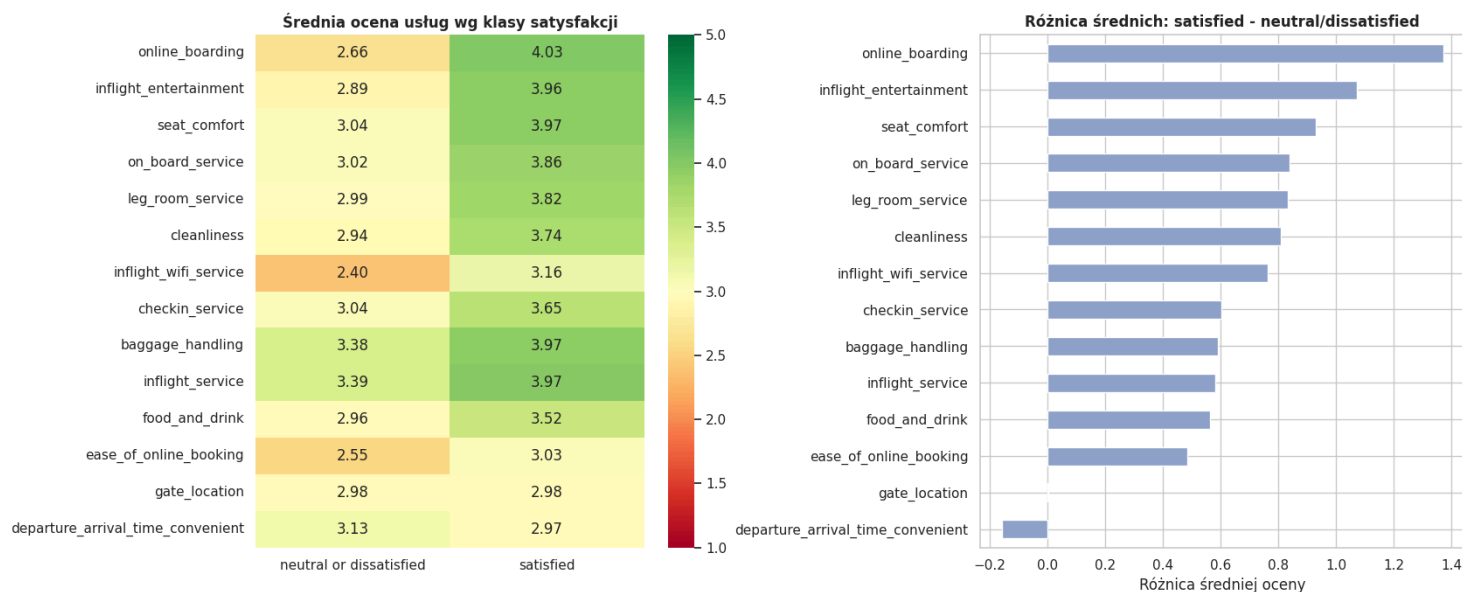
Kod generujący wykres:

```
for ax, col in zip(axes, rating_cols):
    tab = pd.crosstab(train[col], train[TARGET], normalize="index").mul(100)
    tab.plot(kind="bar", stacked=True, ax=ax, colormap="Set2", width=0.85)
    ax.set_title(col)
    ax.set_xlabel("Ocena")
    ax.set_ylabel("% pasażerów")
    ax.legend(title=TARGET, fontsize=8)

for ax in axes[len(rating_cols):]:
    ax.set_axis_off()

plt.tight_layout()
plt.show()
```

9.11. Załącznik 11. Średnie oceny usług



Rysunek 36: Średnie oceny usług według klasy satysfakcji.

Wniosek: Średnie oceny usług są wyraźnie wyższe w grupie **satisfied** prawie dla wszystkich aspektów podróży. Największe różnice dotyczą usług, które pasażer mocno odczuwa w trakcie całego procesu podróży, na przykład onboarding, komfortu i obsługi. To tłumaczy, dlaczego modele używające cech ankietowych osiągają bardzo wysoką jakość predykcji.

Kod generujący wykres:

```
rating_means = train.groupby(TARGET)[rating_cols].mean().T
rating_means["diff_satisfied_minus_neutral"] = (
    rating_means["satisfied"] - rating_means["neutral or dissatisfied"]
)
rating_means = rating_means.sort_values("diff_satisfied_minus_neutral", ascending=False)

fig, axes = plt.subplots(1, 2, figsize=(17, 7))
sns.heatmap(rating_means[["neutral or dissatisfied", "satisfied"]], annot=True, fmt=".2f",
            cmap="RdYlGn", vmin=1, vmax=5, ax=axes[0],)
axes[0].set_title("Średnia ocena usług wg klasy satysfakcji")
axes[0].set_xlabel("")
axes[0].set_ylabel("")

rating_means["diff_satisfied_minus_neutral"].sort_values().plot(
    kind="barh",
    ax=axes[1],
    color="#8da0cb",
)
axes[1].set_title("Różnica średnich: satisfied - neutral/dissatisfied")
axes[1].set_xlabel("Różnica średniej oceny")
axes[1].set_ylabel("")
plt.tight_layout()
plt.show()
```

9.12. Załącznik 12. Macierz korelacji



Rysunek 37: Macierz korelacji zmiennych liczbowych.

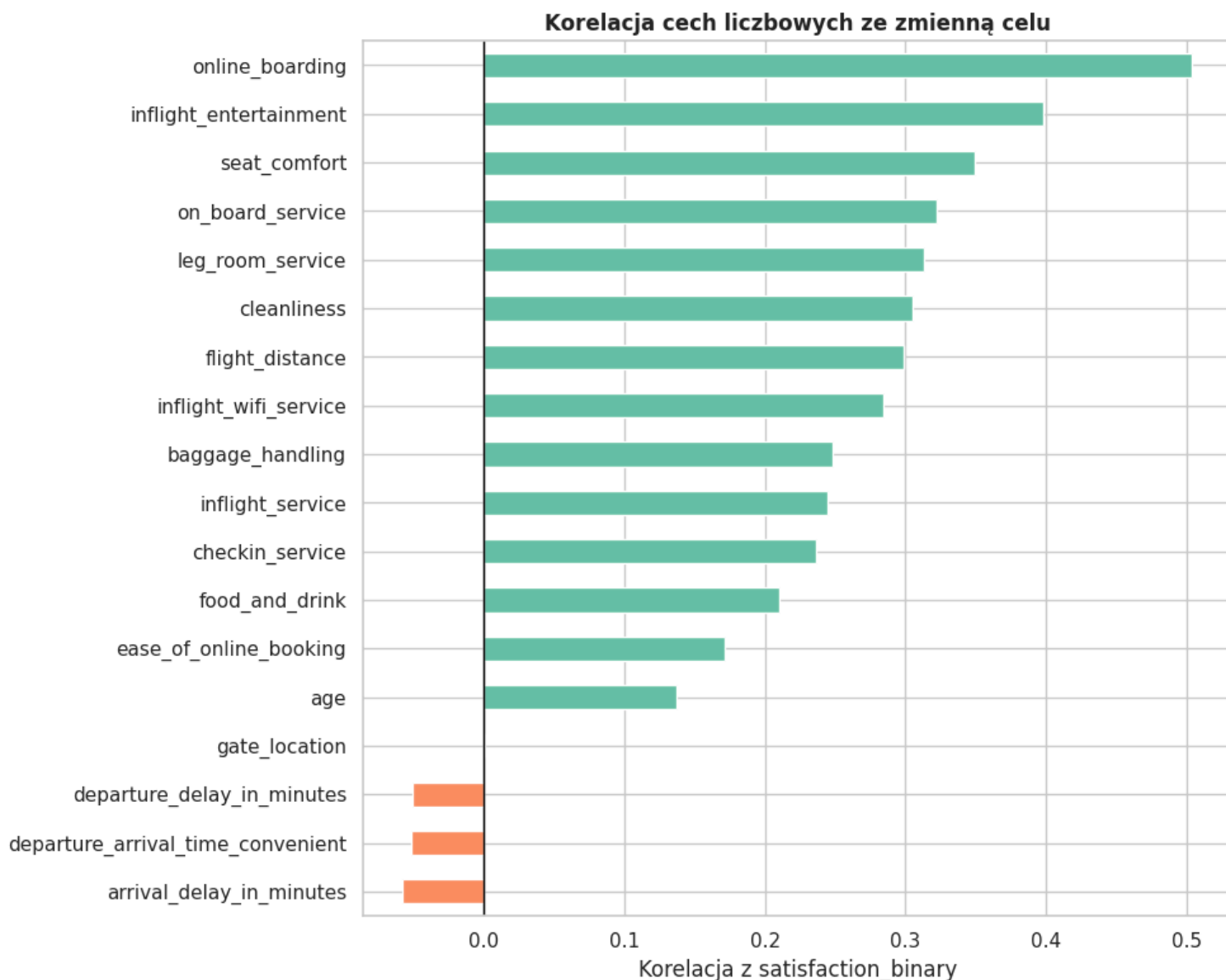
Wniosek: Macierz korelacji pokazuje dwie ważne rzeczy: silną zależność między opóźnieniem odlotu i przylotu oraz grupowanie się ocen usług. Korelacje są w zakresie od -1 do 1 i opisują zależności liniowe, więc nie wyczerpują całej informacji w danych. Mimo to wykres pomaga rozpoznać cechy powiązane ze sobą i cechy potencjalnie istotne dla modelu.

Kod generujący wykres:

```
corr_cols = continuous_cols + rating_cols + [TARGET_BINARY]
corr = train[corr_cols].corr(numeric_only=True)

plt.figure(figsize=(15, 12))
sns.heatmap(corr, cmap="vlag", center=0, linewidths=0.3, cbar_kws={"shrink": 0.8},
)
plt.title("Macierz korelacji zmiennych liczbowych")
plt.tight_layout()
plt.show()
```

9.13. Załącznik 13. Korelacje ze zmienną celu



Rysunek 38: Korelacja cech liczbowych ze zmienną celu.

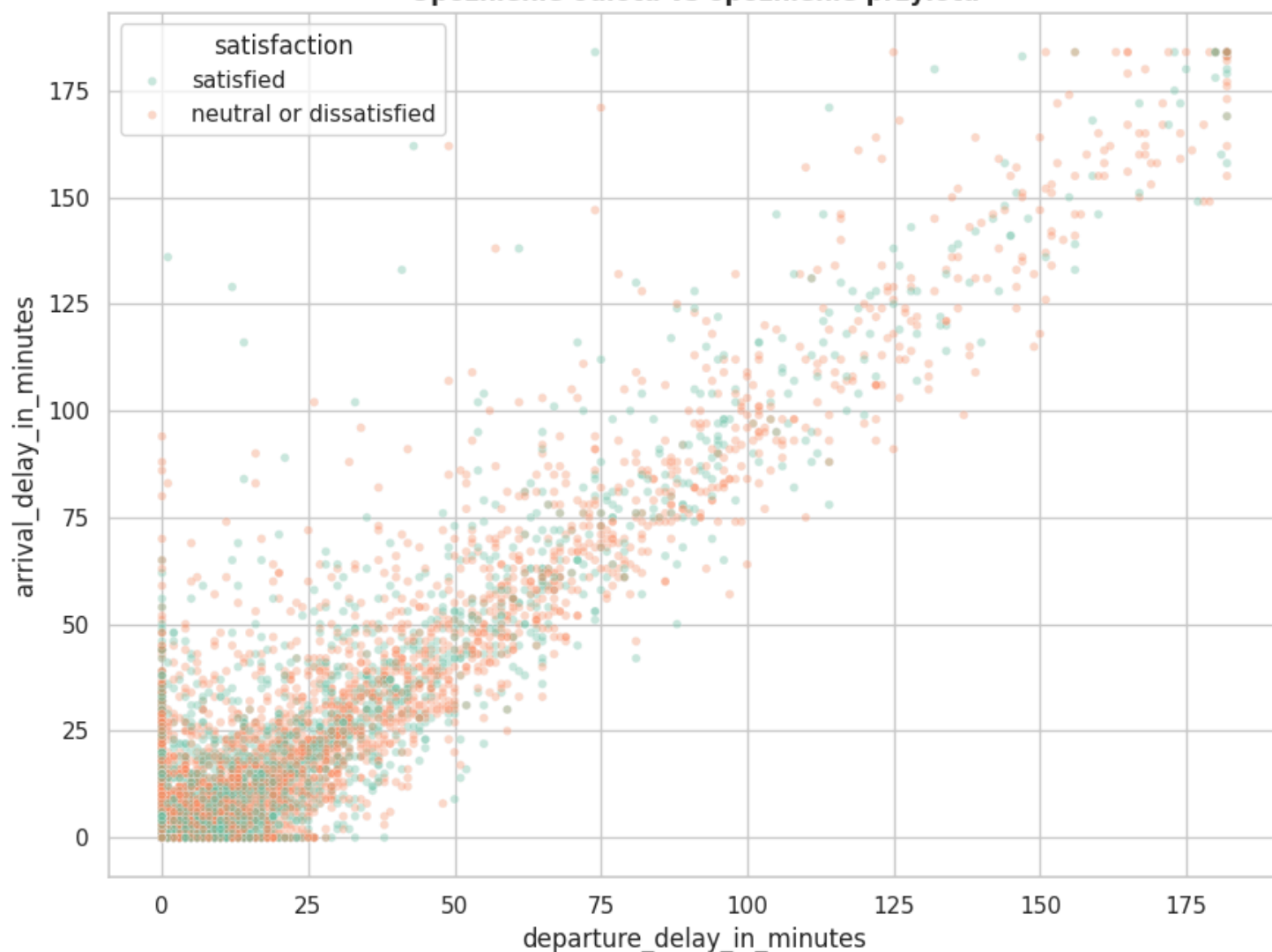
Wniosek: Najwyższe dodatnie korelacje ze zmienną celu mają cechy usługowe, szczególnie `online_boarding`, `inflight_wifi_service` oraz oceny komfortu i obsługi. Opóźnienia mają korelację ujemną, ale dużo słabszą niż oceny usług. Oznacza to, że w tym zbiorze subiektywna ocena doświadczenia jest silniejszym sygnałem niż same parametry operacyjne lotu.

Kod generujący wykres:

```
corr_cols = continuous_cols + rating_cols + [TARGET_BINARY]
corr = train[corr_cols].corr(numeric_only=True)
target_corr = corr[TARGET_BINARY].drop(TARGET_BINARY).sort_values()
display(target_corr.to_frame("correlation_with_satisfaction"))
plt.figure(figsize=(10, 8))
colors = ["#fc8d62" if value < 0 else "#66c2a5" for value in target_corr]
target_corr.plot(kind="barh", color=colors)
plt.title("Korelacja cech liczbowych ze zmienną celu")
plt.xlabel("Korelacja z satisfaction_binary")
plt.ylabel("")
plt.axvline(0, color="black", linewidth=1)
plt.tight_layout()
plt.show()
```

9.14. Załącznik 14. Opóźnienie odlotu i przylotu

Opóźnienie odlotu vs opóźnienie przylotu



Rysunek 39: Zależność opóźnienia odlotu i przylotu.

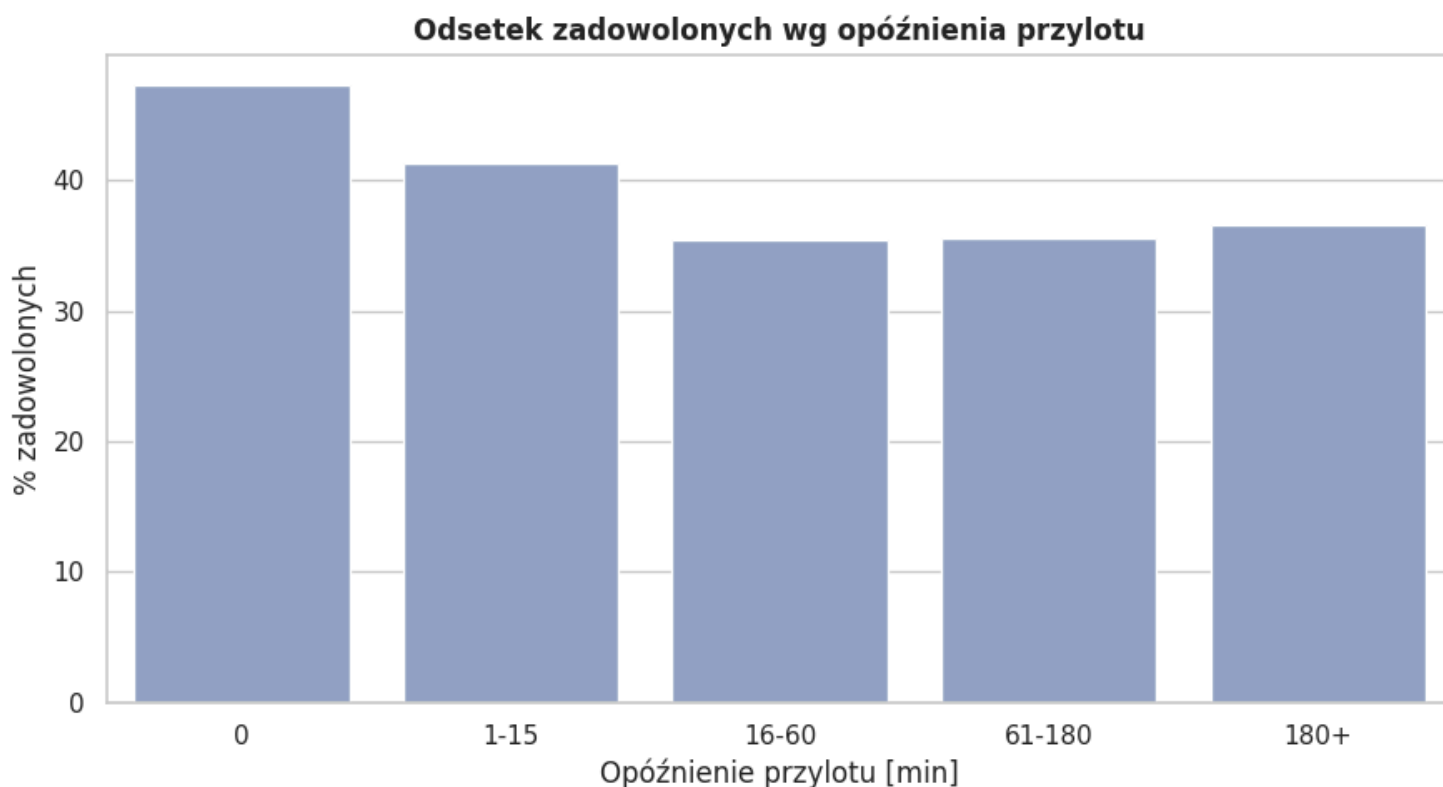
Wniosek: Punkty układają się wzdłuż przekątnej, co potwierdza bardzo silny związek między opóźnieniem odlotu i przylotu. Jednocześnie kolory klas mocno się nakładają, więc samo opóźnienie nie wystarcza do pewnego rozdzielenia satysfakcji. Opóźnienia są ważnym elementem opisu lotu, ale model musi łączyć je z ocenami usług i kontekstem podróży.

Kod generujący wykres:

```
for col in delay_cols:
    delay_sample[col] = delay_sample[col].clip(upper=train[col].quantile(0.99))

plt.figure(figsize=(9, 7))
sns.scatterplot(data=delay_sample, x="departure_delay_in_minutes",
                y="arrival_delay_in_minutes", hue=TARGET, alpha=0.35,s=18,)
plt.title("Opóźnienie odlotu vs opóźnienie przylotu")
plt.tight_layout()
plt.show()
```

9.15. Załącznik 15. Satysfakcja według opóźnienia



Rysunek 40: Satysfakcja według przedziału opóźnienia przylotu.

Wniosek: Wraz ze wzrostem opóźnienia przylotu udział pasażerów zadowolonych spada, co jest intuicyjne biznesowo. Efekt nie jest jednak tak silny, aby samodzielnie wyjaśnić wynik ankiety, ponieważ część pasażerów pozostaje zadowolona mimo opóźnień. To potwierdza, że punktualność jest ważna, ale nie zastępuje informacji o jakości obsługi.

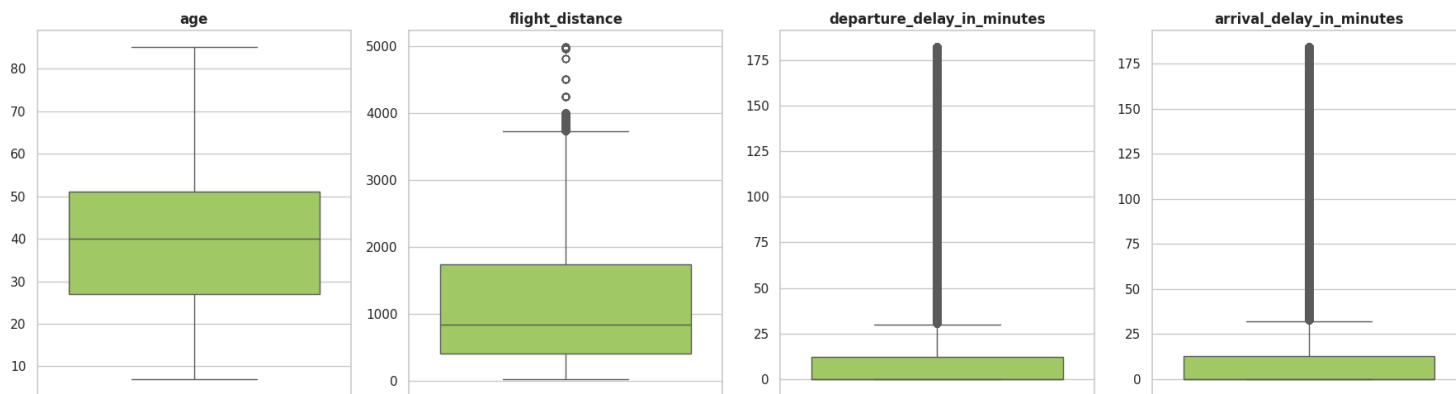
Kod generujący wykres:

```
delay_bins = [-1, 0, 15, 60, 180, np.inf]
delay_labels = ["0", "1-15", "16-60", "61-180", "180+"]
segment_df["arrival_delay_bin"] = pd.cut(segment_df["arrival_delay_in_minutes"],
bins=delay_bins, labels=delay_labels,)

delay_sat = (segment_df.groupby("arrival_delay_bin", observed=False)
[TARGET_BINARY].agg(count="size", satisfied_rate="mean").assign(satisfied_rate_pct=lambda
x: x["satisfied_rate"] * 100))

plt.figure(figsize=(9, 5))
sns.barplot(
    data=delay_sat.reset_index(),
    x="arrival_delay_bin",
    y="satisfied_rate_pct",
    color="#8da0cb",
)
plt.title("Odsetek zadowolonych wg opóźnienia przylotu")
plt.xlabel("Opóźnienie przylotu [min]")
plt.ylabel("% zadowolonych")
plt.tight_layout()
plt.show()
```

9.16. Załącznik 16. Outliery zmiennych liczbowych



Rysunek 41: Analiza obserwacji odstających dla zmiennych liczbowych.

Wniosek: Najwięcej obserwacji odstających występuje w zmiennych opóźnień, co wynika z ich naturalnie skośnego rozkładu. Te wartości nie zostały usunięte, ponieważ reprezentują rzeczywiste sytuacje, które mogą wpływać na ocenę pasażera. Zamiast tego model otrzymał zarówno oryginalne zmienne, jak i cechy pomocnicze, na przykład logarytmy oraz flagi wystąpienia opóźnienia.

Kod generujący wykres:

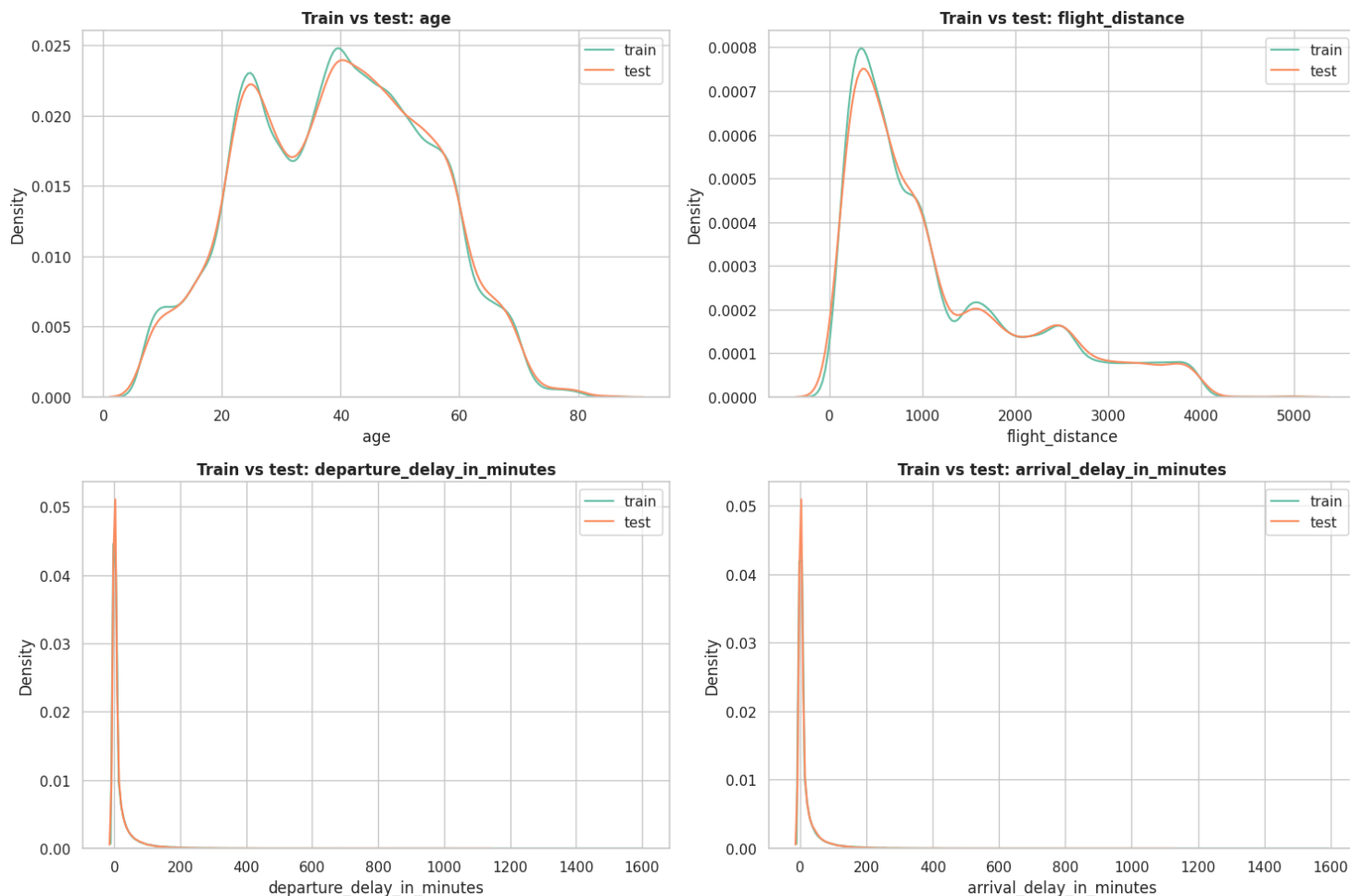
```
outlier_table = iqr_outlier_summary(train, continuous_cols)
display(outlier_table)

fig, axes = plt.subplots(1, len(continuous_cols), figsize=(4.5 * len(continuous_cols), 5))
if len(continuous_cols) == 1:
    axes = [axes]

for ax, col in zip(axes, continuous_cols):
    plot_series = train[col].dropna().copy()
    if "delay" in col:
        plot_series = plot_series.clip(upper=plot_series.quantile(0.99))
    sns.boxplot(y=plot_series, ax=ax, color="#a6d854")
    ax.set_title(col)
    ax.set_ylabel("")

plt.tight_layout()
plt.show()
```

9.17. Załącznik 17. Porównanie rozkładów train i test



Rysunek 42: Porównanie rozkładów zmiennych liczbowych w train i test.

Wniosek: Porównanie pokazuje, że rozkłady zmiennych w train i test są do siebie zbliżone. Nie widać dużego przesunięcia danych, które mogłoby sztucznie pogorszyć albo poprawić wynik testowy. Dzięki temu zbiór testowy można traktować jako sensowną końcową kontrolę jakości modelu.

Kod generujący wykres:

```
compare_numeric = continuous_cols + rating_cols

fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.ravel()

for ax, col in zip(axes, continuous_cols):
    sns.kdeplot(train[col].dropna(), label="train", ax=ax)
    sns.kdeplot(test[col].dropna(), label="test", ax=ax)
    ax.set_title(f"Train vs test: {col}")
    ax.legend()

plt.tight_layout()
plt.show()
```

9.18. Załącznik 18. Porównanie celu train i test



Rysunek 43: Porównanie rozkładu zmiennej celu w train i test.

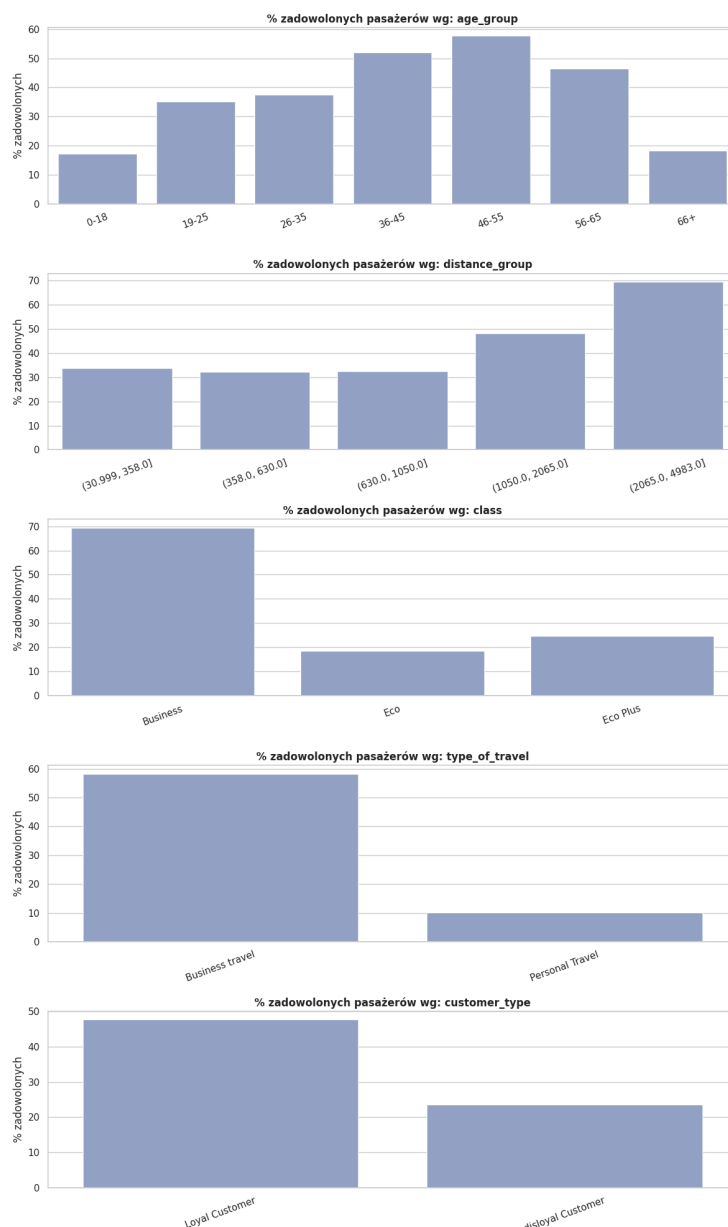
Wniosek: Proporcje klas w zbiorach train i test są bardzo podobne. To ważne, ponieważ metryki klasyfikacji zależą od rozkładu klas, szczególnie gdy raportujemy accuracy i F1. Podobny rozkład celu zwiększa wiarygodność porównania walidacji z końcowym wynikiem na teście.

Kod generujący wykres:

```
target_compare = pd.concat([
    train[TARGET].value_counts(normalize=True).rename("train"),
    test[TARGET].value_counts(normalize=True).rename("test"),
],
    axis=1,
).mul(100)
display(target_compare)

target_compare.plot(kind="bar", figsize=(8, 5), color=["#66c2a5", "#fc8d62"])
plt.title("Porównanie rozkładu zmiennej celu: train vs test")
plt.ylabel("% obserwacji")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

9.19. Załącznik 19. Satysfakcja w segmentach



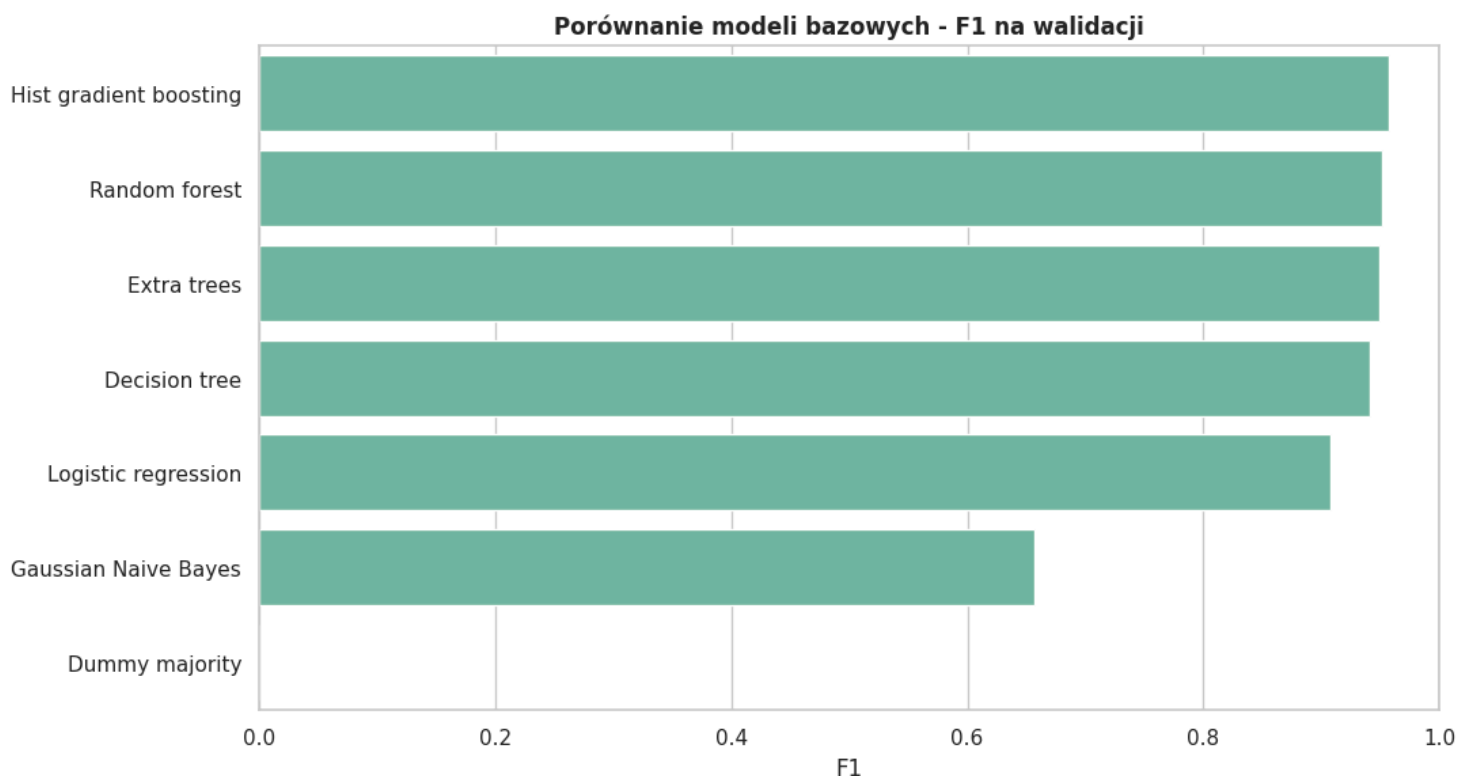
Rysunek 44: Satysfakcja w segmentach pasażerów.

Wniosek: Segmentacja pokazuje, że najwyższa satysfakcja pojawia się w grupach związanych z podróżą biznesową, klasą Business i dłuższymi dystansami. Nie oznacza to przyczynowości, ale wskazuje, że profil pasażera oraz warunki podróży mocno współwystępują z satysfakcją. Wykres uzasadnia pozostawienie cech segmentacyjnych w modelach.

Kod generujący wykres:

```
segment_df = train.copy()
segment_df["age_group"] = pd.cut(segment_df["age"], bins=[0, 18, 25, 35, 45, 55, 65, 100],
labels=["0-18", "19-25", "26-35", "36-45", "46-55", "56-65", "66+"],)
segment_df["distance_group"] = pd.qcut(segment_df["flight_distance"], q=5,
duplicates="drop")
segment_cols = ["age_group", "distance_group", "class", "type_of_travel", "customer_type"]
fig, axes = plt.subplots(len(segment_cols), 1, figsize=(12, 4 * len(segment_cols)))
#...
plt.tight_layout()
plt.show()
```

9.20. Załącznik 20. Porównanie modeli bazowych



Rysunek 45: Porównanie F1 modeli bazowych.

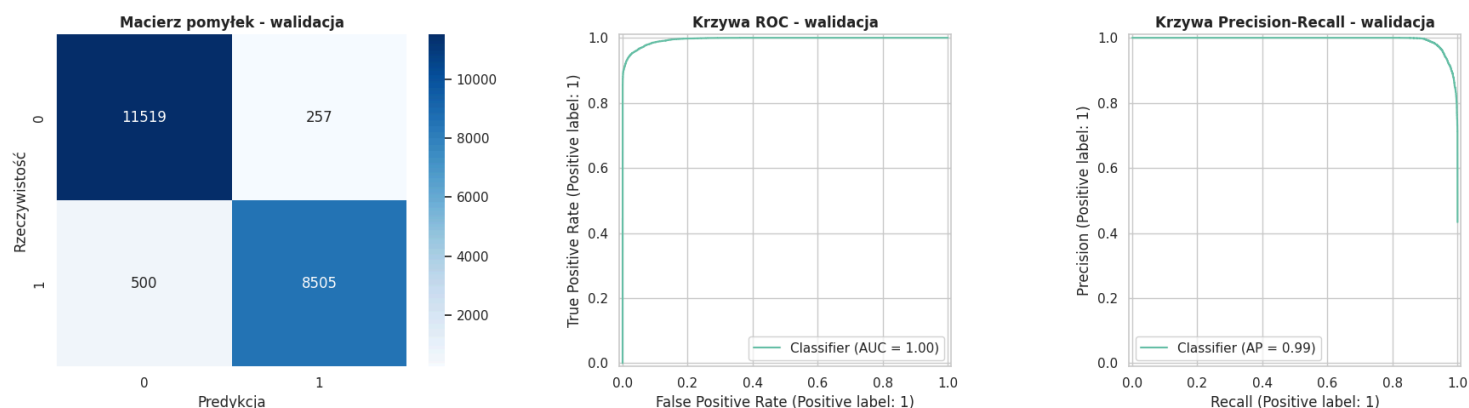
Wniosek: Porównanie F1 pokazuje wyraźną przewagę modeli drzewiastych i boostingowych nad prostszymi punktami odniesienia. Histogram gradient boosting osiąga najlepszy wynik, a Random Forest i Extra Trees są niewiele słabsze. Dummy majority ma F1 równe 0 dla klasy pozytywnej, co pokazuje, dlaczego sama accuracy nie wystarcza do oceny modelu.

Kod generujący wykres:

```
results_df = pd.DataFrame(results).sort_values("f1", ascending=False)
display(results_df)

plt.figure(figsize=(11, 6))
sns.barplot(data=results_df, y="model", x="f1", color="#66c2a5")
plt.title("Porównanie modeli bazowych - F1 na walidacji")
plt.xlabel("F1")
plt.ylabel("")
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```

9.21. Załącznik 21. Walidacja najlepszego modelu



Rysunek 46: Macierz pomyłek, ROC i Precision-Recall dla najlepszego modelu bazowego.

Wniosek: Zestawienie macierzy pomyłek, ROC i Precision-Recall pokazuje, że najlepszy model dobrze rozdziela obie klasy na zbiorze walidacyjnym. Krzywe wskazują wysoką jakość niezależnie od pojedynczego progu decyzyjnego. Macierz pomyłek pozwala dodatkowo zobaczyć, jakiego typu błędy model popełnia najczęściej.

Kod generujący wykres:

```
best_model_name = results_df.iloc[0]["model"]
best_pipe = trained_models[best_model_name]

y_valid_pred = best_pipe.predict(X_valid)
y_valid_score = get_positive_scores(best_pipe, X_valid)

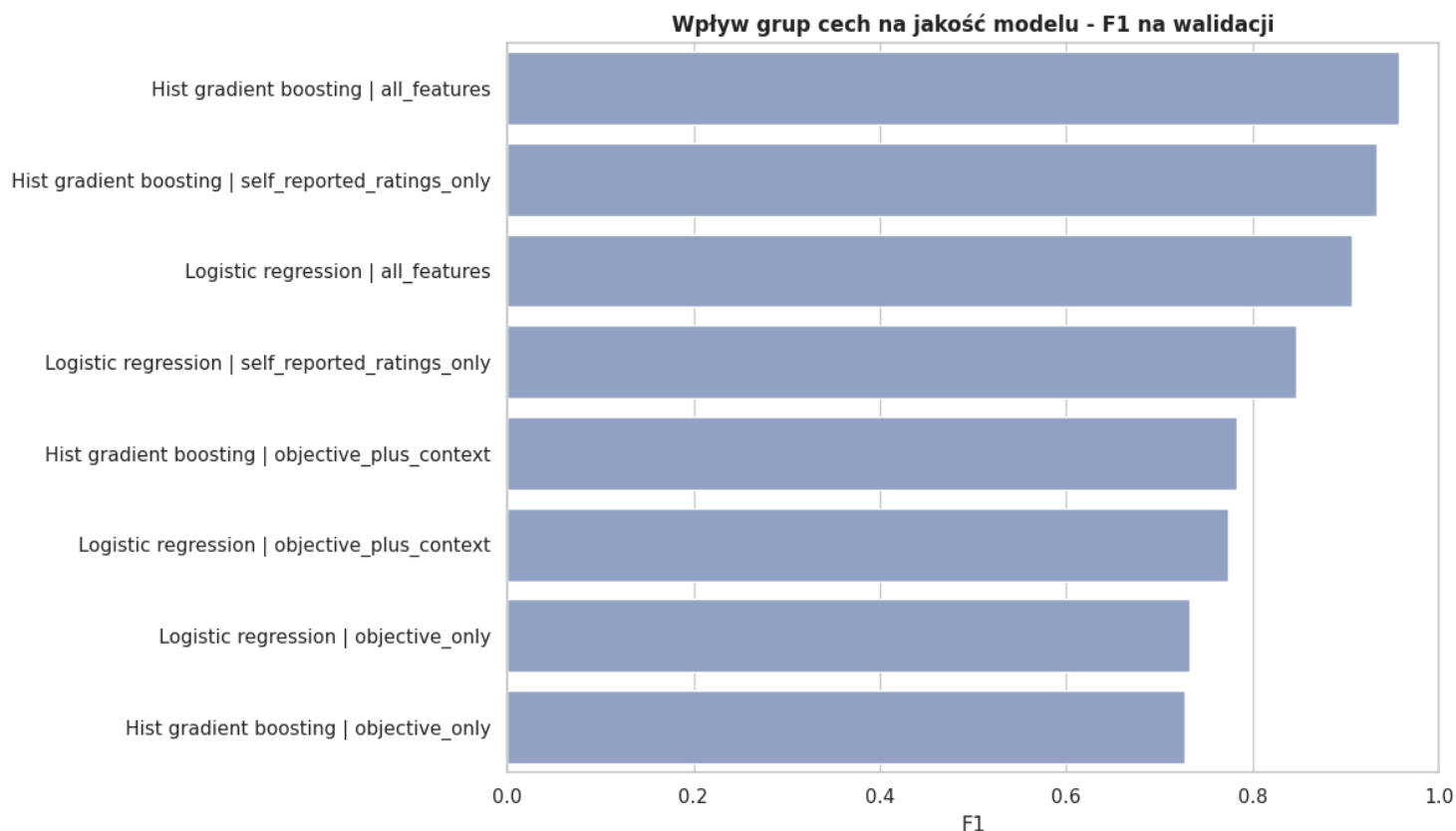
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
sns.heatmap(confusion_matrix(y_valid, y_valid_pred), annot=True, fmt="d", cmap="Blues",
ax=axes[0])
axes[0].set_title("Macierz pomyłek - walidacja")
axes[0].set_xlabel("Predykcja")
axes[0].set_ylabel("Rzeczywistość")

RocCurveDisplay.from_predictions(y_valid, y_valid_score, ax=axes[1])
axes[1].set_title("Krzywa ROC - walidacja")

PrecisionRecallDisplay.from_predictions(y_valid, y_valid_score, ax=axes[2])
axes[2].set_title("Krzywa Precision-Recall - walidacja")

plt.tight_layout()
plt.show()
```

9.22. Załącznik 22. Modele według grup cech



Rysunek 47: Porównanie modeli według grup cech.

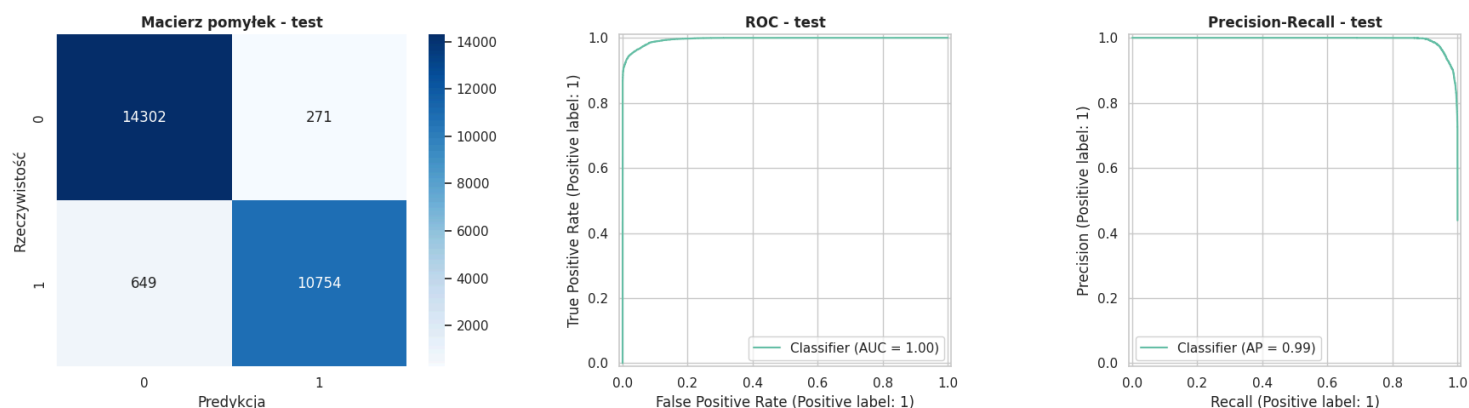
Wniosek: Eksperyment z grupami cech pokazuje, skąd pochodzi skuteczność predykcji. Same dane obiektywne dają zauważalny, ale znacznie słabszy wynik niż pełny zestaw cech. Największą wartość wnoszącą oceny usług, co oznacza, że model pełny jest bardzo dobry po ankiecie, ale model przedankietowy byłby wyraźnie trudniejszym zadaniem.

Kod generujący wykres:

```
feature_set_results = pd.DataFrame(feature_set_rows).sort_values(
    ["f1", "roc_auc"],
    ascending=False,
)
display(feature_set_results)

plt.figure(figsize=(12, 7))
sns.barplot(data=feature_set_results, y="model", x="f1", color="#8da0cb")
plt.title("Wpływ grup cech na jakość modelu - F1 na walidacji")
plt.xlabel("F1")
plt.ylabel("")
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```

9.23. Załącznik 23. Ewaluacja finalna na teście



Rysunek 48: Macierz pomyłek oraz krzywe ROC i Precision-Recall na zbiorze testowym.

Wniosek: Finalna ewaluacja potwierdza, że jakość modelu utrzymuje się na osobnym zbiorze testowym. Bardzo wysokie ROC-AUC oznacza dobrą separację klas, a wysoki F1 potwierdza dobrą równowagę precision i recall. Macierz pomyłek pokazuje, że model częściej traci część pasażerów zadowolonych niż błędnie oznacza niezadowolonych jako zadowolonych.

Kod generujący wykres:

```
y_test_pred = final_model.predict(X_test_final)
y_test_score = get_positive_scores(final_model, X_test_final)

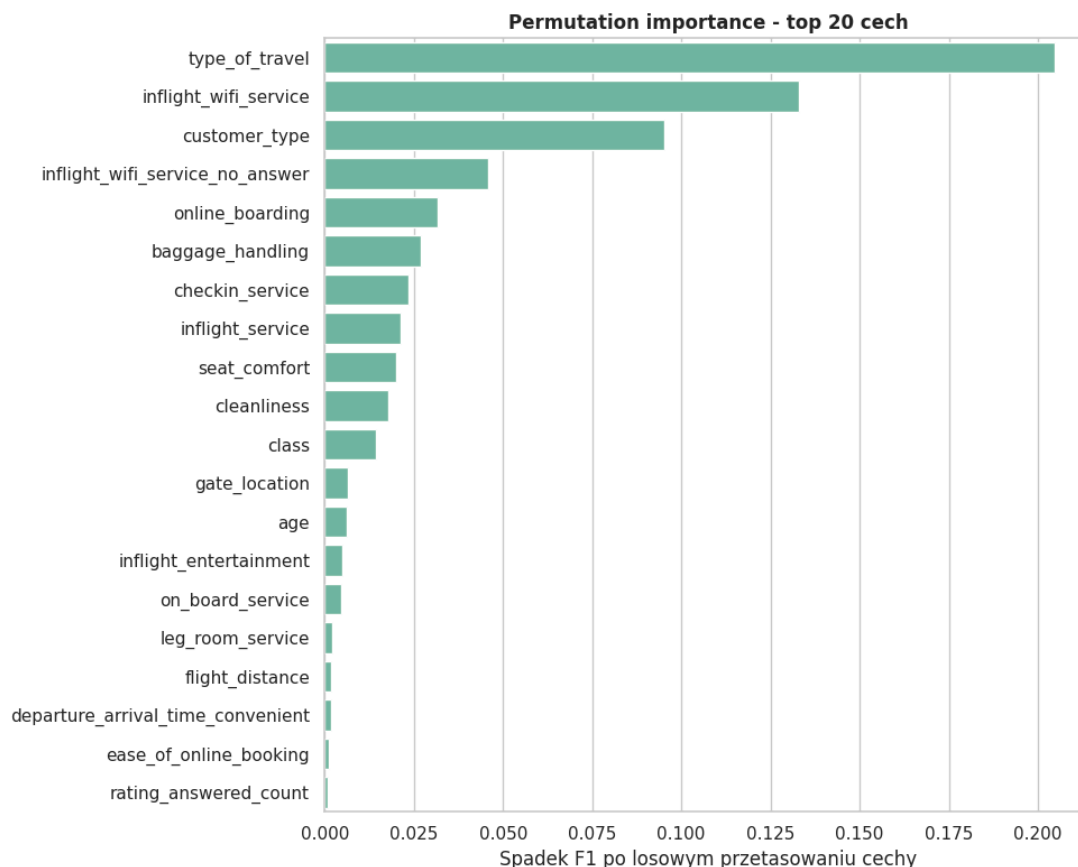
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
sns.heatmap(confusion_matrix(y_test_final, y_test_pred), annot=True, fmt="d",
            cmap="Blues", ax=axes[0])
axes[0].set_title("Macierz pomyłek - test")
axes[0].set_xlabel("Predykcja")
axes[0].set_ylabel("Rzeczywistość")

RocCurveDisplay.from_predictions(y_test_final, y_test_score, ax=axes[1])
axes[1].set_title("ROC - test")

PrecisionRecallDisplay.from_predictions(y_test_final, y_test_score, ax=axes[2])
axes[2].set_title("Precision-Recall - test")

plt.tight_layout()
plt.show()
```

9.24. Załącznik 24. Permutation importance



Rysunek 49: Permutation importance dla finalnego modelu.

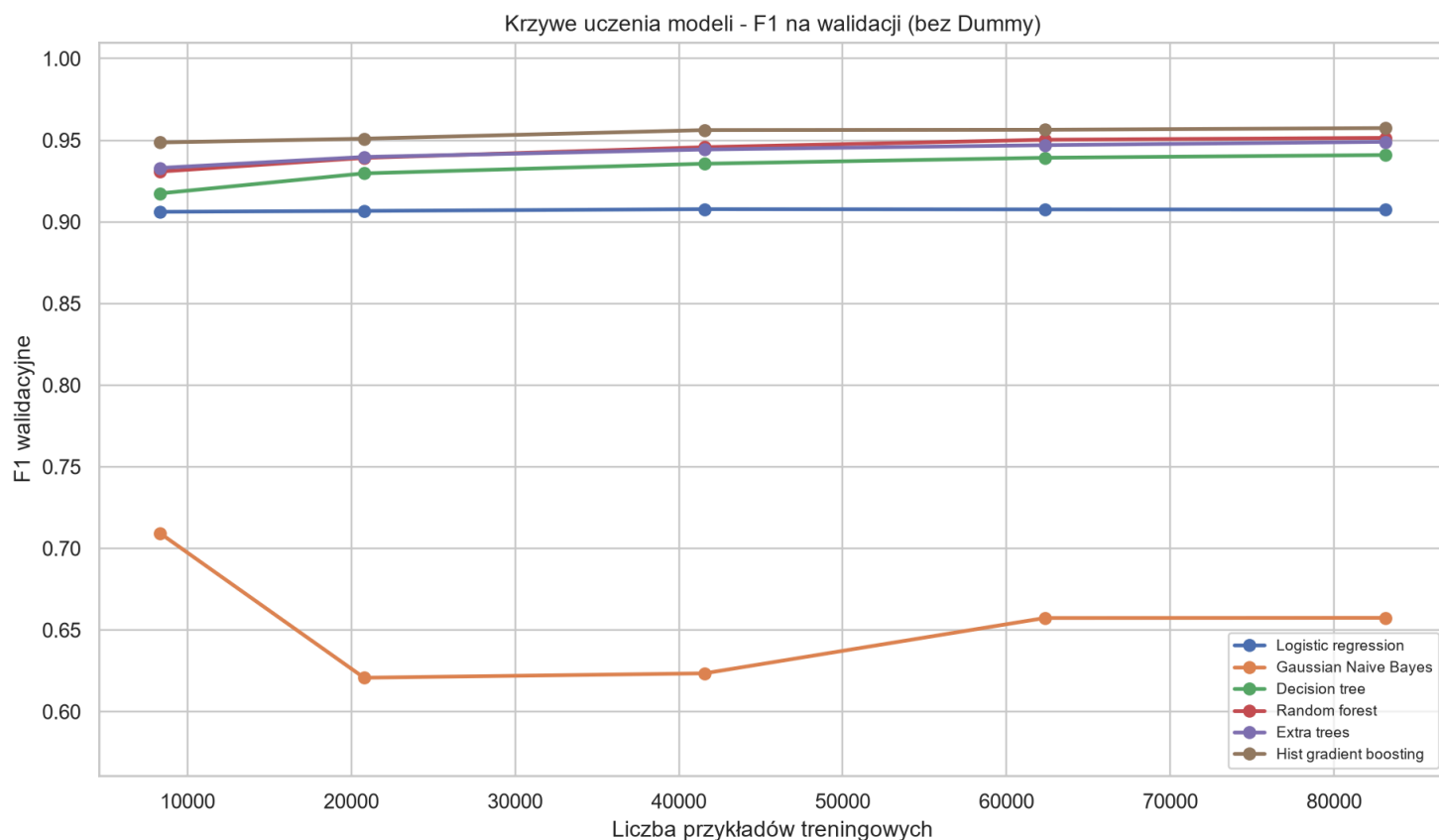
Wniosek: Permutation importance wskazuje, które cechy rzeczywiście pogarszają wynik po losowym przetasowaniu. Najważniejsze okazują się zarówno cechy kontekstowe, jak `type_of_travel`, jak i oceny usług, szczególnie Wi-Fi oraz boarding. Wysoka pozycja flagi braku odpowiedzi dla Wi-Fi potwierdza, że sposób obsługi wartości 0 był potrzebny.

Kod generujący wykres:

```
explain_model = final_model if "final_model" in globals() else best_pipe
X_importance_source = X_test_final if y_test_final is not None else X_valid
y_importance_source = y_test_final if y_test_final is not None else y_valid
importance_sample_size = min(8000, len(X_importance_source))
X_importance = X_importance_source.sample(n=importance_sample_size,
random_state=RANDOM_STATE)
y_importance = y_importance_source.loc[X_importance.index]

perm = permutation_importance(explain_model, X_importance, y_importance, n_repeats=8,
random_state=RANDOM_STATE, scoring="f1", n_jobs=-1,)
#...
plt.figure(figsize=(10, 8))
sns.barplot(data=perm_df.head(20), y="feature", x="importance_mean", color="#66c2a5")
plt.title("Permutation importance - top 20 cech")
plt.xlabel("Spadek F1 po losowym przetasowaniu cechy")
plt.ylabel("")
plt.tight_layout()
plt.show()
```

9.25. Załącznik 25. Krzywe uczenia - porównanie modeli



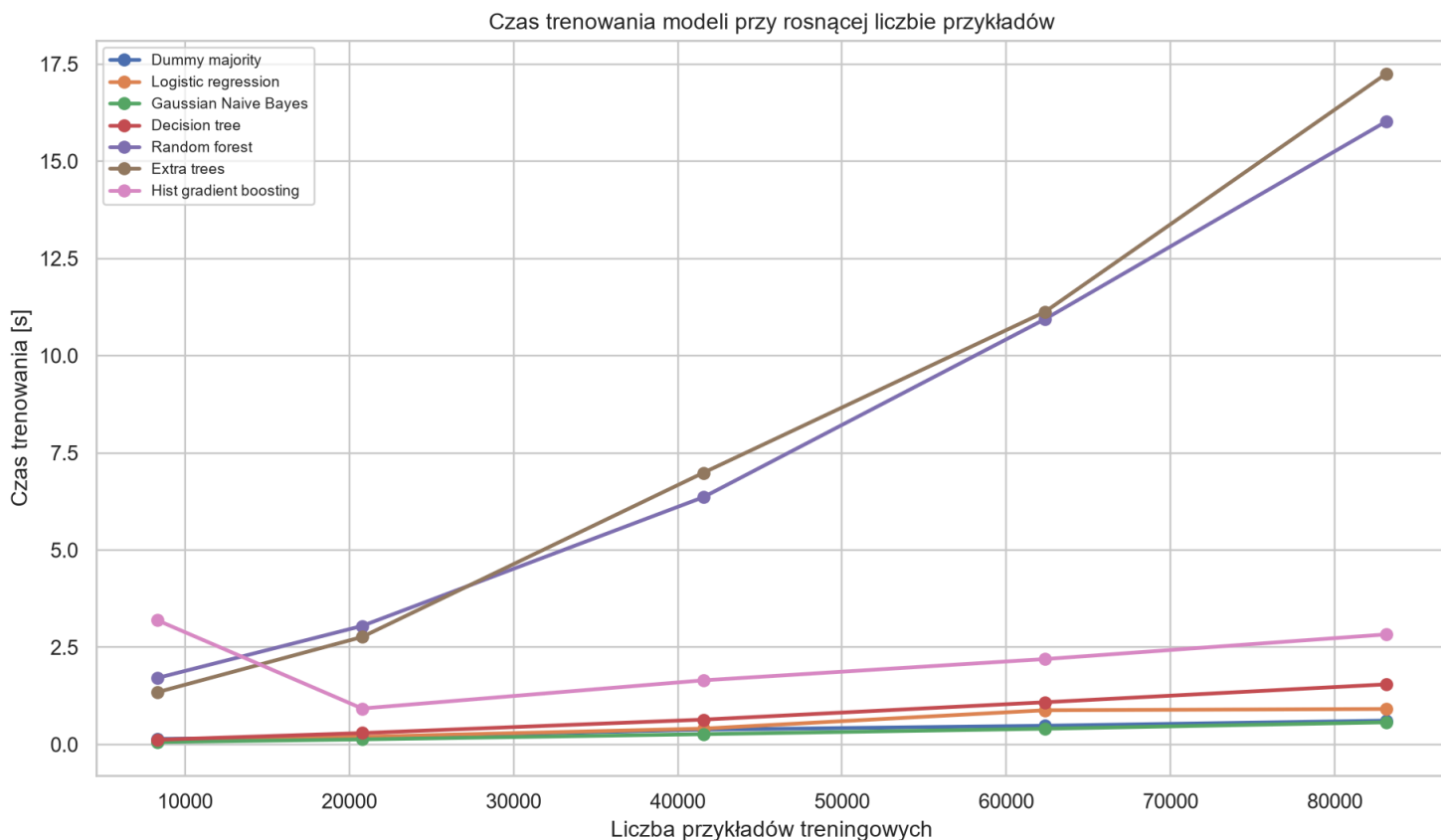
Rysunek 50: F1 walidacyjne przy rosnącej liczbie przykładów treningowych.

Wniosek: Krzywe uczenia pokazują, że najlepsze modele nieliniowe poprawiają wynik walidacyjny wraz ze wzrostem liczby danych. Histogram gradient boosting utrzymuje najwyższy poziom F1 w całym zakresie próbek, a modele lasów również zyskują na większej próbie. Regresja logistyczna jest stabilna, ale osiąga niższy pułap jakości.

Kod generujący wykres:

```
fig, ax = plt.subplots(figsize=(11, 6.5))
zoom_df = learning_curves_df[learning_curves_df["model"] != "Dummy majority"]
for model_name, model_df in zoom_df.groupby("model", sort=False):
    ax.plot(model_df["train_size"], model_df["valid_f1"], marker="o", linewidth=2,
            label=model_name)
ax.set_title("Krzywe uczenia modeli - F1 na walidacji (bez Dummy)")
ax.set_xlabel("Liczba przykładów treningowych")
ax.set_ylabel("F1 walidacyjne")
ax.set_ylim(max(0, zoom_df["valid_f1"].min() - 0.06), 1.01)
ax.legend(loc="lower right", fontsize=8)
plt.tight_layout()
plt.savefig(learning_asset_dir / "learning_curves_models_zoom.png", dpi=170,
            bbox_inches="tight")
plt.show()
```

9.26. Załącznik 26. Czas trenowania modeli



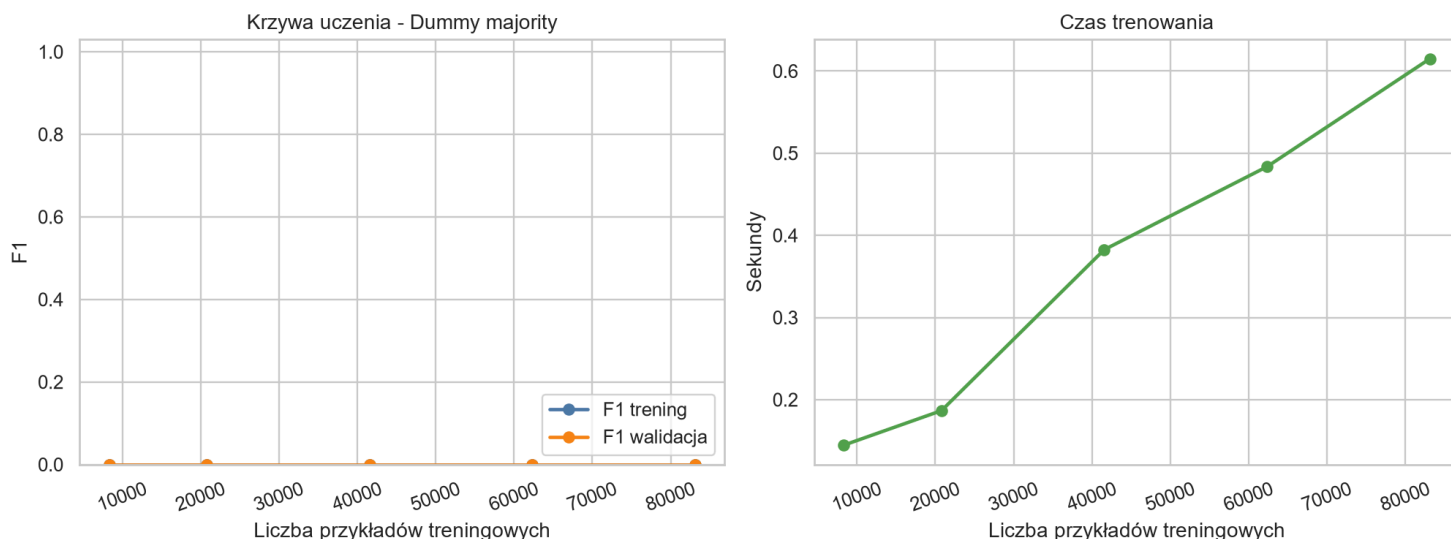
Rysunek 51: Czas trenowania przy rosnącej liczbie przykładów.

Wniosek: Czas trenowania rośnie szczególnie dla modeli zespołowych opartych na wielu drzewach, czyli Random Forest i Extra Trees. Histogram gradient boosting daje bardzo dobry wynik jakościowy przy relatywnie rozsądnym czasie, więc jest najlepszym kompromisem w tym projekcie. Pomiary czasu należy traktować orientacyjnie, bo zależą od środowiska uruchomieniowego.

Kod generujący wykres:

```
fig, ax = plt.subplots(figsize=(11, 6.5))
for model_name, model_df in learning_curves_df.groupby("model", sort=False):
    ax.plot(model_df["train_size"], model_df["fit_time_seconds"], marker="o", linewidth=2,
            label=model_name)
ax.set_title("Czas trenowania modeli przy rosnącej liczbie przykładów")
ax.set_xlabel("Liczba przykładów treningowych")
ax.set_ylabel("Czas trenowania [s]")
ax.legend(loc="upper left", fontsize=8)
plt.tight_layout()
plt.savefig(learning_asset_dir / "learning_times_models.png", dpi=170,
            bbox_inches="tight")
plt.show()
```

9.27. Załącznik 27. Krzywa uczenia - Dummy majority



Rysunek 52: Krzywa uczenia i czas trenowania modelu Dummy majority.

Wniosek: Dummy majority jest punktem odniesienia, a nie realnym modelem predykcyjnym. Ponieważ przewiduje wyłącznie klasę większościową, nie wykrywa klasy `satisfied`, więc F1 pozostaje równe 0 niezależnie od liczby danych. Dzięki temu dobrze pokazuje minimalny poziom, który sensowny model musi zdecydowanie przekroczyć.

Kod generujący wykres:

```
model_name = "Dummy majority"
model_df = learning_curves_df[learning_curves_df["model"] == model_name]

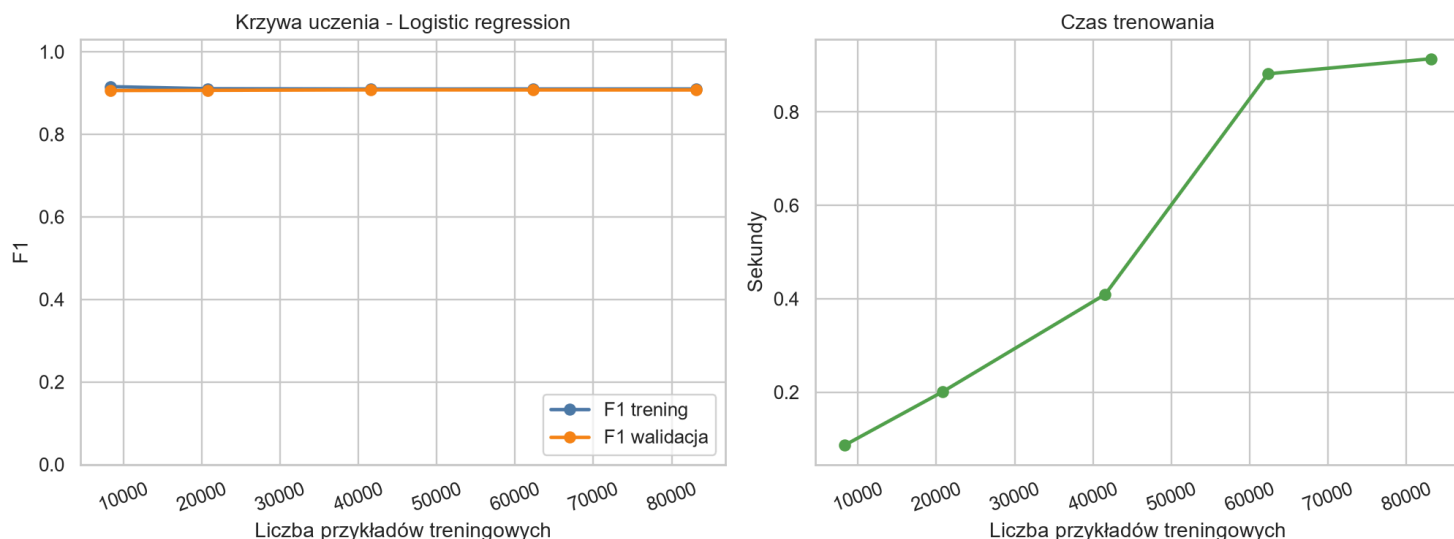
fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
axes[0].plot(model_df["train_size"], model_df["train_f1"], marker="o", linewidth=2,
label="F1 trening")
axes[0].plot(model_df["train_size"], model_df["valid_f1"], marker="o", linewidth=2,
label="F1 walidacja")
axes[0].set_title(f"Krzywa uczenia - {model_name}")
axes[0].set_xlabel("Liczba przykładów treningowych")
axes[0].set_ylabel("F1")
axes[0].set_ylim(0, 1.03)
axes[0].legend(loc="lower right")

axes[1].plot(model_df["train_size"], model_df["fit_time_seconds"], marker="o",
linewidth=2, color="#54a24b")
axes[1].set_title("Czas trenowania")
axes[1].set_xlabel("Liczba przykładów treningowych")
axes[1].set_ylabel("Sekundy")

for ax in axes:
    ax.tick_params(axis="x", rotation=20)

plt.tight_layout()
plt.savefig(learning_asset_dir / "learning_curve_dummy_majority.png", dpi=170,
bbox_inches="tight")
plt.show()
```

9.28. Załącznik 28. Krzywa uczenia - Logistic regression



Rysunek 53: Krzywa uczenia i czas trenowania regresji logistycznej.

Wniosek: Regresja logistyczna szybko osiąga stabilny wynik i prawie nie korzysta z dodatkowych danych po przekroczeniu pierwszej części próby. Oznacza to, że ograniczeniem jest raczej liniowa forma modelu niż brak danych. Model pozostaje użytecznym punktem odniesienia, bo jest szybki i interpretowalny, ale nie osiąga jakości modeli nieliniowych.

Kod generujący wykres:

```
model_name = "Logistic regression"
model_df = learning_curves_df[learning_curves_df["model"] == model_name]

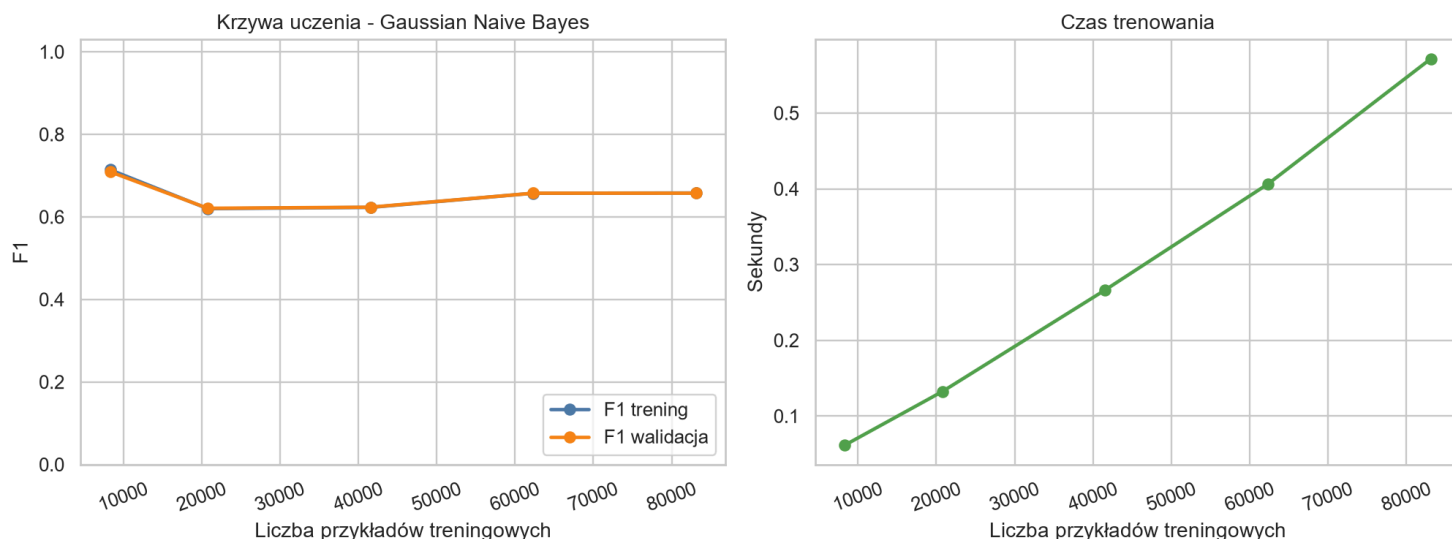
fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
axes[0].plot(model_df["train_size"], model_df["train_f1"], marker="o", linewidth=2,
label="F1 trening")
axes[0].plot(model_df["train_size"], model_df["valid_f1"], marker="o", linewidth=2,
label="F1 walidacja")
axes[0].set_title(f"Krzywa uczenia - {model_name}")
axes[0].set_xlabel("Liczba przykładów treningowych")
axes[0].set_ylabel("F1")
axes[0].set_ylim(0, 1.03)
axes[0].legend(loc="lower right")

axes[1].plot(model_df["train_size"], model_df["fit_time_seconds"], marker="o",
linewidth=2, color="#54a24b")
axes[1].set_title("Czas trenowania")
axes[1].set_xlabel("Liczba przykładów treningowych")
axes[1].set_ylabel("Sekundy")

for ax in axes:
    ax.tick_params(axis="x", rotation=20)

plt.tight_layout()
plt.savefig(learning_asset_dir / "learning_curve_logistic_regression.png", dpi=170,
bbox_inches="tight")
plt.show()
```

9.29. Załącznik 29. Krzywa uczenia - Gaussian Naive Bayes



Rysunek 54: Krzywa uczenia i czas trenowania Gaussian Naive Bayes.

Wniosek: Gaussian Naive Bayes jest bardzo szybki, ale jego wynik walidacyjny jest wyraźnie niższy i mniej stabilny. Model zakłada prostą strukturę zależności między cechami, co słabo pasuje do danych ankietowych, kategoriycznych i silnie powiązanych ocen usług. W projekcie pełni więc rolę porównawczą, a nie kandydata do modelu finalnego.

Kod generujący wykres:

```
model_name = "Gaussian Naive Bayes"
model_df = learning_curves_df[learning_curves_df["model"] == model_name]

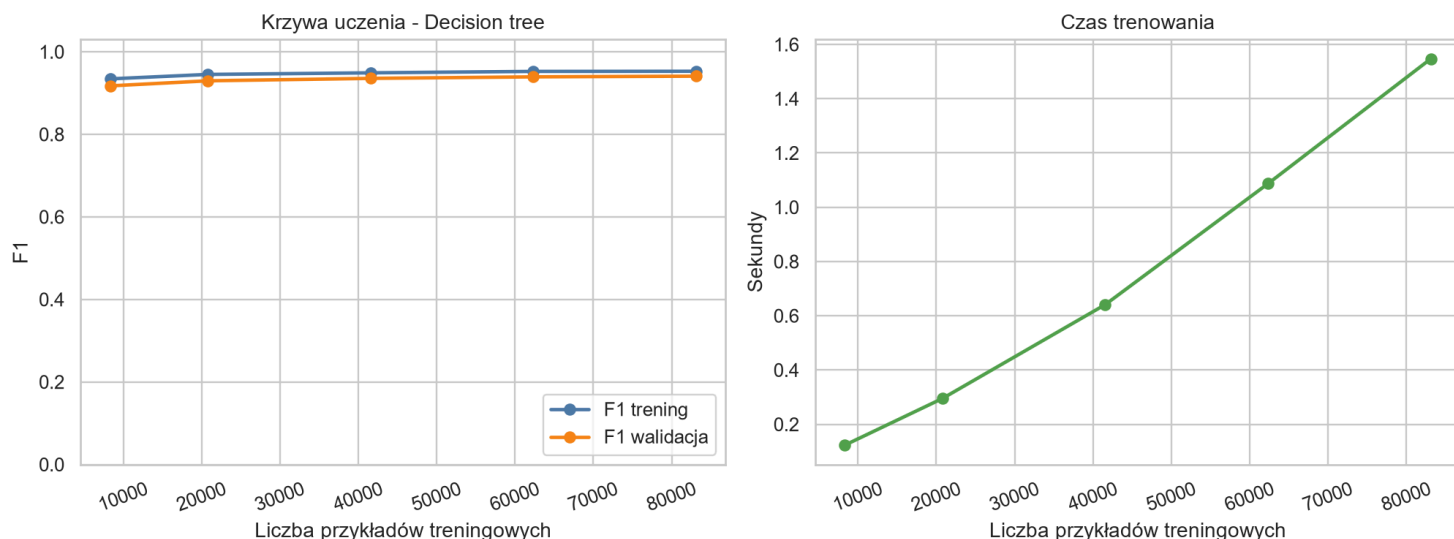
fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
axes[0].plot(model_df["train_size"], model_df["train_f1"], marker="o", linewidth=2,
label="F1 trening")
axes[0].plot(model_df["train_size"], model_df["valid_f1"], marker="o", linewidth=2,
label="F1 walidacja")
axes[0].set_title(f"Krzywa uczenia - {model_name}")
axes[0].set_xlabel("Liczba przykładów treningowych")
axes[0].set_ylabel("F1")
axes[0].set_ylim(0, 1.03)
axes[0].legend(loc="lower right")

axes[1].plot(model_df["train_size"], model_df["fit_time_seconds"], marker="o",
linewidth=2, color="#54a24b")
axes[1].set_title("Czas trenowania")
axes[1].set_xlabel("Liczba przykładów treningowych")
axes[1].set_ylabel("Sekundy")

for ax in axes:
    ax.tick_params(axis="x", rotation=20)

plt.tight_layout()
plt.savefig(learning_asset_dir / "learning_curve_gaussian_naive_bayes.png", dpi=170,
bbox_inches="tight")
plt.show()
```

9.30. Załącznik 30. Krzywa uczenia - Decision tree



Rysunek 55: Krzywa uczenia i czas trenowania drzewa decyzyjnego.

Wniosek: Drzewo decyzyjne poprawia wynik wraz ze wzrostem liczby przykładów, co pokazuje, że korzysta z większej próby. Nadal pozostaje słabsze od modeli zespołowych, ponieważ pojedyncze drzewo łatwiej dopasowuje się do specyficznych podziałów danych. Jest jednak ważne interpretacyjnie, bo pokazuje podstawowy mechanizm działania modeli drzewiastych.

Kod generujący wykres:

```
model_name = "Decision tree"
model_df = learning_curves_df[learning_curves_df["model"] == model_name]

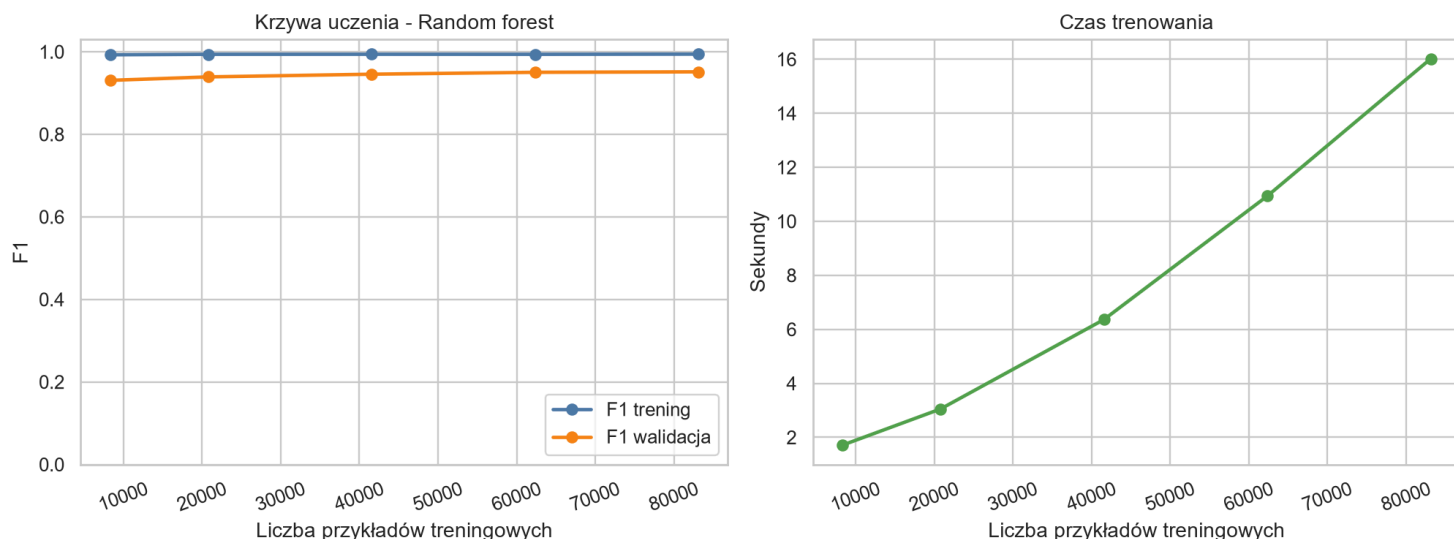
fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
axes[0].plot(model_df["train_size"], model_df["train_f1"], marker="o", linewidth=2,
label="F1 trening")
axes[0].plot(model_df["train_size"], model_df["valid_f1"], marker="o", linewidth=2,
label="F1 walidacja")
axes[0].set_title(f"Krzywa uczenia - {model_name}")
axes[0].set_xlabel("Liczba przykładów treningowych")
axes[0].set_ylabel("F1")
axes[0].set_ylim(0, 1.03)
axes[0].legend(loc="lower right")

axes[1].plot(model_df["train_size"], model_df["fit_time_seconds"], marker="o",
linewidth=2, color="#54a24b")
axes[1].set_title("Czas trenowania")
axes[1].set_xlabel("Liczba przykładów treningowych")
axes[1].set_ylabel("Sekundy")

for ax in axes:
    ax.tick_params(axis="x", rotation=20)

plt.tight_layout()
plt.savefig(learning_asset_dir / "learning_curve_decision_tree.png", dpi=170,
bbox_inches="tight")
plt.show()
```

9.31. Załącznik 31. Krzywa uczenia - Random forest



Rysunek 56: Krzywa uczenia i czas trenowania Random Forest.

Wniosek: Random Forest osiąga bardzo wysokie F1 treningowe, a wynik walidacyjny systematycznie rośnie wraz z liczbą danych. Różnica między treningiem a walidacją wskazuje, że model jest bardzo elastyczny i częściowo dopasowuje się do danych treningowych. Większa liczba przykładów pomaga mu generalizować, ale odbywa się to kosztem wyraźnie dłuższego trenowania.

Kod generujący wykres:

```
model_name = "Random forest"
model_df = learning_curves_df[learning_curves_df["model"] == model_name]

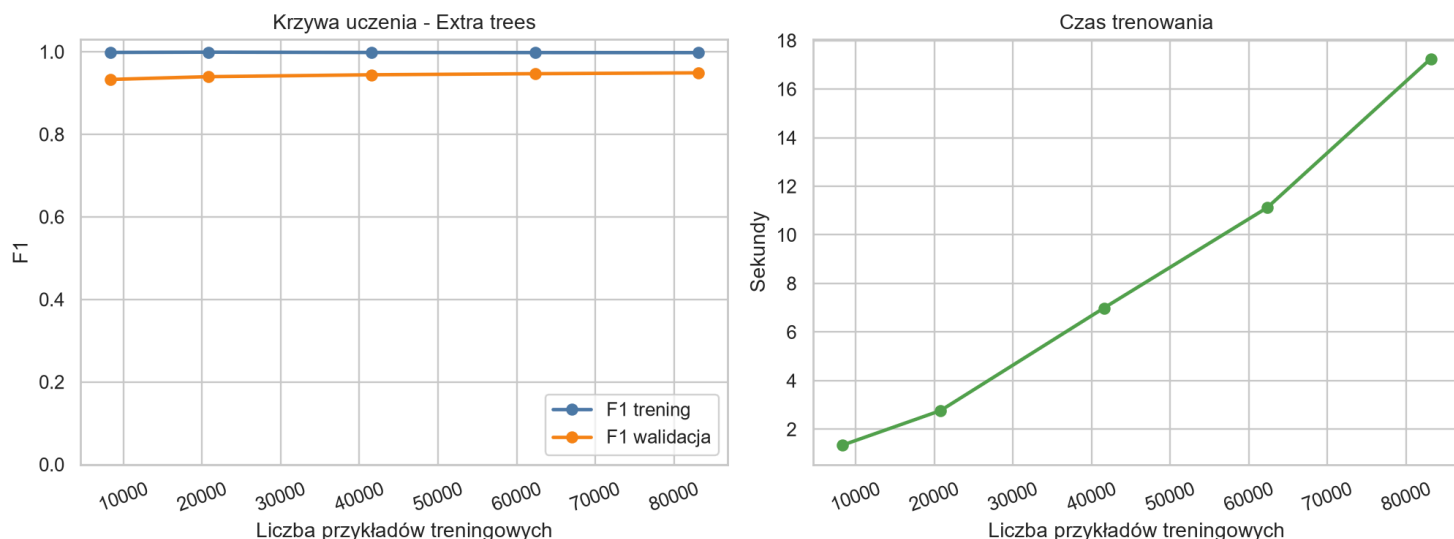
fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
axes[0].plot(model_df["train_size"], model_df["train_f1"], marker="o", linewidth=2,
label="F1 trening")
axes[0].plot(model_df["train_size"], model_df["valid_f1"], marker="o", linewidth=2,
label="F1 walidacja")
axes[0].set_title(f"Krzywa uczenia - {model_name}")
axes[0].set_xlabel("Liczba przykładów treningowych")
axes[0].set_ylabel("F1")
axes[0].set_ylim(0, 1.03)
axes[0].legend(loc="lower right")

axes[1].plot(model_df["train_size"], model_df["fit_time_seconds"], marker="o",
linewidth=2, color="#54a24b")
axes[1].set_title("Czas trenowania")
axes[1].set_xlabel("Liczba przykładów treningowych")
axes[1].set_ylabel("Sekundy")

for ax in axes:
    ax.tick_params(axis="x", rotation=20)

plt.tight_layout()
plt.savefig(learning_asset_dir / "learning_curve_random_forest.png", dpi=170,
bbox_inches="tight")
plt.show()
```

9.32. Załącznik 32. Krzywa uczenia - Extra trees



Rysunek 57: Krzywa uczenia i czas trenowania Extra Trees.

Wniosek: Extra Trees zachowuje się podobnie do Random Forest: wynik treningowy jest prawie maksymalny, a walidacyjny poprawia się wraz ze wzrostem próby. Losowość podziałów pomaga ograniczać część wariancji, ale model nadal jest obliczeniowo kosztowny. W projekcie daje bardzo dobry wynik, choć minimalnie ustępuje histogram gradient boosting.

Kod generujący wykres:

```
model_name = "Extra trees"
model_df = learning_curves_df[learning_curves_df["model"] == model_name]

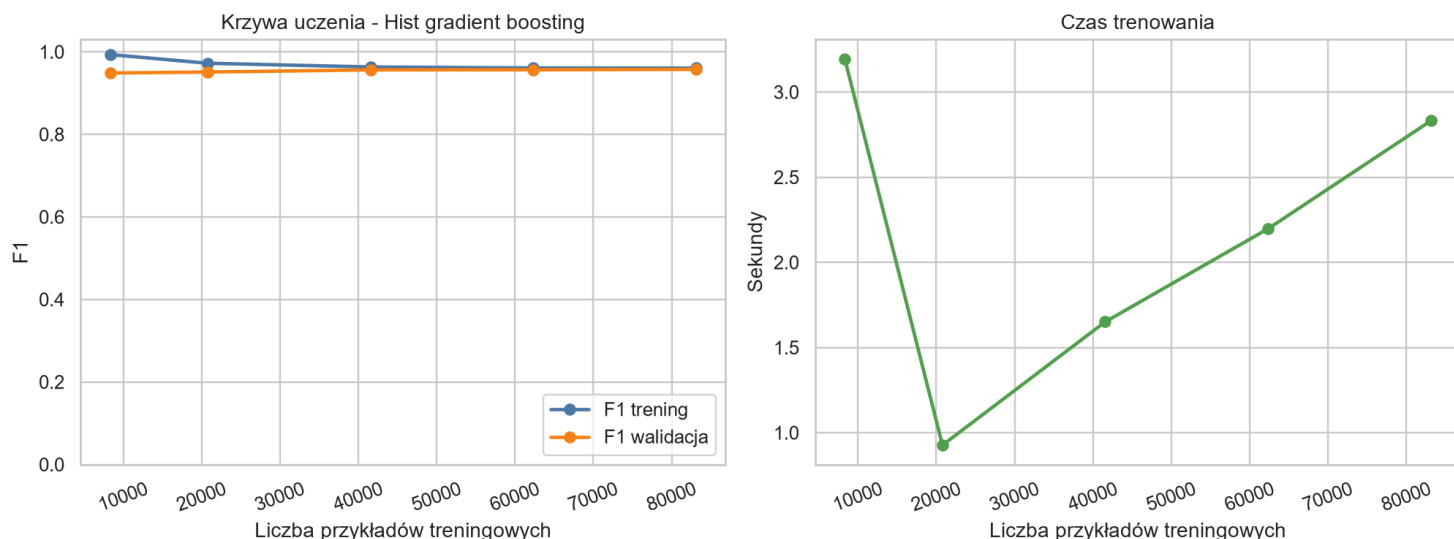
fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
axes[0].plot(model_df["train_size"], model_df["train_f1"], marker="o", linewidth=2,
label="F1 trening")
axes[0].plot(model_df["train_size"], model_df["valid_f1"], marker="o", linewidth=2,
label="F1 walidacja")
axes[0].set_title(f"Krzywa uczenia - {model_name}")
axes[0].set_xlabel("Liczba przykładów treningowych")
axes[0].set_ylabel("F1")
axes[0].set_ylim(0, 1.03)
axes[0].legend(loc="lower right")

axes[1].plot(model_df["train_size"], model_df["fit_time_seconds"], marker="o",
linewidth=2, color="#54a24b")
axes[1].set_title("Czas trenowania")
axes[1].set_xlabel("Liczba przykładów treningowych")
axes[1].set_ylabel("Sekundy")

for ax in axes:
    ax.tick_params(axis="x", rotation=20)

plt.tight_layout()
plt.savefig(learning_asset_dir / "learning_curve_extra_trees.png", dpi=170,
bbox_inches="tight")
plt.show()
```

9.33. Załącznik 33. Krzywa uczenia - Hist gradient boosting



Rysunek 58: Krzywa uczenia i czas trenowania Histogram Gradient Boosting.

Wniosek: Histogram gradient boosting osiąga najlepsze F1 walidacyjne i bardzo dobrze wykorzystuje dodatkowe dane. Wraz ze wzrostem próby maleje różnica między F1 treningowym i walidacyjnym, co sugeruje lepszą generalizację. Ten wykres wspiera wybór boostingu jako modelu finalnego, bo łączy najwyższą jakość z rozsądnym kosztem trenowania.

Kod generujący wykres:

```
model_name = "Hist gradient boosting"
model_df = learning_curves_df[learning_curves_df["model"] == model_name]

fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
axes[0].plot(model_df["train_size"], model_df["train_f1"], marker="o", linewidth=2,
label="F1 trening")
axes[0].plot(model_df["train_size"], model_df["valid_f1"], marker="o", linewidth=2,
label="F1 walidacja")
axes[0].set_title(f"Krzywa uczenia - {model_name}")
axes[0].set_xlabel("Liczba przykładów treningowych")
axes[0].set_ylabel("F1")
axes[0].set_ylim(0, 1.03)
axes[0].legend(loc="lower right")

axes[1].plot(model_df["train_size"], model_df["fit_time_seconds"], marker="o",
linewidth=2, color="#54a24b")
axes[1].set_title("Czas trenowania")
axes[1].set_xlabel("Liczba przykładów treningowych")
axes[1].set_ylabel("Sekundy")

for ax in axes:
    ax.tick_params(axis="x", rotation=20)

plt.tight_layout()
plt.savefig(learning_asset_dir / "learning_curve_hist_gradient_boosting.png", dpi=170,
bbox_inches="tight")
plt.show()
```

10. Bibliografia i źródła

1. Kaggle, *Airline Passenger Satisfaction*, zbiór danych wskazany w wymaganiach projektu.
2. Materiały projektowe: Projekt końcowy - założenia.pdf.
3. Materiały projektowe: Projekt - struktura raportu (1).pdf.
4. Notebook projektowy: PUM_PROJ_(1).ipynb.
5. Dokumentacja scikit-learn: klasyfikatory, pipeline, imputacja, preprocessing, walidacja krzyżowa i permutation importance.